

**Machine-Checked Verification of the
Formal
Provable-Security Model of the HAETAE
Post-Quantum Signature Scheme**

Dissertation-Style Technical Monograph
for the Local Scoped ROM Security Verification Record

Local HAETAE EasyCrypt Verification Project

Prepared for cryptographic proof review
Formal-model scope, trusted bridges, and residual assumptions stated
explicitly

Review status:

Compiled technical monograph for local proof audit

Not an institutional submission copy

2026

This dissertation documents a local EasyCrypt development under
`haetae-security/provable-security/`.

Abstract

This dissertation studies the machine-checked verification of the formal provable-security model of the lattice-based signature scheme HAETAE. The verification is performed in **EasyCrypt** and targets the formal, reduction-style random-oracle security argument rather than implementation certification artifacts or a standard-model proof for SHAKE. The central machine-checked statement is a scoped **EUF-CMA** implication for the modeled scheme in the random oracle model, with the counted-ROM loss ledger instantiated at the checked wrapper constants and under the explicit hop and hardness premises of the final **EasyCrypt** composition lemma. The current dissertation also records a vacuity warning: one final composition premise contains **rejection_sampling_loss_term**, which is larger than one under the present constants, so the checked implication should not be read as a non-vacuous concrete security instantiation until that antecedent or loss normalization is repaired. It is not claimed as a checked equivalence theorem between the February 2026 HAETAE specification and the **EasyCrypt** model. The proof surface is supported by formal modules for signature games, lazy random oracles, transcript games, signing simulators, paper-simulation samplers, exact hyperball sampling, rejection sampling, special soundness, and concrete loss accounting.

A major practical contribution is the creation of a separated proof-management surface under **provable-security/**. The manifest **provable-security/proof-files.txt** lists the **EasyCrypt** files that define the provable-security gate, and **provable-security/easycrypt/** contains the corresponding source copies. The verification script compiles this separated folder with **EasyCrypt** and excludes generated known-answer-test material such as **HAETAE_FIPS202_KAT_Empty_Generated.ec**, which is useful for implementation validation but not necessary for the reduction proof.

The dissertation explains the theorem architecture from the generic signature ROM interface in **Sig_ROM.ec** through the HAETAE-specific theorem layer in

`HAETAE_Security.ec`. It gives particular attention to the proof-engineering obligations that are easy to hide in informal cryptographic papers: random-oracle programming freshness, adversarial query logging, `sampler_bad_prequery`, clean-conditioned lifting through signing oracles, exact hyperball sampling, rejection-sampling losses, and the algebra of final concrete bounds. It also records a dependency layer separating checked EasyCrypt theorems, explicit modeling assumptions, audited specification correspondences, vacuity checks, and out-of-scope cryptanalytic or implementation claims. The resulting document is both a dissertation-style explanation of the checked formal-model proof surface and a guide to maintaining it. The model/specification correspondence, non-vacuous discharge of the final hop premise, concrete hash instantiation, implementation correctness, concrete MLWE/Module-SIS security estimates, and reduction runtime overhead are identified as explicit boundary obligations rather than hidden theorem claims.

Acknowledgements

This dissertation-style monograph documents the local machine-checked HAETAE provable-security proof surface. It is written for technical proof audit rather than as an institutional submission copy.

The proof development described here depends on the EasyCrypt ecosystem, Why3, automated SMT provers, and prior work on game-based cryptographic verification, Fiat-Shamir proofs, and lattice-based signatures.

Contents

Abstract	iii
Acknowledgements	v
1 Introduction	1
1.1 Problem Statement	2
1.2 Contributions	2
1.3 Reading Guide	3
2 Background	5
2.1 Signature Correctness and EUF-CMA Security	5
2.2 The Random Oracle Model	6
2.3 Fiat-Shamir with Aborts	7
2.4 Lattice Assumptions and Reduction Targets	8
2.5 EasyCrypt	9
3 Proof Scope and Repository Structure	11
3.1 Separated Dissertation and Proof Folders	11
3.2 Supplemental Material	12
3.3 Proof Manifest	12
3.4 EasyCrypt File Relationships	12

3.5	Theorem Dependency Classes	14
3.6	Excluded Generated KAT Artifact	15
3.7	ROM-to-FIPS202 Boundary	16
4	Model–Specification Correspondence	19
4.1	Signature-scheme interface	19
4.2	Algorithm correspondence	21
4.3	Exact scope statement	24
4.4	Internal-algorithm fidelity audit ledger	26
4.5	Parameter correspondence	28
4.6	Hash and random-oracle correspondence	29
4.7	Residual model/spec obligations	30
5	EasyCrypt Framework	33
5.1	Modules and Adversaries	33
5.2	Probabilistic Judgments	34
5.3	Fundamental Lemma of Game Playing	34
5.4	Proof Style	35
6	Signature Games, Theorem Chain, and Paper Proof	37
6.1	Generic ROM Signature Interface	37
6.2	Game-Hopping Calculus	37
6.3	Pen-and-Paper EUF-CMA Game	38
6.4	Machine-Checked Theorem Chain	40
6.5	Detailed EasyCrypt Code Analysis	40
7	Random Oracle Programming: Paper Proof and Code Analysis	43

7.1	Lazy ROM State	43
7.2	Pen-and-Paper Programming Lemma	44
7.3	Programming Sites	44
7.4	Detailed EasyCrypt Code Analysis	45
7.5	Bad Events and Fundamental Lemma Use	45
8	Sampler Freshness and Exact Hyperball Sampling	47
8.1	Sampler Interfaces	47
8.2	Pen-and-Paper Freshness Bound	47
8.3	The sampler_bad_prequery Event	48
8.4	Clean-Conditioned Lifting	48
8.5	Detailed EasyCrypt Code Analysis	48
9	Transcript Games and Erasure	51
9.1	Transcript Simulation	51
9.2	State Preservation	51
9.3	Erasure Exactness	51
10	Budgeted Lifting Through Oracles	53
10.1	Adversarial Query Budgets	53
10.2	Pen-and-Paper Fundamental-Lemma Lifting	53
10.3	One-Call Signing Loss	54
10.4	Detailed EasyCrypt Code Analysis	54
10.5	From One Call to NMA Wrapper Obligations	54
11	Reductions, Concrete Bounds, and Paper Algebra	57
11.1	Reduction Terms	57

11.2	Pen-and-Paper Bound Composition	57
11.3	Nonnegativity and Cardinality	58
11.4	Detailed EasyCrypt Code Analysis	58
11.5	Hardness Boundary	59
12	Pen-and-Paper Security Proof Reconstructed from EasyCrypt	61
12.1	Proof convention	61
12.2	Correctness and the starting game	63
12.3	Removing the direct signing view	63
12.4	Internal transcript and random-oracle view	64
12.5	ROM programming and counted bad events	65
12.6	From chosen-message to no-message structure	65
12.7	Rejection sampling and the paper sampler	66
12.8	Exact hyperball sampling	67
12.9	Tight RO-signing-attempt to RO-exact-hyperball lifting	67
12.10	Forgery extraction and hardness reduction	68
12.11	Composing the inequalities	69
12.12	What this reconstruction does and does not claim	70
13	Mathematical and Cryptological Synthesis	71
13.1	Formal security object	71
13.2	Exact checked bound expression	72
13.3	Hardness-term classification	77
13.4	Denotational reading of the proof scripts	79
13.5	Local clean-conditioned evidence and NMA-lifting gap	80
13.6	Wrapper counts and runtime cost	82

13.7 Trusted base	83
14 Reproducibility and Evaluation	85
14.1 Build and Proof Commands	85
14.2 Why Attempts Are Configurable	85
14.3 Evaluation Criteria	86
15 Conclusion	87
A Proof Manifest	91
B Detailed EasyCrypt Proof-Code Analysis	93
B.1 Manifest-level proof-code inventory	93
B.2 Trust boundary and checked claims	94
B.3 File-by-file analysis	95
B.3.1 HAETAE_Security.ec	95
B.3.2 Sig_ROM.ec	96
B.3.3 HAETAE_HopGames.ec	96
B.3.4 HAETAE_Reductions.ec	98
B.3.5 HAETAE_ROM.ec	98
B.3.6 HAETAE_ROM_Programming.ec	99
B.3.7 HAETAE_Scheme.ec	99
B.3.8 HAETAE_Transcript.ec	99
B.3.9 HAETAE_Events.ec	100
B.3.10 HAETAE_Assumptions.ec	100
B.3.11 HAETAE_Distributions.ec	101
B.3.12 HAETAE_Rejection.ec	101

B.3.13	HAETAE_Algebra.ec	102
B.3.14	HAETAE_Params.ec	103
B.3.15	HAETAE_FIPS202.ec	103
B.3.16	HAETAE_Keccak1600.ec	104
B.4	Cross-file dependency analysis	104
B.5	Residual proof obligations outside EasyCrypt	105
C	Mathematical Correspondence of the EasyCrypt Proof Objects	107
C.1	Global interpretation principle	107
C.2	Types as carrier sets	108
C.3	Operations as functions, predicates, distributions, and bounds	110
C.4	Modules as games, algorithms, oracles, and reductions	112
C.5	Lemmas as propositions, invariants, equivalences, and inequalities	113
C.6	Hoare and equivalence judgments as program-logic theorems	115
C.7	File-wise mathematical correspondence	117
C.8	Mathematical reading of the final proof DAG	119
D	Formal Mathematical Semantics of the EasyCrypt Security Proof Surface	121
D.1	Judgmental setting	121
D.2	Types	122
D.3	Operations	123
D.4	Modules and module types	124
D.5	Lemmas	125
D.6	Hoare, probabilistic Hoare, and equivalence judgments	127
D.7	Exact correspondence of proof-object classes	129

E	Extracted EasyCrypt Declaration Index	131
F	Curated EasyCrypt Proof-Object Case Studies	167
F.1	The generic signature interface	167
F.2	The HAETAE scheme wrapper	168
F.3	Parameters as mathematical constants	169
F.4	The counted-ROM bound	170
F.5	Lazy-ROM freshness	171
F.6	Active signing and clean lifting	171
F.7	Hoare and equivalence judgments as proof objects	173
F.8	Why these cases are representative	173
	Bibliography	175

List of Figures

- 3.1 Manifest-level dependency graph for the HAETAE EasyCrypt proof.
An arrow points from an importing file to a directly imported manifest
file; the darker arrows highlight the main top-level assembly paths. . . 17

List of Tables

3.1	Immediate manifest-file relationships induced by <code>EasyCrypt</code> imports.	13
3.2	Dependency classes used for theorem-scope statements.	14
4.1	Signature-interface correspondence.	20
4.2	Algorithm-level model/spec correspondence.	21
4.3	Internal-algorithm correspondence obligations for reading <code>ECModel</code> as the HAETAE specification.	27
4.4	Parameter correspondence between the specification and <code>EasyCrypt</code> .	28
4.5	Model/spec obligations outside the central game proof.	30
13.1	Final counted-ROM bound terms.	77
B.1	Structural summary of the manifest <code>EasyCrypt</code> files.	94
C.1	Mathematical reading of all manifest type declarations.	109
C.2	Mathematical reading of all manifest operation declarations.	110
C.3	Mathematical reading of all manifest module declarations.	112
C.4	Mathematical reading of all manifest lemma declarations.	114
C.5	Mathematical reading of all manifest Hoare/equivalence proof commands.	116
C.6	Per-file mathematical correspondence of proof-object classes.	117
D.1	Exact mathematical correspondence for proof-object classes.	129

Listings

13.1	Checked bound definitions in <code>HAETAE_Reductions.ec</code>	72
13.2	Concrete lazy-ROM sampler freshness laws.	80
13.3	Sampler-ROM coverage preservation for one active signing call.	81
F.1	Signature objects and ROM-query objects in <code>Sig_ROM.ec</code>	167
F.2	Scheme and adversary interfaces in <code>Sig_ROM.ec</code>	168
F.3	Signing wrapper in <code>HAETAE_Scheme.ec</code>	168
F.4	Representative parameter declarations in <code>HAETAE_Params.ec</code>	169
F.5	Final counted-ROM bound expression.	170
F.6	Concrete freshness lemma for a sampler ROM point.	171
F.7	Clean sampler-ROM preservation across one active signing call.	171

Chapter 1

Introduction

Digital signatures are a basic mechanism for public-key authentication. A signature scheme allows a signer with a secret key to authenticate messages so that anyone with the corresponding public key can verify them. The standard adaptive security notion for signatures is existential unforgeability under chosen-message attack, or EUF-CMA. In this notion an adversary receives a public key, adaptively queries a signing oracle, and wins if it outputs a valid signature on a message that the signing oracle did not sign.

The post-quantum transition makes the assurance problem more difficult. Modern lattice-based signatures combine algebraic structure, randomized sampling, Fiat-Shamir transformations, rejection sampling, and concrete hash functions. Their proofs involve many games and many small probability losses. A missing freshness side condition or an incorrectly counted oracle query can invalidate a security claim even when the high-level proof idea is correct.

This dissertation documents a machine-checked provable-security proof surface for a formal HAETAE random-oracle model in **EasyCrypt**. The qualifier “formal model” is essential: the checked surface is a conditional implication about the EasyCrypt game model and its counted random-oracle interface, not a separate machine-checked equivalence proof for every line of the February 2026 HAETAE specification or for a concrete SHAKE implementation. It is “machine checked” because the games, modules, lemmas, and inequalities are accepted by EasyCrypt rather than only argued in prose. Its current cryptographic reading must remain conditional: the final counted-ROM conclusion depends on explicit hop and hardness premises, and one final hop premise is vacuous under the present `rejection_sampling_loss_term` constants unless that premise or loss normalization is strengthened in future proof

work.

1.1 Problem Statement

The problem addressed by this work is the following:

How can the random-oracle-model proof of the formal HAETAE scheme’s EUF-CMA security be represented, checked, maintained, and audited as an EasyCrypt development whose provable-security surface is clearly separated from unrelated generated certification artifacts, while the model/specification boundary remains explicit?

This formulation has two parts. The first is mathematical: the proof must connect the EUF-CMA game to the underlying assumptions through valid game hops and reductions. The second is engineering-oriented: the proof repository must make it clear which EasyCrypt files are part of the security theorem and how to re-run the proof.

1.2 Contributions

The dissertation records six contributions.

- C1.** A separated EasyCrypt proof-management folder for HAETAE provable security under `provable-security/`.
- C2.** A manifest-driven verification gate that compiles only the files needed for the formal security proof.
- C3.** A structured account of the theorem chain from generic signature games to HAETAE-specific EUF-CMA security.
- C4.** A detailed explanation of checked random-oracle programming, sampler freshness, budgeted lifting, rejection sampling, and concrete bound algebra.
- C5.** A dependency ledger separating checked theorem content from modeling assumptions, audited specification correspondences, and out-of-scope cryptanalytic or implementation claims.

- C6.** A clear boundary between provable-security artifacts, modeled hash interfaces, concrete FIPS202/SHAKE validation material, and generated known-answer-test files.

1.3 Reading Guide

Chapter 2 reviews the cryptographic and formal background. Chapter 3 defines the checked proof surface. Later chapters follow the proof architecture: EasyCrypt framework, signature games, ROM programming, sampler freshness, transcript hops, budgeted lifting, reductions, and reproducibility.

Chapter 2

Background

2.1 Signature Correctness and EUF-CMA Security

A signature scheme is a triple of probabilistic algorithms

$$(\text{KeyGen}, \text{Sign}, \text{Verify}).$$

At security parameter λ , key generation samples $(pk, sk) \leftarrow \text{KeyGen}(1^\lambda)$, signing samples $\sigma \leftarrow \text{Sign}(sk, m, ctx)$, and verification returns a bit $\text{Verify}(pk, m, ctx, \sigma) \in \{0, 1\}$. In the HAETA development the security parameter is represented indirectly by a fixed parameter mode and by the symbolic hardness and loss terms attached to that mode, but the mathematical reading is the usual asymptotic one.

Correctness is the assertion that honestly signed messages verify except with negligible probability:

$$\Pr[\text{Verify}(pk, m, ctx, \sigma) = 1 \mid (pk, sk) \leftarrow \text{KeyGen}, \sigma \leftarrow \text{Sign}(sk, m, ctx)] \geq 1 - \varepsilon_{\text{corr}}.$$

Perfect correctness is the special case $\varepsilon_{\text{corr}} = 0$. For EUF-CMA security, an adversary receives pk , random-oracle access, and adaptive signing-oracle access. If Q_S is the set of signed pairs (m, ctx) , its success event is

$$\text{Forge}(A) \equiv \text{Verify}(pk, m^*, ctx^*, \sigma^*) = 1 \wedge (m^*, ctx^*) \notin Q_S.$$

The advantage is

$$\text{Adv}_{\Pi}^{\text{EUF-CMA}}(A) = \Pr[\text{Forge}(A)].$$

The proof objective is not to show that this probability is zero, but to bound it by hardness terms and explicitly counted statistical losses. A reductionist statement has the shape

$$\text{Adv}_{\Pi}^{\text{EUF-CMA}}(A) \leq \sum_i \text{Adv}_{\mathcal{P}_i}(B_i) + \sum_j L_j(q_H, q_S, \lambda),$$

where, in a conventional paper proof, the B_i are adversaries constructed against underlying mathematical problems and the L_j are loss terms arising from collisions, prequeries, sampling changes, and rejection events. In the final EasyCrypt theorem checked in this dissertation, the corresponding MLWE and Module-SIS solvers are declared reduction modules supplied through explicit antecedents rather than constructed inside the top-level composition lemma.

The checked development models these notions in `Sig_ROM.ec`. This file contains the generic interfaces for random oracles, schemes, signing oracles, chosen-message adversaries, and no-message adversaries. Keeping these games generic is important: it prevents HAETAE-specific proof assumptions from being smuggled into the security definition.

2.2 The Random Oracle Model

The proof is a random-oracle-model proof. A random oracle is an ideal sampled function

$$H : \mathcal{X} \rightarrow \mathcal{Y}$$

such that the values $H(x)$ are independent samples from the prescribed output distribution, subject only to consistency on repeated inputs. In a lazy implementation, the oracle maintains a finite partial map

$$T : \mathcal{X} \rightharpoonup \mathcal{Y}.$$

On query x , the oracle returns $T(x)$ if $x \in \text{dom}(T)$; otherwise it samples $y \leftarrow D_x$, stores $T[x \mapsto y]$, and returns y . This lazy semantics is distributionally equivalent to sampling the whole function in advance, but it exposes the finite set of adversarially observed queries.

ROM proofs often program the oracle: a simulator chooses a point x_0 and sets $H(x_0)$ to a value needed by the reduction. Programming is sound only when x_0 is fresh:

$$x_0 \notin Q_H,$$

where Q_H is the adversary's prior query set. If x_0 has min-entropy at least $\log_2 N$ against the adversary's view and $|Q_H| \leq q_H$, then

$$\Pr[x_0 \in Q_H] \leq q_H/N.$$

This inequality is the mathematical source of many counted ROM losses. The machine-checked development does not allow this argument to remain implicit: the query log, budget counter, freshness predicate, and bad event are explicit program state.

In the machine-checked proof, the random oracle is lazy and stateful. Its map, query log, bad flags, and counters are explicit program variables. This makes freshness and query-budget arguments checkable.

2.3 Fiat-Shamir with Aborts

HAETAE follows the general pattern of Fiat-Shamir signatures with rejection sampling. The abstract pattern starts from an identification protocol with a commitment, a challenge, and a response. The Fiat-Shamir transform replaces the verifier's challenge by a hash value

$$c = H(\text{commitment}, m, ctx).$$

Security then depends on two interacting principles.

First, special soundness says that two accepting transcripts sharing the same commitment but carrying different challenges reveal a witness or a solution to a hard algebraic problem. In lattice signatures this solution is typically a short relation, hence the appearance of Module-SIS-style assumptions.

Second, the signing algorithm must produce transcripts with the same adversary-visible distribution as the honest protocol would produce. Rejection sampling enforces this distributional property by discarding responses whose shape would leak information about the secret key. Mathematically, if D_{real} is the real signing

distribution and D_{sim} is the simulated transcript distribution, the proof needs a bound on statistical distance:

$$\Delta(D_{\text{real}}, D_{\text{sim}}) = \frac{1}{2} \sum_x |\Pr[D_{\text{real}} = x] - \Pr[D_{\text{sim}} = x]|.$$

The EasyCrypt development decomposes this distance into named sampling, rejection, and clean-conditioned lifting lemmas rather than treating it as one informal statement.

The word “aborts” refers to the rejection step. It is not merely an implementation branch: it is a proof device that shapes the output distribution. A correct provable-security proof must track both the probability of aborting and the distribution conditioned on not aborting.

2.4 Lattice Assumptions and Reduction Targets

HAETAE’s formal security proof ultimately reduces forgeries to structured lattice problems. At the level needed for the dissertation, the key point is not the exact estimator cost of those problems, but the way the reduction connects transcript algebra to assumption games.

Module-LWE is a pseudorandomness assumption. In a typical module form, samples have the shape

$$(A, As + e)$$

over a module R_q^k , where s and e are short or noisy. The assumption says that these samples are computationally indistinguishable from uniform samples of the same type.

Module-SIS is a short-relation assumption. Given a public matrix A , the goal is to find a nonzero short vector z such that

$$Az = 0 \pmod{q}$$

or, in inhomogeneous variants, $Az = t \pmod{q}$. Fiat-Shamir extraction typically produces such a relation from two accepting transcripts with the same commitment and different challenges.

The checked theorem keeps the hardness terms symbolic. This is the right proof engineering boundary: conditional on the declared hop and reduction premises,

EasyCrypt checks that the formal game bound composes the declared reduction modules, symbolic assumptions, and declared probability losses. It does not perform independent lattice-estimator analysis for HAETAE parameter levels.

2.5 EasyCrypt

EasyCrypt supports game-based cryptographic proofs using probabilistic programs, module systems, Hoare logic, probabilistic Hoare logic, and relational equivalence. The HAETAE development uses these features to express game hops, oracle wrappers, state-preservation invariants, and concrete probability bounds.

The mathematical interpretation of the main judgment forms is as follows. A Hoare judgment proves that a program transforms states satisfying a precondition into states satisfying a postcondition. A probabilistic Hoare judgment proves an upper or lower bound on the probability of a postcondition. An equivalence judgment proves a coupling between two probabilistic programs, usually the formal counterpart of a game hop. Losslessness judgments prove that procedures terminate with probability one, which is required before probabilities can be compared without hidden nontermination mass.

For a provable-security proof, these judgments correspond to familiar paper proof obligations:

state invariant	$\leftrightarrow$	Hoare judgment,
bad-event probability	$\leftrightarrow$	probabilistic Hoare judgment,
game indistinguishability	$\leftrightarrow$	equivalence judgment,
well-defined experiment	$\leftrightarrow$	losslessness.

The value of the machine-checked proof is that these obligations are not collapsed into prose. Each must be represented as a typed statement over the actual programs used by the security theorem.

Chapter 3

Proof Scope and Repository Structure

3.1 Separated Dissertation and Proof Folders

The repository now contains a dissertation folder with the same structure as the reference NTT dissertation:

```
1 dissertation/  
2   main.tex  
3   Makefile  
4   bibliography/refs.bib  
5   chapters/  
6   figures/README.txt  
7   record.md
```

The proof itself is managed separately under:

```
1 provable-security/  
2   proof-files.txt  
3   verify-provable-security.sh  
4   easycrypt/  
5   logs/
```

The dissertation folder explains the proof. The `provable-security/` folder defines and checks the proof surface.

3.2 Supplemental Material

Material that is useful for traceability but too mechanical for the main argument is stored under:

```
1 dissertation/etc/
```

This includes the exhaustive generated individual EasyCrypt correspondence appendix. The main dissertation keeps curated proof-object analyses and mathematical ledgers; the supplemental folder preserves the generated audit material without forcing the reader through thousands of templated entries.

3.3 Proof Manifest

The manifest lists the EasyCrypt files that constitute the provable-security proof gate. It includes the main theorem layer, generic signature games, hop games, reductions, ROM programming, scheme and transcript definitions, distribution and algebra support, parameters, and modeled hash-interface support.

3.4 EasyCrypt File Relationships

The files in the manifest form a directed proof-dependency graph. In Table 3.1, an entry $F \leftarrow G$ means that F imports G directly through an EasyCrypt `require import` declaration. Standard EasyCrypt library imports such as `AllCore`, `Real`, `List`, `Distr`, and `PROM` are omitted; Figure 3.1 and the table record only relationships between manifest files.

Read from the leaves upward, the graph has five layers. First, `HAETAE_Keccak1600.ec`, `HAETAE_FIPS202.ec`, and `HAETAE_Params.ec` provide the hash and parameter foundations. Second, `HAETAE_Algebra.ec`, `HAETAE_Distributions.ec`, `HAETAE_Assumptions.ec`, and `HAETAE_Events.ec` define the algebraic, probabilistic, hardness-relation, and event vocabulary. Third, `Sig_ROM.ec`, `HAETAE_ROM.ec`, `HAETAE_Scheme.ec`, `HAETAE_Transcript.ec`, `HAETAE_Rejection.ec`, and `HAETAE_ROM_Programming.ec` build the signature, transcript, rejection, and random-oracle programming interfaces. Fourth, `HAETAE_HopGames.ec` proves the hybrid and simulation relationships. Finally, `HAETAE_Security.ec` imports the

supporting layers and assembles the correctness and EUF-CMA theorem family.

Table 3.1: Immediate manifest-file relationships induced by EasyCrypt imports.

File	Direct manifest imports
HAETAE_Security.ec	Sig_ROM.ec, HAETAE_Scheme.ec, HAETAE_Algebra.ec, HAETAE_Distributions.ec, HAETAE_Reductions.ec, HAETAE_Assumptions.ec, HAETAE_Rejection.ec, HAETAE_Transcript.ec, HAETAE_ROM_Programming.ec, HAETAE_HopGames.ec.
Sig_ROM.ec	None inside the manifest; it depends only on EasyCrypt library files.
HAETAE_HopGames.ec	Sig_ROM.ec, HAETAE_Scheme.ec, HAETAE_Params.ec, HAETAE_Algebra.ec, HAETAE_Distributions.ec, HAETAE_Assumptions.ec, HAETAE_Transcript.ec, HAETAE_Reductions.ec, HAETAE_Rejection.ec, HAETAE_ROM.ec, HAETAE_ROM_Programming.ec.
HAETAE_Reductions.ec	HAETAE_Events.ec.
HAETAE_ROM.ec	HAETAE_Params.ec, HAETAE_Algebra.ec, HAETAE_Distributions.ec.
HAETAE_ROM_Programming.ec	HAETAE_Params.ec, HAETAE_Algebra.ec, HAETAE_Distributions.ec, HAETAE_ROM.ec, HAETAE_Transcript.ec, HAETAE_Rejection.ec, HAETAE_Reductions.ec.

File	Direct manifest imports
HAETAE_Scheme.ec	Sig_ROM.ec, HAETAE_Params.ec, HAETAE_Algebra.ec, HAETAE_Distributions.ec, HAETAE_ROM.ec.
HAETAE_Transcript.ec	HAETAE_Params.ec, HAETAE_Algebra.ec, HAETAE_ROM.ec, HAETAE_Assumptions.ec.
HAETAE_Events.ec	HAETAE_Params.ec, HAETAE_Algebra.ec.
HAETAE_Assumptions.ec	HAETAE_Params.ec, HAETAE_Algebra.ec.
HAETAE_Distributions.ec	HAETAE_Params.ec, HAETAE_Algebra.ec.
HAETAE_Rejection.ec	HAETAE_Params.ec, HAETAE_Algebra.ec, HAETAE_Distributions.ec, HAETAE_Events.ec, HAETAE_Reductions.ec, HAETAE_Transcript.ec.
HAETAE_Algebra.ec	HAETAE_Params.ec, HAETAE_FIPS202.ec.
HAETAE_Params.ec	None inside the manifest; it depends only on EasyCrypt library files.
HAETAE_FIPS202.ec	HAETAE_Keccak1600.ec.
HAETAE_Keccak1600.ec	None inside the manifest; it depends only on EasyCrypt library files.

3.5 Theorem Dependency Classes

To avoid overstating the proof, the dissertation classifies every final security claim into one of four dependency classes.

Table 3.2: Dependency classes used for theorem-scope statements.

Class	Meaning
Checked	A statement accepted by EasyCrypt for the files in provable-security/proof-files.txt.

Class	Meaning
Assumed	A named modeling hypothesis needed to read the checked EasyCrypt model as the intended HAETAE specification model.
Audited	A specification-to-model correspondence supported by local inspection and tables, but not by a checked equivalence theorem.
Out of scope	A claim deliberately not proved by the EasyCrypt security development, such as concrete SHAKE standard-model security, implementation parsing correctness, concrete MLWE/Module-SIS bit estimates, or reduction runtime bounds.

The central theorem is in the checked class only for the EasyCrypt formal model. The specification-level reading is conditional on the assumed and audited classes. This dependency discipline is part of the proof artifact: it prevents a verified game proof from being silently promoted into a stronger end-to-end certification claim.

3.6 Excluded Generated KAT Artifact

`HAETAE_FIPS202_KAT_Empty_Generated.ec` is not part of the provable-security gate. It is a generated FIPS202 known-answer-test artifact. It may be useful for implementation validation or certification evidence, but it is not a reduction step in the EUF-CMA proof. This distinction prevents a large generated file from obscuring the status of the security proof.

The same distinction applies to modeled hash-interface support more generally. The security theorem is a random-oracle-model theorem for the formal model. FIPS202 and Keccak support files help define or validate modeled hash objects, but they do not by themselves prove standard-model security of a concrete SHAKE instantiation.

3.7 ROM-to-FIPS202 Boundary

The proof uses typed random-oracle queries and lazy-ROM laws to reason about freshness, programming, and adversarial prequeries. The FIPS202 and Keccak files are therefore not a replacement for the ROM theorem; they occupy a different verification layer. They may support implementation conformance or hash-interface documentation, but the security theorem continues to assume the random-oracle abstraction. Any theorem replacing the ROM by SHAKE would need an additional assumption or proof principle, for example an indifferentiability or computational random-oracle-instantiation argument. No such theorem is part of the checked provable-security gate described here.

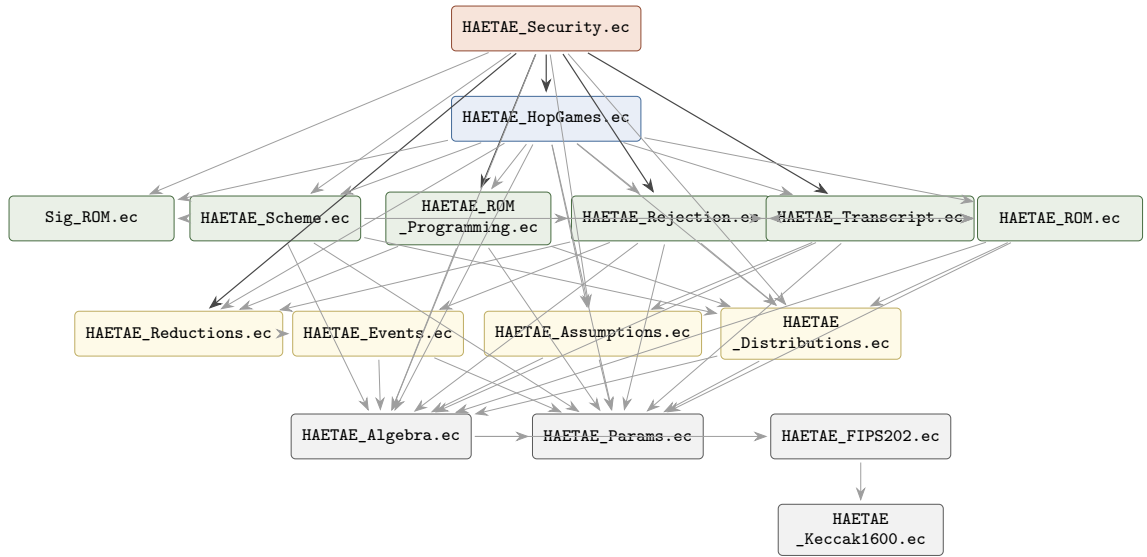


Figure 3.1: Manifest-level dependency graph for the HAETAE EasyCrypt proof. An arrow points from an importing file to a directly imported manifest file; the darker arrows highlight the main top-level assembly paths.

Chapter 4

Model–Specification Correspondence

A machine-checked security proof is only as persuasive as the correspondence between its formal model and the scheme that the proof purports to study. The HAETAE proof surface therefore has two distinct obligations. The first is the internal EasyCrypt obligation: the games, hops, and reductions must check. The second is the external modeling obligation: the objects in those games must faithfully represent the HAETAE specification. This chapter records that second obligation explicitly.

The specification reference used here is `HAETAE_v260204.pdf` [Che+26]. The local copy has 46 pages and was generated on February 4, 2026. The relevant parts for provable security are Definition 1 for the signature-scheme interface, Section 4.2 for the HAETAE algorithms, Figure 8 for the deterministic full algorithms, and Table 1 for the three parameter sets. The EasyCrypt files used for the formal model are principally `Sig_ROM.ec`, `HAETAE_Scheme.ec`, `HAETAE_Params.ec`, `HAETAE_ROM.ec`, `HAETAE_Distributions.ec`, `HAETAE_Algebra.ec`, and `HAETAE_Rejection.ec`.

4.1 Signature-scheme interface

Definition 1 of the specification presents a signature scheme as a triple of algorithms

$$\Pi = (\text{KeyGen}, \text{Sign}, \text{Verify})$$

with public keys, secret keys, messages, signatures, and a verification predicate. The corresponding EasyCrypt interface is intentionally generic. It first defines abstract types and then requires any concrete scheme to implement key generation, signing, and verification procedures.

Table 4.1: Signature-interface correspondence.

Specification object	EasyCrypt object	Mathematical reading
Public key space	type pkey Sig_ROM.ec	in A carrier set $\mathcal{PK}$.
Secret key space	type skey	A carrier set $\mathcal{SK}$.
Message and context	type message, context	The security game works over pairs $(m, ctx) \in \mathcal{M} \times \mathcal{C}$.
Signature space	type signature	A carrier set Σ .
Key generation	proc kg() : pkey * skey	A probabilistic kernel $\text{KeyGen} : 1 \rightarrow \mathcal{D}(\mathcal{PK} \times \mathcal{SK})$.
Signing	proc sign(sk,m,ctx) : signature	A random-oracle relative kernel $\text{Sign}^H(sk, m, ctx)$.
Verification	proc verify(pk,m,ctx,sig) : bool	A predicate $\text{Verify}^H(pk, m, ctx, \sigma) \in \{0, 1\}$.
Chosen-message and strong freshness	fresh_msg, fresh_sig	fresh_msg is the EUF-CMA condition that the final message/context pair was not previously signed. fresh_sig is the stronger condition that the exact message/context/signature triple was not returned before.

The generic interface is not itself a HAETAE model. It is the mathematical ambient category in which the HAETAE model is later instantiated. In categorical language, `HAETAE_Scheme.ec` supplies one object of the class of all random-oracle signature schemes described by `Sig_ROM.ec`.

4.2 Algorithm correspondence

Figure 8 of the specification gives deterministic HAETAE algorithms. The EasyCrypt wrapper in `HAETAE_Scheme.ec` exposes the same three algorithms, but separates the random-oracle queries from the internal algebraic operations. This separation is essential for the game proof: the ROM table, query logs, and programming events must be visible to EasyCrypt.

Table 4.2: Algorithm-level model/spec correspondence.

Specification step	EasyCrypt representation	Proof-relevant interpretation
Sample key-generation and $seed_A, seed_{sk}, K$.	<code>sd <\$ dseed;</code> <code>haetae_keygen_rhoprime</code> <code>sd; keygen_internal.</code>	The model records the entropy source and delegates deterministic key material construction to the internal HAETAE operations.
Expand the public matrix from $seed_A$.	<code>H.get(matrix_expand_query</code> <code>haetae_mode rhoprime)</code> and <code>keygen_internal.</code>	The wrapper records a typed matrix-expansion ROM query, while <code>keygen_internal</code> constructs the modeled matrix through deterministic internal algebra. The ROM output is not consumed by <code>keygen_internal</code> ; it is part of the proof instrumentation for the hash-interface audit.
Compute $b = A_{gen}s_{gen} + e_{gen} + a_{gen}$, split low/high bits, and form (pk, sk) .	<code>keygen_internal</code> <code>haetae_mode rhoprime.</code>	The algebraic construction is part of the scheme model, not part of the ROM wrapper. Its soundness depends on the algebra and parameter files.

Specification step	EasyCrypt representation	Proof-relevant interpretation
Compute $\mu = H_{gen}(seed_A, b_1, M)$.	<code>H.get(message_hash_query pk ctx m)</code> followed by <code>ro_message_hash</code> .	The signing wrapper performs and logs a typed message-hash ROM query. In the current internal algebra, <code>sign_internal</code> recomputes the message hash through the deterministic operation <code>message_hash</code> ; the wrapper's ROM output is therefore observational instrumentation unless the model is changed to pass it into <code>sign_internal</code> .
Derive signing randomness through $seed_{ybb} = H_{gen}(K, \mu)$ and $expandYbb$.	<code>coins <\$ drandom_coins;</code> <code>H.get(sampler_expand_query coins);</code> <code>ro_signing_coins</code> .	The formal proof models the signing sampler as a ROM-mediated randomness source. The model/spec audit must justify that this is the intended abstraction of the deterministic specification path.

Specification step	EasyCrypt representation	Proof-relevant interpretation
Hash the commitment data to obtain challenge seed ρ , then sample the binary challenge.	<code>commitment_highbits,</code> <code>commitment_lowbits,</code> <code>H.get(challenge_hash_query,</code> <code>haetae_mode highbits</code> <code>lowbits mu),</code> and <code>sign_internal.</code>	The challenge point is a separately typed ROM site for freshness and programming analysis. The explicit wrapper call uses the wrapper value <code>mu <- ro_message_hash ro_y</code> . The signature returned by <code>sign_internal</code> , and the honest programming site used by the programming lemmas, instead use the deterministic <code>message_hash</code> recomputed inside the internal algebra. Thus this wrapper call records a typed challenge-hash query near the proof site; it should not be identified with the transcript-derived programming site unless a separate bridge equates the two message-hash values.
Apply rejection tests and return encoded signature components.	<code>sign_internal,</code> <code>HAETAET_Rejection.ec,</code> and distribution lemmas.	The proof reasons about support, rejection predicates, and exact-hyperball sampling through explicit events and loss lemmas.
Decode, recompute challenge data, and check norm/challenge conditions.	<code>verify_internal</code> <code>haetae_mode pk m ctx</code> <code>sig.</code>	Verification is modeled as the deterministic predicate used by the correctness and EUF-CMA games.

The most important modeling distinction is the treatment of signing randomness. The specification describes a deterministic derivation from K and μ , whereas the EasyCrypt signing wrapper first samples `random_coins` and then routes the sampler-expansion point through the random oracle. This is not a minor notation change. It is the reason that the proof contains `sampler_expand_query`, `sampler_bad_prequery`, `sampler_rom_covered`, and the concrete lazy-ROM freshness lemmas. The formal theorem is therefore a theorem about the modeled ROM interface; a complete certification story must additionally maintain a model/spec argument that the deterministic HAETAE sampling path realizes the same distributional role.

4.3 Exact scope statement

The model/specification correspondence in this chapter should be read as an audited mathematical bridge, not as an additional EasyCrypt theorem silently imported into the proof. Let

$$\text{Spec}_{260204}$$

denote the algorithms in the February 2026 specification, and let

$$\text{ECModel}$$

denote the EasyCrypt scheme obtained from `HAETAE_Scheme.ec` and its supporting files. The machine-checked EasyCrypt surface proves the following conditional formal-model implication, where $\mathcal{H}_{\text{EC}}$ abbreviates the final hop/reduction premise and the MLWE and Module-SIS hardness premises appearing in the cited EasyCrypt composition lemma:

$$\mathcal{H}_{\text{EC}} \implies \text{Adv}_{\text{ECModel}, \text{ROM}}^{\text{EUF-CMA}}(A) \leq \text{haetae_euf_counted_rom_bound}.$$

The dissertation does not claim a separately checked theorem of the form

$$\text{Spec}_{260204} \equiv \text{ECModel}.$$

Instead, it records the correspondence evidence needed for a reviewer to judge whether the formal model is the intended model of the specification.

This distinction matters most for signing randomness. In the specification, the signing path derives `seedybb` from secret key material and the message digest, then

expands it. In the EasyCrypt proof, the sampler expansion site is exposed as a typed random-oracle query fed by sampled `random_coins`. The security theorem is exact for the EasyCrypt model; the specification-level conclusion additionally depends on the distributional justification that the exposed sampler interface faithfully represents the specified signing-randomness role.

Assumption 4.1 (Signing-randomness modeling bridge). The specification-level reading of the EasyCrypt theorem assumes the following modeling hypotheses.

1. The specification’s call

$$\text{seed}_{ybb} = H_{\text{gen}}(K, \mu), \quad (y, b, b') = \text{expandYbb}(\text{seed}_{ybb}, \text{counter})$$

is represented in the EasyCrypt model by a domain-separated sampler-expansion random-oracle site

$$q_s = \text{sampler_expand_query}(\rho_s)$$

and interpretation map `ro_signing_coins`.

2. The sampler, message-hash, challenge-hash, and matrix-expansion query constructors are injectively tagged, so that programming or observing one domain does not program or observe another.
3. Before the signing call under consideration, q_s has not appeared in the adversary hash log or in the sampler self-log from earlier signing calls except on the local wrapper event `sampler_bad_prequery`. The counted-ROM prequery and min-entropy terms are separate final-bound ledger entries, not this local event.
4. At a fresh sampler site, the concrete lazy-ROM law

$$\text{ro_signing_coins}(H(q_s)) \equiv D_{\text{coins}}$$

is the law expressed by `concrete_lazy_rom_sampler_expand_query_fresh_eq`.

Under these hypotheses, the distribution of the specification signing sampler, viewed through the random-oracle abstraction, is identified with the EasyCrypt sampler distribution on the clean event. Without conditioning on the clean event, two distinct proof-accounting facts are involved: the local `sampler_bad_prequery` wrapper event is bounded by the combined adversary-hash and sampler-self-log budget, while the

final counted-ROM theorem contains the separate `counted_rom_prequery_term` and `counted_rom_min_entropy_term` summands.

Pen-and-paper justification. By domain separation, the sampler site is disjoint from the message, challenge, and matrix sites. On the clean event, the sampler site is fresh in the lazy-ROM table. The lazy-ROM semantics therefore samples a new oracle output from the declared output distribution, and the interpretation `ro_signing_coins` maps that output to the same coin distribution used by the EasyCrypt signing sampler. The local sampler equivalence can fail before the fresh call only if the adversary has already queried the sampler site or an earlier signing sampler has already logged the same sampler site. These failures are represented by `sampler_bad_prequery` and have their own combined-log budget in the active-signing wrapper. The counted-ROM prequery and min-entropy terms are separate final-bound summands for the ROM programming ledger: the former is a programmed-site prequery term, and the latter records the `bad_min_entropy` predicate on observed transcripts. Thus the bridge is exact under $\neg \text{`sampler\_bad\_prequery`}$, with the remaining accounting split between the local sampler-prequery bound and the counted-ROM terms.

This assumption is intentionally not presented as an EasyCrypt theorem. It is the strongest pen-and-paper bridge currently supplied by the dissertation. Mechanizing it would require a formal relation between the deterministic $H_{gen}(K, \mu)$ -based specification sampler and the typed `sampler_expand_query` interface used by the current proof.

4.4 Internal-algorithm fidelity audit ledger

The previous assumption isolates the signing-randomness abstraction. The same discipline is needed for the remaining internal algorithms: a theorem about `ECModel` is only a theorem about `Spec260204` after the following correspondences have either been mechanized or accepted as audited modeling assumptions.

The status column uses the dependency classes from Table 3.2. In particular, “checked for `ECModel`” means that the EasyCrypt model has the stated internal property; it does not mean that the corresponding line of the PDF specification has been formally related to the EasyCrypt definition. “Audited” means reviewed correspondence evidence. “Assumed” means a named bridge that remains outside the checked proof.

Table 4.3: Internal-algorithm correspondence obligations for reading `ECModel` as the `HAETAE` specification.

Specification object	EasyCrypt object	Current evidence	Status
Figure 8 KeyGen, lines 1–11	<code>keygen_internal</code> , <code>HAETAE_Scheme.kg</code>	The parameter, matrix-expansion, secret-sampling, and public-key equations are represented by the algebraic model and the parameter table in Section 4.5.	Audited; not a checked PDF-to-model equivalence.
Figure 8 Sign, lines 1–17	<code>sign_internal</code> , <code>HAETAE_Scheme.sign</code>	The proof tracks rejection, sampler expansion, message hashing, challenge hashing, and signature validity through the hop-game lemmas and the clean sampler instrumentation.	Checked for <code>ECModel</code> ; specification bridge assumed.
Figure 8 Verify, lines 1–10	<code>verify_internal</code> , <code>HAETAE_Scheme.verify</code>	The verification predicate is the formal accept/reject event used by the EUF-CMA game, but byte-level parsing and malformed encodings are outside the central probability theorem.	Audited; implementation parsing separate.
Table 1 parameters	<code>Mode2</code> , <code>Mode3</code> , <code>Mode5</code> , <code>mode_*</code>	The dissertation records the main numerical correspondences, including $n = 256$, $q = 64513$, (k, ℓ) , τ , and squared norm bounds.	Documented correspondence.

Specification object	EasyCrypt object	Current evidence	Status
$H_{gen}(K, \mu)$ and <code>expandYbb</code>	<code>sampler_expand_query</code> , <code>ro_signing_coins</code>	The EasyCrypt proof has concrete lazy-ROM freshness and self-query logging for the modeled sampler site. The deterministic specification-to-ROM distribution argument is exactly Assumption 4.1.	Explicit assumption.

4.5 Parameter correspondence

Table 1 of the specification gives the parameter sets for security levels 2, 3, and 5. The EasyCrypt file `HAETAETAE_Params.ec` encodes the same modes through `Mode2`, `Mode3`, and `Mode5`. The following table records the main correspondences used by the proof.

Table 4.4: Parameter correspondence between the specification and EasyCrypt.

Specification parameter	EasyCrypt operation	Values / interpretation
Security level	<code>mode</code> with constructors <code>Mode2</code> , <code>Mode3</code> , <code>Mode5</code>	The three formal modes correspond to the three specification parameter sets.
Ring dimension n	<code>n</code>	256.
Modulus q	<code>q</code>	64513; <code>dq</code> is $2q$.
Module dimensions (k, ℓ)	<code>mode_k</code> , <code>mode_l</code>	(2, 4), (3, 6), and (4, 7).
Challenge weight τ	<code>mode_tau</code>	58, 80, and 128.

Specification parameter	EasyCrypt operation	Values / interpretation
Norm/radius bounds B, B', B''	<code>mode_b0sq</code> , <code>mode_b1sq</code> , <code>mode_b2sq</code>	The EasyCrypt model stores squared integer bounds. The correspondence is therefore between the specification radii and the squared thresholds used by norm predicates.
Hint parameters	<code>mode_alpha_hint</code> , <code>mode_m</code>	Mode-dependent constants used by encoding, hint, and verification predicates.
Public-key and signature sizes	<code>mode_publickeybytes</code> , <code>mode_crypto_bytes</code>	Size constants used for specification alignment and encoding-side modeling.

The squared-bound convention is a common source of confusion. The main EasyCrypt validity predicates compare squared norms against squared integer thresholds with non-strict bounds. The proof should not silently identify ordinary norm notation with the exact predicate used in the model; the intended reading for these validity checks is

$$\|z\|^2 \leq B^2 \quad \text{implemented as} \quad \text{normsq}(z) \leq \text{mode_b*sq}(\text{mode}).$$

Some rejection subchecks use strict inequalities against separate scaled rejection bounds; those are distinct abort predicates, not the main `mode_b*sq` validity checks.

4.6 Hash and random-oracle correspondence

The specification uses hash/XOF building blocks H_{gen} , H , $expandA$, $expandS$, and $expandYbb$. The EasyCrypt proof distinguishes three layers:

typed query constructors $\longrightarrow$ lazy ROM state $\longrightarrow$ interpretation functions on ROM outputs.

For example, `message_hash_query`, `challenge_hash_query`, `sampler_expand_query`, and `matrix_expand_query` prevent accidental identification of distinct hash domains.

`HAETAE_RO.FRO.get` supplies the lazy random-oracle semantics. Operations such as `ro_message_hash` and `ro_signing_coins` interpret the raw oracle output as a typed object when a wrapper consumes that output. In the current scheme wrapper, the sampler output is consumed as signing coins, while the matrix, message-hash, and challenge ROM calls are proof-visible instrumentation around deterministic internal operations.

This is modeled hash-interface support, not a standard-model SHAKE theorem. `HAETAE_FIPS202.ec` and `HAETAE_Keccak1600.ec` are useful for specifying or validating concrete hash components, but the provable-security theorem proved in the dissertation is a random-oracle-model theorem for the formal HAETAE scheme.

4.7 Residual model/spec obligations

The following obligations remain outside the central EasyCrypt security theorem and should be tracked whenever the model or the specification changes.

Table 4.5: Model/spec obligations outside the central game proof.

Obligation	Reason
Internal algorithm fidelity	<code>keygen_internal</code> , <code>sign_internal</code> , and <code>verify_internal</code> must be audited against Figure 8 and the algebraic specification. The ROM wrapper exposes oracle structure but does not by itself prove that every internal operation is the same as the specification formula.
Deterministic sampling path	The specification derives signing randomness from secret material and the message digest. The proof model uses typed randomness and ROM-mediated sampler expansion. The correspondence is the explicit modeling assumption in Assumption 4.1, not a checked theorem.
Concrete hash instantiation	The checked implication is in the ROM. Replacing the ROM by SHAKE is a computational-assumption or indistinguishability-style step not proved by the game theorem itself.

Obligation	Reason
Encoding and parsing	The proof-level signature type abstracts over concrete byte encodings. Implementation-level validity of decoding, rejection, and malformed-input behavior is a separate verification target.
Runtime overhead	The checked implication accounts for probabilities and query budgets. Concrete running time and circuit-size overhead of reductions are recorded as an external ledger unless a future cost-aware formalization is added.

This chapter does not weaken the machine-checked implication. It states precisely what the checked surface proves: a conditional ROM security result for the formal HAETAE model, together with an explicit audit trail showing how that model corresponds to the February 2026 specification.

Chapter 5

EasyCrypt Framework

5.1 Modules and Adversaries

The proof uses EasyCrypt modules to represent schemes, oracles, adversaries, and reductions. A module type describes the operations available to a game or adversary. A concrete module instantiates those operations. This is essential for cryptographic reductions because an adversary is not a function in isolation; it is code linked against a particular oracle interface.

Mathematically, an EasyCrypt module should be read as an indexed family of probabilistic transition kernels. A procedure

$$\text{proc } f(x) : \tau$$

denotes a subdistribution over return values and final memories. A module type specifies which kernels an environment may call. This is the formal analogue of the oracle-access notation in cryptographic papers:

$$A^{H, \text{Sign}}(pk).$$

The superscript is not decoration; it is an interface contract. The EasyCrypt module system makes that contract explicit and prevents a proof from silently using state or procedures that the adversary should not possess.

Functor-style modules are the proof mechanism behind reductions. A reduction is code that links an adversary against one game to oracles for another game. In mathematical notation this is usually written as $B^O(A)$. In EasyCrypt it is a module

parameterized by both adversary and oracle modules. The type checker therefore enforces which information crosses the reduction boundary.

5.2 Probabilistic Judgments

The proof uses several EasyCrypt judgment forms. Hoare judgments prove state properties of programs. Probabilistic Hoare judgments prove probability bounds. Equivalence judgments prove that two games produce related outputs under related states. Losslessness lemmas establish that procedures terminate with probability one.

The corresponding mathematical readings are:

$$\begin{aligned} \{P\} c \{Q\} & \quad \text{for every initial memory satisfying } P, \text{ every terminating final memory satisfies } Q, \\ \Pr_c[Q \mid P] \leq \alpha & \quad \Pr[c \text{ terminates in a } Q\text{-state} \mid P] \leq \alpha, \\ c_1 \sim c_2 : P \implies Q & \quad \text{there is a coupling of } c_1, c_2 \text{ taking } P\text{-related states to } Q\text{-related states.} \end{aligned}$$

The exact EasyCrypt surface syntax differs from this display, but the semantic content is this combination of state transformers, probability measures, and relations.

Game-hopping proofs usually require all three forms. A relational equivalence shows that two games are identical until a bad event; a probabilistic Hoare judgment bounds the bad event; ordinary Hoare judgments maintain invariants such as query-log consistency and budget monotonicity. This decomposition is the machine-checked version of the fundamental lemma of game playing.

5.3 Fundamental Lemma of Game Playing

The fundamental lemma can be stated in total-variation language. Let G_0 and G_1 be two games with Boolean outputs, and let **Bad** be an event. If the games are coupled so that their views and outputs agree whenever **Bad** does not occur, then

$$|\Pr[G_0 \Rightarrow 1] - \Pr[G_1 \Rightarrow 1]| \leq \Pr[\mathbf{Bad}].$$

Many HAETAЕ proof obligations are instances of this lemma. The difficult part is not the inequality itself; it is proving that the relevant games are truly identical on the complement of the bad event. For ROM programming, this means the programmed

point was fresh. For sampler lifting, this means the sampler query was not present in the adversary hash log or in the sampler self-log. For budgeted wrappers, this means oracle counters and logs remain synchronized.

5.4 Proof Style

The development uses named lemmas heavily. This makes the proof auditable: rather than compressing all reasoning into a single top-level statement, it exposes intermediate facts such as oracle preservation, bad-event bounds, query-budget invariants, and reduction-term arithmetic.

A useful way to read the proof scripts is as a typed refinement of a pen-and-paper proof outline. Paper proofs often use phrases such as “by freshness,” “by rejection sampling,” or “by a standard hybrid argument.” In the EasyCrypt development each such phrase must correspond to a theorem with explicit hypotheses. The main review question is therefore not only whether the top-level checked implication compiles, but whether every lemma name and hypothesis corresponds to the intended mathematical invariant.

Chapter 6

Signature Games, Theorem Chain, and Paper Proof

6.1 Generic ROM Signature Interface

The file `Sig_ROM.ec` defines the generic games. Important declarations include `Oracle`, `POracle`, `Scheme`, `SignOracle`, `Adversary`, `NMA_Adversary`, `EUFCMA`, and `UFNMA`. These definitions give the proof a stable target: HAETAE security means security in this game interface.

The distinction between EUFCMA and UFNMA is proof-theoretic. The chosen message game gives the adversary an adaptive signing oracle. The no-message game removes that oracle and is therefore closer to the algebraic extraction surface. Much of the theorem chain is devoted to justifying that signing-oracle access can be simulated and internalized without changing the adversary's view except for explicitly bounded losses. In paper notation this is often a single sentence; in the checked development it becomes a sequence of modules and relational lemmas.

6.2 Game-Hopping Calculus

Let $G_0, \dots, G_n$ be a sequence of games with the same Boolean success event. The standard telescoping inequality is

$$|\Pr[G_0 \Rightarrow 1] - \Pr[G_n \Rightarrow 1]| \leq \sum_{i=0}^{n-1} |\Pr[G_i \Rightarrow 1] - \Pr[G_{i+1} \Rightarrow 1]|.$$

Each adjacent term must be justified by either perfect equivalence, a fundamental-lemma bad-event bound, or a reduction to a hardness assumption. The HAETAE proof is long because the adjacent games are not cosmetic. They expose oracle logs, signing transcripts, sampler sites, rejection events, and exact sampling distributions at different stages.

This calculus also explains why loss ledgers are delicate. A loss should appear exactly once: either as a local bad-event probability, as a statistical-distance term, or as part of a reduction advantage. Double-counting makes the theorem weaker and conceptually inaccurate; omitting a term makes it false.

6.3 Pen-and-Paper EUF-CMA Game

Let $\Pi = (\text{KeyGen}, \text{Sign}, \text{Verify})$ be the modeled HAETAE signature scheme. The EUF-CMA experiment is:

1. Sample $(pk, sk) \leftarrow \text{KeyGen}()$.
2. Give pk and random-oracle access to an adversary A .
3. Answer each signing query (m, ctx) with $\sigma \leftarrow \text{Sign}(sk, m, ctx)$, and record (m, ctx) .
4. Let A output (m^*, ctx^*, σ^*) .
5. The adversary wins if $\text{Verify}(pk, m^*, ctx^*, \sigma^*) = 1$ and (m^*, ctx^*) was not a previous signing query.

The advantage is the probability of this event over key generation, oracle randomness, signing randomness, and adversarial randomness.

The resource parameters q_H and q_S are part of the paper-style loss ledger and of the budgeted wrapper games, not syntactic caps on the source Sig_ROM.EUF_CMA game. In a concrete proof, these parameters enter collision bounds, freshness bounds, and wrapper lemmas. A theorem checked only for fixed wrapper constants q_H, q_S should not be cited as a symbolic-query or arbitrary-query theorem without a separate parameterized development.

Schematic paper-style ledger. A fully parameterized paper theorem would have the form

$$\text{Adv}_{\text{HAETAE}}^{\text{EUF-CMA}}(A) \leq \sum_i \text{Adv}_i(B_i) + \text{Loss}_{\text{ROM}}(q_H, q_S) + \text{Loss}_{\text{local}}(q_H, q_S),$$

where $\text{Loss}_{\text{local}}$ denotes intermediate rejection, sampling, and lifting costs before they are composed into a final implication. The dissertation does not claim that this symbolic-query statement has been checked as written. The checked counted-ROM implication fixes $q_H = q_S = 2$ and uses the concrete bound `haetae_euf_counted_rom_bound`; in that final bound, rejection and exact-sampling obligations are discharged inside the hop premises rather than added again as separate final summands.

Pen-and-paper proof sketch. The proof proceeds by a sequence of games. First, replace real signing by a signing simulator while preserving the adversary-visible distribution. Second, expose internal signing transcripts and erase components not visible to the adversary. Third, move from chosen-message access to an NMA-style game by embedding signatures into simulated transcript generation. Fourth, program the random oracle at challenge sites and account for the event that the adversary prequeried a programmed point. Fifth, pass through the rejection-sampling and paper-simulation sampler surface, recording the checked rejection and lifting obligations. Sixth, replace the paper/structural sampler view by the exact hyperball sampler used in the formal model, then apply special soundness and the bimodal-to-Module-SIS bridge to turn a forgery into a lattice-assumption adversary. Summing the statistical distances and bad-event probabilities gives the schematic inequality above; the machine-checked counted theorem uses the fixed-ledger composition and grouping described later in the dissertation. $\square$

The proof sketch is intentionally schematic. The checked proof strengthens it by replacing each informal distributional claim with an EasyCrypt theorem. For example, “preserving the adversary-visible distribution” becomes a relational equivalence between games; “account for the event that the adversary prequeried a programmed point” becomes a logged-ROM bad-event probability bound; and “summing the losses” becomes arithmetic over named bound operations in `HAETAE_Reductions.ec`.

6.4 Machine-Checked Theorem Chain

`HAETAE_Security.ec` contains the top-level composition chain. Representative lemmas include `correctness_perfect`, `euf_cma_security`, `euf_cma_security_with_hop_interfaces`, and later bridge lemmas that connect ROM transcript games, counted bimodal reductions, paper-simulation NMA games, rejection sampling, and exact hyperball sampling.

The proof follows the paper argument, but each paper step is made explicit:

Paper step	EasyCrypt realization
Real EUF-CMA game	<code>Sig_ROM.EUF_CMA</code> and <code>HAETAE_Security.euf_cma_security</code> .
Signing simulation	<code>EUF_CMA_SimulatedSign</code> and <code>RealSigningSimulator</code> .
Transcript exposure	<code>EUF_CMA_TranscriptSimulatedSign</code> , <code>EUF_CMA_InternalTranscriptSign</code> .
ROM-internal transcript game	<code>EUF_CMA_ROMInternalTranscriptSign</code> .
NMA reduction surface	<code>ROMInternalTranscriptAsNMA</code> and budgeted variants.
Paper sampling to exact sampling	<code>ROSigningAttemptPaperSimSampler</code> , <code>ROExactHyperballPaperSimSampler</code> .
Final concrete bound	<code>HAETAE_Reductions.ec</code> grouped bound lemmas.

6.5 Detailed EasyCrypt Code Analysis

The design of `Sig_ROM.ec` is intentionally parametric. The module type `Scheme(0 : POracle)` prevents the proof from assuming a concrete hash implementation in the generic security game. The adversary receives only the interfaces exposed by `POracle` and `SignOracle`; therefore, any state used internally by the simulator must be hidden unless explicitly exposed by a later game.

The `HAETAE`-specific theorem file does not redefine security. Instead, it instantiates and bounds the generic security game. This is a crucial design choice. If `HAETAE_Security.ec` had its own custom forgery game, the proof would also need

a separate theorem showing equivalence with the standard signature notion. By reusing `Sig_ROM.ec`, the development makes the security target auditable.

The long chain of lemmas in `HAETAE_Security.ec` should be read as a machine-checked proof outline. Lemmas with names containing `from_rom_transcript`, `from_structural_nma`, `from_paper_sim_nma`, and `from_exact_hyperball` correspond to successive paper-proof transformations. This naming discipline is valuable: it lets a reviewer locate the exact checked boundary for each cryptographic claim.

Chapter 7

Random Oracle Programming: Paper Proof and Code Analysis

7.1 Lazy ROM State

The random oracle is modeled by a finite map and query log. On a new query it samples and records an answer; on a repeated query it returns the stored answer. This lazy representation allows the proof to state exactly what the adversary has queried and exactly when programming is safe.

Formally, the lazy ROM invariant says that the table T is a partial realization of an ideal function. If $x \notin \text{dom}(T)$, then the conditional law of $H(x)$, given the current transcript, is still the declared output law D_x . If $x \in \text{dom}(T)$, then consistency requires all future calls on x to return the stored value. Thus the table domain is the mathematical object that separates fresh randomness from already observed randomness.

For HAETAE, the input space is not a single untyped string space in the proof. The EasyCrypt model uses typed constructors for message hashing, challenge hashing, matrix expansion, and sampler expansion. This typed interface is a formal domain-separation discipline. It prevents a proof about one oracle site from accidentally applying to another site with a different mathematical role.

7.2 Pen-and-Paper Programming Lemma

Lemma 7.1 (ROM programming at a fresh point). *Let H be a random oracle and let x be a point sampled independently of an adversary's prior query set Q . If $|Q| \leq q_H$ and x has support min-entropy at least $\log_2 N$ against the adversary's view, equivalently every point has conditional probability at most $1/N$, then*

$$\Pr[x \in Q] \leq \frac{q_H}{N}.$$

Conditioned on $x \notin Q$, assigning a value to $H(x)$ changes no adversary-visible answer to any previous query. If the assigned value is sampled from the same conditional output law D_x , independently of the prior view, then the future lazy-oracle interface has the same distribution as ordinary lazy sampling at x . If instead the simulator assigns a fixed or otherwise biased value, freshness alone gives only the previous-answer preservation fact; the distributional effect of future queries to x must be justified by the reduction or accounted for as a separate game-hop condition.

Proof. For any fixed query set Q , the union bound gives $\Pr[x \in Q] \leq \sum_{u \in Q} \Pr[x = u]$. If every point has probability at most $1/N$, this is at most $|Q|/N \leq q_H/N$. If $x \notin Q$, the lazy random oracle has not assigned a value to x . Assigning a value at x therefore leaves the map entries for all previous queries unchanged. When the assigned value has law D_x , the programmed table is distributed exactly as if the lazy oracle had sampled $H(x)$ on its first future query. Without that same-law condition, the lemma should be used only as a preservation statement for the already observed oracle answers. $\square$

7.3 Programming Sites

A programming site is the oracle input where the simulator wants to control the challenge. The file `HAETAE_ROM_Programming.ec` proves that honest signing produces programming sites with the necessary structural and freshness properties. Lemmas such as `programming_site_self_no_reprogramming_failure` and `honest_signing_programming_site_query_spread_bound` capture the core obligations.

The relevant mathematical object is a random variable X_{prog} whose value is the site to be programmed. The proof must establish three facts: the site equation identifying

X_{prog} with the query produced by the signing or challenge construction, the freshness or spread bound against the adversary's log, and the preservation statement saying that programming X_{prog} changes no previously visible oracle answer. Missing any one of these facts invalidates the programming step.

7.4 Detailed EasyCrypt Code Analysis

`HAETAE_ROM_Programming.ec` separates four concerns that are often merged in paper proofs.

First, the counted lazy ROM lemmas, such as `counted_lazy_rom_init_clears` and `counted_lazy_rom_get_preserves_clear`, establish basic state invariants. These lemmas say that initialization clears logs and that ordinary queries do not accidentally set programming-failure flags.

Second, programming-site lemmas characterize the exact query that honest signing will need. For example, `honest_signing_programming_site_queryE` is an equation-style lemma: it identifies the formal query produced by the signing algorithm. Without such lemmas, later freshness statements would refer to a site whose connection to signing is only informal.

Third, spread and budget lemmas translate entropy into probability bounds. `honest_signing_programming_site_query_spread_bound` is the machine-checked counterpart of the paper inequality $\Pr[x \in Q] \leq q_H/N$. The EasyCrypt proof must explicitly name the distribution, support, and query log.

Fourth, bad-event lemmas connect the local ROM facts to game-level losses. Lemmas such as `programmed_site_prequery_one_step_challenge_concrete_bound` and `programmed_site_prequery_one_step_message_hash_counter_bound` are the concrete interface consumed by the hop games.

7.5 Bad Events and Fundamental Lemma Use

The proof makes bad events explicit. Prequery events, reprogramming conflicts, and entropy failures are represented by predicates or flags. Fundamental-lemma arguments then bound the difference between games by the probability of these bad events. In the EasyCrypt code, this means proving both a relational similarity

condition before the bad event and a probability bound for setting that event.

This is the point where informal proofs most often hide work. A statement such as “the adversary did not query this point except with probability q_H/N ” contains an independence claim, a support-size claim, a query-budget claim, and a union-bound argument. The checked development separates these into concrete lemmas so that the final ROM loss is not a heuristic estimate but the result of named program invariants and probability judgments.

Chapter 8

Sampler Freshness and Exact Hyperball Sampling

8.1 Sampler Interfaces

The proof contains several sampling modules, including `ROSigningAttemptPaperSimSampler`, `HAETAERejectionPaperSimSampler`, `ExactHyperballPaperSimSampler`, and `ROExactHyperballPaperSimSampler`. These modules allow the proof to compare paper-style structural sampling with exact formal sampling.

8.2 Pen-and-Paper Freshness Bound

Lemma 8.1 (Sampler prequery bound). *Let s be fresh sampler seed coins drawn from the distribution D_{seed} . Let $\text{expand}(s)$ be the random-oracle query used by the sampler. Let Q_A be the set of adversarial hash queries logged before sampling, let Q_{sampler} be the sampler self-log from earlier signing calls, and let $Q = Q_A \cup Q_{\text{sampler}}$. If $|Q_A| \leq q_H$, $|Q_{\text{sampler}}| \leq q_S$, and $\text{expand}(s)$ has min-entropy at least $\log_2 N$, then*

$$\Pr[\text{expand}(s) \in Q_A \cup Q_{\text{sampler}}] \leq (q_H + q_S)/N.$$

Proof. Condition on the game view before the fresh seed is drawn. The joint logged query set $Q_A \cup Q_{\text{sampler}}$ is then fixed and has cardinality at most $q_H + q_S$. The min-entropy condition says that every query value has probability at most $1/N$. The union bound over all entries in the joint log gives the result. The conditioning is

valid because the seed is sampled after the logged adversarial queries and earlier sampler self-queries and is independent of both logs. $\square$

8.3 The `sampler_bad_prequery` Event

A central event is `sampler_bad_prequery`. It records whether the random-oracle input derived from fresh sampler seed coins has already appeared in the adversary hash log or in the sampler self-log from earlier signing calls. The tight proof routes adversary hash queries and sampler self-queries through logs and proves that a fresh seed collides with the joint log only with probability bounded by the combined query budget.

8.4 Clean-Conditioned Lifting

Under the clean condition that `sampler_bad_prequery` is false, the proof lifts the one-step structural-to-exact sampling result through the budgeted signing-oracle wrapper. This is represented by lemmas such as `concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_clean_condit`.

8.5 Detailed EasyCrypt Code Analysis

The EasyCrypt code decomposes sampler freshness into local and lifted facts. Local lemmas such as `concrete_lazy_rom_sampler_expand_query_fresh_phoare` reason about one ROM call and one seed. Structural lemmas such as `concrete_ro_signing_attempt_sample_with_seed_fresh_structural_le` connect that local fact to the signing-attempt sampler. Guarded-clean lemmas then add the condition that the bad event is false.

The exact-hyperball lemmas, including `concrete_ro_exact_hyperball_sample_with_seed_exact` and `concrete_ro_signing_attempt_to_exact_hyperball_sample_with_seed_clean_loss`, form the bridge from the structural paper sampler to the exact formal sampler. The code is written this way because each level has a different proof burden: ROM freshness is local, sample distribution equivalence is sampler-specific, and lifting through the adversary is game-specific.

A reviewer should pay close attention to the distinction between lemmas ending in `fresh_loss`, `clean_loss`, and `clean_conditioned_lifting`. The first bounds the probability of a collision. The second states the loss under the clean event. The third transports the result through the full budgeted game wrapper.

Chapter 9

Transcript Games and Erasure

9.1 Transcript Simulation

Transcript games expose the internal structure needed for simulation and extraction. The proof includes modules for transcript-simulated signing, internal transcript signing, and ROM-internal transcript signing. These games bridge the gap between a real signing oracle and the NMA-style games used in the reduction.

9.2 State Preservation

Representative lemmas prove that signing oracles preserve transcript validity, log readiness, min-entropy conditions, and ROM state invariants. These are not cosmetic properties; they are the invariants needed to apply later bad-event and special-soundness arguments.

9.3 Erasure Exactness

Erasure lemmas show that internal transcript information can be removed without changing the adversary-visible distribution. In a paper proof this might be a sentence. In EasyCrypt it is a relational statement about two games with related states and identical observable outputs.

Chapter 10

Budgeted Lifting Through Oracles

10.1 Adversarial Query Budgets

ROM proofs depend on query counts. The development therefore proves that hash and signing query budgets are preserved by the relevant wrappers. The budgeted interfaces certify that prequery and programming losses are multiplied by the right quantities.

10.2 Pen-and-Paper Fundamental-Lemma Lifting

Lemma 10.1 (Budgeted one-call lifting). *Suppose two signing-oracle implementations O_0 and O_1 are identical unless a clean event C fails during one signing call. If for every state satisfying the budget invariant*

$$\Pr[\neg C \text{ during one call}] \leq \epsilon,$$

and the adversary makes at most q_S signing calls, then the distinguishing loss between the two full games is at most $q_S\epsilon$, plus any separately accounted hash-query bad events.

Proof. Index the signing calls from 1 to q_S . Define a hybrid sequence in which the first i calls use O_1 and the remaining calls use O_0 . By the fundamental lemma, adjacent hybrids differ by at most the probability that the clean event fails in call $i + 1$. The budget invariant ensures that each such probability is bounded by ϵ .

Summing over at most q_S calls gives $q_S\epsilon$. Hash-query bad events are added by another union-bound or fundamental-lemma step. $\square$

10.3 One-Call Signing Loss

The concrete one-call signing loss relates an attempt-side clean projection to an exact-side projection. Lemmas such as `concrete_budgeted_ro_signing_attempt_o_sign_active_clean_si`, `concrete_budgeted_ro_exact_hyperball_o_sign_active_signature_exact`, and `concrete_budgeted_o_sign_clean_one_call_loss_from_self_logged_surface` encode this local proof.

10.4 Detailed EasyCrypt Code Analysis

The module `ROMInternalTranscriptBudgetedPaperSimAsNMA` is the central budgeted wrapper. Its signing oracle `0.sign` is where a one-step sampler loss must be lifted through adversarial interaction. The code first proves projection lemmas: the attempt side projects from `0.sign` to the self-logged `sample_with_seed` event, and the exact side projects from `sample_with_seed` back to exact signing.

The lemma `concrete_budgeted_self_logged_signature_clean_event_loss_surface` acts as a reusable surface theorem. It abstracts over the local event and requires both projections. The final lemma `concrete_budgeted_o_sign_clean_one_call_loss_from_self_logged` instantiates that surface for the concrete HAETA lazy ROM. This structure is stronger than a coarse local wrapper bound because it proves that the exact events used by the signing oracle are the events bounded by the sampler proof.

10.5 From One Call to NMA Wrapper Obligations

The one-call bound supplies the local surface that a non-vacuous NMA-level lift should consume. The checked lemmas whose names contain `rom_internal_budgeted_nma` expose the bridge from local signing facts to adversarial-game facts, including the concrete clean event and the `sampler_bad_prequery` log. The present top-level path still has coarse fallback steps: some NMA bridge lemmas use probability bounds whose usefulness depends on the oversized `rejection_sampling_loss_term`. The

remaining proof obligation is to make the final NMA composition consume the local clean one-call surface directly.

Chapter 11

Reductions, Concrete Bounds, and Paper Algebra

11.1 Reduction Terms

`HAETAE_Reductions.ec` collects the algebra of the final security bound. It defines and relates terms for signing-query scaling, ROM collision losses, prequery losses, min-entropy losses, rejection-sampling losses, and bimodal to Module-SIS reduction losses.

11.2 Pen-and-Paper Bound Composition

Let the game hops produce losses $\epsilon_1, \dots, \epsilon_k$, and let the final extractor produce reductions $B_1, \dots, B_r$. At the schematic paper-theorem level, the standard game-hop composition gives

$$\text{Adv}_{\text{HAETAE}}^{\text{EUF-CMA}}(A) \leq \sum_{j=1}^r \text{Adv}_j(B_j) + \sum_{i=1}^k \epsilon_i.$$

For HAETAE, the intermediate ϵ_i ledger includes ROM collision terms, programmed-site prequery terms, sampler-prequery terms, min-entropy terms, rejection-sampling losses, and exact-sampling replacement losses. This is the paper-style hop ledger, not the final counted-ROM right-hand side. In the final checked implication, the rejection and exact-sampling obligations are antecedents or bridge-lemma costs, while the displayed final bound adds only the hardness terms, the bimodal-to-Module-SIS reduction term, and the counted ROM collision/prequery/min-entropy terms. This is

a logical grouping statement, not by itself a non-vacuity statement: with the current constants the paper-style `rejection_sampling_loss_term` is larger than one, so any final antecedent that adds this term to a probability must be separately checked for satisfiability before the implication is cited as a useful concrete bound. The checked instantiation discussed in this dissertation replaces the schematic left-hand side by $\text{Adv}_{\text{ECModel,ROM}}^{\text{EUF-CMA}}(A)$, while the right-hand-side bound ledger is instantiated at $q_H = q_S = 2$.

Pen-and-paper proof of composition. Let G_0 be the real EUF-CMA game and G_k be the final reduction game. For each adjacent pair, prove either exact equality of the relevant event or a statistical-distance bound $|\Pr[G_i] - \Pr[G_{i+1}]| \leq \epsilon_i$. By the triangle inequality, $|\Pr[G_0] - \Pr[G_k]| \leq \sum_i \epsilon_i$. The final game success is bounded by the reduction advantages $\sum_j \text{Adv}_j(B_j)$. Combining the inequalities gives the result. $\square$

11.3 Nonnegativity and Cardinality

Concrete bounds require side conditions. The proof includes nonnegativity lemmas for budgets and loss terms, lower bounds for challenge support, and subunit bounds where probabilities must be at most one. This arithmetic layer is critical: it ensures the final implication composes valid inequalities rather than relying on informal real-number reasoning.

11.4 Detailed EasyCrypt Code Analysis

`HAETAEReductions.ec` should be read as the arithmetic ledger of the security proof. Lemmas ending in `E` are usually expansion lemmas: they rewrite named loss terms into their concrete expressions. Lemmas ending in `nonnegative`, `positive`, `le1`, or `subunit` discharge side conditions required by probability reasoning.

For example, `counted_rom_programming_loss_term_concreteE` connects a named loss to its concrete budget/support expression, while `counted_rom_programming_loss_term_subunit` proves that the expression is a valid probability-scale quantity. Similarly, `challenge_support_cardinality_lower_bound_positive` prevents a hidden division-by-zero assumption in entropy-based bounds.

The theorem file `HAETAE_Security.ec` consumes these arithmetic facts rather than re-proving arithmetic at every hop. This separation improves proof maintenance: changing a bound expression should affect the reduction ledger and a small number of bridge lemmas, not the entire game-hop development.

11.5 Hardness Boundary

The final proof is conditional on the assumptions encoded in `HAETAE_Assumptions.ec`. The machine-checked result therefore states that the modeled HAETAE adversary's advantage is bounded by formal reduction advantages and checked loss terms under those assumptions.

Chapter 12

Pen-and-Paper Security Proof Reconstructed from EasyCrypt

This chapter reconstructs the pen-and-paper proof of HAETAE provable security using the EasyCrypt development as a checked source of lemmas. The aim is not to analyze the proof script line by line. The aim is to write the mathematical hybrid argument that a cryptographer would expect on paper, while citing the EasyCrypt theorem names that certify each step.

12.1 Proof convention

Let A be a probabilistic polynomial-time adversary in the modeled random-oracle interface. The adversary interacts with a hash oracle H and a signing oracle O , then outputs a candidate forgery (m^*, ctx^*, σ^*) . The event **Forge** is the event that verification accepts and the message-context pair was not previously signed.

For the informal target cryptographic game, we write

$$\text{Adv}_{\text{HAETAE,ROM}}^{\text{EUF-CMA}}(A) = \Pr[\text{Forge in the real EUF-CMA game}].$$

The checked theorem reconstructed below uses the EasyCrypt-model advantage; the fixed query constants belong to the proof ledger and bound terms rather than to a syntactic restriction on the source EUF-CMA game. All probabilities in this chapter are over the coins of key generation, signing, the random oracle, and the adversary. When this reconstruction mirrors paper-style notation, q_H and q_S denote the fixed

wrapper counts used by the proof ledger; in the EasyCrypt development these are represented by `hash_query_budget_count` and `signature_query_budget_count`.

The concrete counted-ROM loss expression is the one defined in `HAETAE_Reductions.ec`. For the theorem emphasized in this dissertation we write it schematically as

$$B_{\text{HAETAE}} = B_{\text{hard}} + B_{\text{ROM}},$$

where B_{hard} expands to the MLWE, Module-SIS, and bimodal-to-MSIS terms, and B_{ROM} expands to the counted collision, prequery, and min-entropy terms. The paper-style `haetae_euf_bound` contains a separate rejection/Fiat–Shamir-with-aborts ledger, but it is not the final counted-ROM theorem. The counted-ROM grouping is certified by `haetae_euf_counted_rom_bound_groupedE`.

Theorem 12.1 (Reconstructed formal-model counted-ROM implication). *For every adversary A satisfying the declared EasyCrypt module interfaces, with the proof and bound ledger instantiated through the fixed wrapper constants $q_H = q_S = 2$, if the explicit hop/reduction premise and the MLWE and Module-SIS hardness premises required by `euf_cma_security_from_checked_ro_sampling_hop_and_counted_bimodal` hold for the declared reduction modules appearing in those antecedents, then the formal EasyCrypt model satisfies*

$$\text{Adv}_{\text{ECModel,ROM}}^{\text{EUF-CMA}}(A) \leq \text{haetae_euf_counted_rom_bound}.$$

Equivalently, after expanding the counted-ROM bound,

$$\text{Adv}_{\text{ECModel,ROM}}^{\text{EUF-CMA}}(A) \leq L_{\text{MLWE}} + L_{\text{ModSIS}} + L_{\text{bimodal} \rightarrow \text{MSIS}} + L_{\text{ROM}},$$

where the hardness terms and ROM loss terms are the concrete real expressions in `HAETAE_Reductions.ec`. This is not a specification-level theorem for `Spec260204`, and it is not an arbitrary-query theorem.

Proof. The proof is a sequence of games. Each displayed transition is an ordinary paper-proof step; the cited EasyCrypt lemmas are the machine-checked certificates that the step is valid. The final composition is not a bare closed theorem over all adversaries: it is a fixed-ledger formal-model implication whose antecedents are the final NMA/hardness hop and the declared MLWE and Module-SIS bounds. $\square$

12.2 Correctness and the starting game

The starting game G_0 is the standard, unbudgeted EUF-CMA game for the random-oracle signature interface:

$$G_0 = \text{Sig_ROM.EUF_CMA}(\text{HAETAE}, H, A).$$

Thus, inside the EasyCrypt source game used by the checked composition,

$$\Pr[G_0(A) \Rightarrow 1] = \text{Adv}_{\text{ECModel,ROM}}^{\text{EUF-CMA}}(A).$$

The fixed constants $q_H = q_S = 2$ enter through the later budgeted wrappers and the right-hand-side bound, not through this source-game notation. The correctness side of the theorem is independent of the unforgeability hybrid. It states that honestly generated signatures verify:

$$\Pr[\text{Verify}(pk, m, ctx, \text{Sign}(sk, m, ctx)) = 1] = 1.$$

The EasyCrypt proof anchors are `correctness_obligation_holds` and `correctness_perfect`. They discharge the paper proof's statement that the modeled signing and verification procedures are consistent.

12.3 Removing the direct signing view

The first unforgeability step replaces real signing calls by a signing simulator that exposes the same distributional interface to the adversary. Let G_1 be the simulated-signing game:

$$G_1 = \text{EUF_CMA_SimulatedSign}.$$

The paper claim is that the adversary's success probability is unchanged, or bounded by the explicit signing-hop loss when one moves through the more refined interfaces:

$$\Pr[G_0(A) \Rightarrow 1] \leq \Pr[G_1(A) \Rightarrow 1] + L_{\text{sign}}.$$

The corresponding checked statements are `euf_cma_security_from_simulated_hop`, `euf_cma_security_from_explicit_boundaries`, and `euf_cma_security_with_hop_interfaces`.

At the paper level, this step is justified by the fact that a signature is fully

determined by the transcript fields sampled by the signing algorithm. The transcript predicates record the required public equation, challenge query, response, hint, and rejection-branch consistency. These facts are certified by `transcript_valid_public_equation`, `transcript_valid_signature_challenge_query`, `transcript_fields_match_signature_challenge_query`, and `transcript_valid_commitment_reco`

12.4 Internal transcript and random-oracle view

The next step internalizes the transcript log and the random-oracle state. Let G_2 be the transcript-simulated signing game, G_3 the internal transcript game, and G_4 the ROM-internal transcript game:

$$G_2 = \text{EUF_CMA_TranscriptSimulatedSign}, \quad G_3 = \text{EUF_CMA_InternalTranscriptSign}, \quad G_4 = \text{EUF_CMA_ROMInternalTranscriptSign}$$

The paper-proof statement is

$$\Pr[G_1(A) \Rightarrow 1] \leq \Pr[G_4(A) \Rightarrow 1] + L_{\text{tr}}.$$

The reason is that transcript logging and lazy-ROM internalization do not change the adversary's observable interface, except at explicitly counted bad events.

The EasyCrypt invariants used here are:

<code>transcript_log_consistent</code>	logged signatures match transcript fields,
<code>transcript_log_ready</code>	transcripts are ready for programming,
<code>internal_transcript_state_sound</code>	internal state agrees with key and transcript data,
<code>sampler_rom_covered</code>	sampler ROM calls are represented in the ROM state.

Their preservation under signing is checked by `transcript_log_consistent_sign_internal_cons`, `internal_transcript_state_sound_sign_internal_cons`, and the sampler-coverage lemmas `sampler_rom_covered_fresh_after_clean_seed`, `sampler_rom_covered_self_log`, and `sampler_rom_covered_old_log_clean_boundary`.

In paper terms, this is the point where the proof stops treating the random oracle as an external black box and begins treating it as a lazy table whose queries, programmed sites, and bad events can be counted.

12.5 ROM programming and counted bad events

The Fiat–Shamir-with-aborts proof programs challenge sites in the random oracle. A programming site is a pair

$$s = (q, y) \in Q_{\text{RO}} \times Y_{\text{RO}},$$

where q is the query to be programmed and y is the desired output. The programming is sound when the site is fresh:

$$q \notin Q_A,$$

where Q_A is the adversary’s previous random-oracle query log.

The paper proof decomposes the failure probability into counted events:

$$L_{\text{ROM}} = L_{\text{coll}} + L_{\text{pre}} + L_{\text{me}}.$$

This corresponds to the EasyCrypt terms `counted_rom_collision_term`, `counted_rom_prequery_term`, and `counted_rom_min_entropy_term`. Their grouping is certified by `counted_rom_programming_loss_termE` and `counted_rom_programming_loss_splitE`.

The preservation laws for the lazy ROM are the checked form of the paper argument that lazy sampling remains consistent after query, observation, and programming:

<code>counted_lazy_rom_init_clears</code>	initial state has no bad event,
<code>counted_lazy_rom_get_preserves_clear</code>	ordinary queries preserve clear
<code>counted_lazy_rom_program_preserves_bad_clear_if_fresh</code>	fresh programming does not c
<code>counted_lazy_rom_bad_false_when_clear</code>	the clear invariant excludes th

12.6 From chosen-message to no-message structure

After transcript and ROM internalization, the signing oracle is represented by logged samples. The proof then factors the active signing interaction into a no-message attack against a simulated transcript distribution. At the paper level this says that the signing oracle has been replaced by a sampler whose outputs are independent of the final forgery except through the public transcript and ROM state.

Let G_5 be the ROM-internal NMA game obtained from the chosen-message game:

$$G_5 = \text{ROMInternalTranscriptAsNMA}.$$

The generic exact adapter is certified by `nma_as_cma_exact`, and the HAETAE-specific chain is certified by `euf_cma_security_from_structural_nma_and_counted_bimodal`, `euf_cma_security_from_public_sim_nma_and_counted_bimodal`, and `euf_cma_security_from_paper_sim_nma_and_counted_bimodal`.

Thus the paper inequality is

$$\Pr[G_4(A) \Rightarrow 1] \leq \Pr[G_5(A') \Rightarrow 1] + L_{\text{NMA}},$$

where A' is the induced no-message adversary.

12.7 Rejection sampling and the paper sampler

The paper proof next replaces the real rejection-sampling view by the paper sampler view. This step is justified by the fact that the signing attempt state contains all fields needed to reconstruct an accepted signature:

$$(y_1, y_2, w, c, z, h, \rho) \mapsto \sigma.$$

The paper claim is that the sampler distributions are related up to the rejection-sampling loss:

$$\Pr[G_5(A') \Rightarrow 1] \leq \Pr[G_6(A') \Rightarrow 1] + L_{\text{rej}}.$$

The checked anchors are `signing_attempt_verifies`, `signing_simulation_verifies`, `signing_attempt_simulates`, `structural_signing_sample_pair_sampler_ok`, `bounded_unit_signing_sample_pair_sampler_ok`, `checked_hyperball_signing_sample_pair_sampler_ok`, and `exact_hyperball_signing_sample_pair_sampler_ok`.

In the top-level chain this is reflected by `euf_cma_security_from_real_signing_paper_sim_nma_and_counted_bimodal`, `euf_cma_security_from_haetae_rejection_paper_sim_nma_and_counted_bimodal`, and `exact_hyperball_haetae_rejection_nma_bound_from_paper_sim`.

12.8 Exact hyperball sampling

The exact hyperball game replaces the checked paper sampling distribution by the ideal exact distribution on the hyperball. The mathematical step is

$$\Pr[G_6(A') \Rightarrow 1] \leq \Pr[G_7(A') \Rightarrow 1] + L_{\text{hyp}}.$$

The distributional facts are encoded in `HAETAETAE_Distributions.ec`; the proof uses losslessness, support, and point bound lemmas for the relevant signing sample distributions.

The security chain exposes this step through `haetae_rejection_nma_bound_from_exact_hyperball`, `haetae_rejection_exact_hyperball_sampling_hop_from_real_signing_exact_hyperball_h`, and `haetae_rejection_ro_exact_hyperball_sampling_hop_from_real_signing_ro_exact_h`.

12.9 Tight RO-signing-attempt to RO-exact-hyperball lifting

The most delicate part of the proof is the transition from the RO signing attempt sampler to the RO exact hyperball sampler. Let R be fresh signing randomness sampled from μ_{coins} , and define the sampler query

$$q_R = \text{sampler_expand_query}(R).$$

Let Q_A be the adversary's logged hash-query set. The clean event is

$$\text{Clean} = \{q_R \notin Q_A\} \wedge \{q_R \notin Q_{\text{sampler}}\} \wedge \text{sampler_rom_covered} \wedge \text{budget discipline}.$$

Here Q_{sampler} is the self-log of sampler-expansion queries from earlier signing calls. The checked event `sampler_bad_prequery` is set if the fresh sampler query is already in either log. On `Clean`, one signing call in the two games is coupled so that its visible output is identical. Therefore the local fundamental-lemma contribution of that call is bounded by

$$\Pr[q_R \in Q_A \cup Q_{\text{sampler}}].$$

Since R is fresh, Q_A contains at most q_H logged adversary hash queries, and Q_{sampler} contains at most q_S sampler self-log queries over the whole budgeted execution, the

one-call loss is

$$\Pr[q_R \in Q_A \cup Q_{\text{sampler}}] \leq (q_H + q_S) \cdot \max_r \Pr[R = r].$$

The displayed inequality is a local signing-call bound. The full G_7 to G_8 hop must also union over the at most q_S signing calls, so the full-game sampler-prequery contribution is

$$|\Pr[G_7(A') \Rightarrow 1] - \Pr[G_8(A') \Rightarrow 1]| \leq q_S(q_H + q_S) \cdot \max_r \Pr[R = r].$$

In the EasyCrypt development the point bound on R is supplied by the signing-randomness distribution lemmas and the budgeted support lower bound, yielding the checked full-game term

$$\text{rom_signature_query_budget} \cdot \frac{\text{rom_hash_query_budget} + \text{rom_signature_query_budget}}{\text{challenge_support_cardinality_lower_bound}}.$$

The NMA lifting lemmas separately carry `rom\_signature\_query\_budget` · `rejection\_sampling\_loss\_term`; this is distinct from the sampler-prequery loss above.

The checked anchors are `rom\_internal\_budgeted\_nma\_ro\_signing\_attempt\_ro\_exact\_hyperball\_cl`, `rom\_internal\_budgeted\_nma\_ro\_signing\_attempt\_ro\_exact\_hyperball\_loss\_from\_paper\_sample`, `concrete\_budgeted\_self\_logged\_signature\_clean\_event\_loss\_surface`, `concrete\_budgeted\_o\_sign\_clean\_one\_call\_loss\_from\_self\_logged\_surface`, and the active projection lemmas for the attempt side and exact side.

The top-level security file uses this lifting through `real\_signing\_ro\_exact\_hyperball\_hop\_from\_budgeted\_lifting`, `haetae\_rejection\_ro\_exact\_hyperball\_sampling\_hop\_from\_budgeted\_lifting`, `real\_signing\_ro\_exact\_hyperball\_hop\_from\_paper\_sample\_lifting`, and `euf\_cma\_security\_from\_budgeted\_paper\_sample\_lifting\_and\_counted\_bimodal`.

12.10 Forgery extraction and hardness reduction

In the final exact-hyperball NMA game, extraction is a forking-pair special-soundness step, not a conclusion from one valid transcript alone. When the preceding game and reduction interfaces provide two compatible accepting transcripts with the required shared public data and distinct challenges, the local extraction obligation has the

following form:

$$\text{ValidForkingPair}(\text{tr}_1, \text{tr}_2) \implies \text{Solve}_{\text{Bimodal/MSIS}}.$$

The transcript-to-relation facts are checked by `extract_bimodal_forking_some`, `extract_bimodal_forking_nonempty`, `valid_forking_pair_public_equations`, and `bimodal_to_module_sis_valid`.

The reduction adapters are exact in the sense that they preserve the success event of the induced solver:

$$\Pr[\text{BimodalSolver}(A) \Rightarrow 1] \leq \Pr[\text{ModuleSIS}(B_A) \Rightarrow 1].$$

The checked anchors are `bimodal_solver_lift_exact`, `bimodal_mode_solver_lifted_distrib` and `bimodal_msis_as_module_sis_exact`. The top-level composition uses `bimodal_to_module_sis_reduction_bound` and the counted-bimodal security lemmas in `HAETAE_Security.ec`.

12.11 Composing the inequalities

Combining the checked counted-ROM path under the final hop/reduction premise and the MLWE and Module-SIS hardness premises gives

$$\begin{aligned} \text{Adv}_{\text{ECModel,ROM}}^{\text{EUF-CMA}}(A) &= \Pr[G_0(A) \Rightarrow 1] \\ &\leq \Pr[G_8(A') \Rightarrow 1] + L_{\text{ROM}} \\ &\leq L_{\text{MLWE}} + L_{\text{ModSIS}} + L_{\text{bimodal} \rightarrow \text{MSIS}} + L_{\text{ROM}} \end{aligned}$$

Here L_{ROM} is the counted collision, prequery, and min-entropy loss. Rejection, exact-sampling, and active-signing lifting obligations are real proof obligations in the hop chain, but in the counted theorem they are pushed into explicit hypotheses and bridge lemmas rather than appearing as additional final summands. This is not the same as proving that the hypotheses are non-vacuously satisfiable. In the current EasyCrypt constants, `rejection_sampling_loss_term` is already larger than one, so a final antecedent containing $\Pr[\cdot] + \text{rejection_sampling_loss_term} \leq \Pr[\cdot] + \Pr[\cdot]$ must be treated as a vacuity warning until the rejection ledger is normalized or the antecedent is replaced by a stronger checked hop. The separate expression `haetae_euf_bound` records the older paper-style rejection/Fiat–Shamir-with-aborts ledger; it should not be added to `haetae_euf_counted_rom_bound`. The

real-arithmetic side conditions required to make this a probability-scale bound are checked in `HAETAE_Reductions.ec`:

<code>counted_rom_programming_loss_term_nonnegative</code>	$L_{\text{ROM}} \geq 0$,
<code>counted_rom_programming_loss_term_subunit</code>	$L_{\text{ROM}} \leq 1$,
<code>challenge_support_cardinality_lower_bound_positive</code>	$ \mathcal{C} > 0$,
<code>counted_rom_support_dominates_budget</code>	the support lower bound dominates the
<code>counted_rom_loss_from_query_budgets_and_support</code>	the counted ROM loss follows from the

Finally, `euf_cma_security_from_checked_ro_sampling_hop_and_counted_bimodal` and `euf_cma_security_from_real_signing_exact_hyperball_hop_and_counted_bimodal` compose the checked sampling hop with the counted bimodal reduction. This is the fixed-ledger formal-model implication reconstructed at the beginning of the chapter, subject to the explicit antecedents and vacuity warning above.

12.12 What this reconstruction does and does not claim

This chapter is a pen-and-paper reconstruction of the proof. It does not add a new EasyCrypt theorem. The machine-checked content remains the EasyCrypt files listed in the provable-security manifest. The contribution of this chapter is expository: it recovers the mathematical hybrid proof from the checked lemma graph and records which EasyCrypt lemma certifies each paper step.

The proof remains conditional on the intended interpretation of the abstract hardness games and on the fidelity of the HAETAE model to the specification. Within those boundaries, the game hops, sampler transformations, ROM programming steps, bad-event bounds, and final arithmetic composition are machine checked by the cited EasyCrypt development.

Chapter 13

Mathematical and Cryptological Synthesis

This chapter gives the central mathematical reading of the machine-checked HAETAE security proof. Its purpose is to make the checked implication legible to a cryptologist: the game experiment, the exact bound expression, the hardness ledger, the vacuity checks, the local non-vacuity evidence, and the remaining trusted base are stated in mathematical form and connected to their EasyCrypt names.

The proof is conditional. EasyCrypt checks the game transformations, random-oracle programming, sampler freshness arguments, active-signing lifting, and algebraic composition of loss terms. It does not prove MLWE hardness, Module-SIS hardness, or implementation fidelity outside the formal model. The correct theorem is therefore a theorem about the modeled HAETAE random-oracle scheme.

13.1 Formal security object

Let A be a chosen-message adversary with access to a random oracle H and a signing oracle O . The random-oracle EUF-CMA experiment samples

$$(pk, sk) \leftarrow \text{KeyGen}^H(),$$

answers signing queries by $\text{Sign}^H(sk, \cdot)$, answers hash queries by lazy sampling, and receives a final candidate

$$(m^*, ctx^*, \sigma^*) \leftarrow A^{H,O}(pk).$$

The success event is

$$\text{Forge} = \left[\text{Verify}^H(pk, m^*, ctx^*, \sigma^*) = 1 \wedge (m^*, ctx^*) \notin Q_S \right],$$

where Q_S is the signing-query log. For the informal target HAETAE scheme, the advantage is

$$\text{Adv}_{\text{HAETAE,ROM}}^{\text{EUF-CMA}}(A) = \Pr[\text{Forge}].$$

In EasyCrypt, the generic experiment is supplied by `Sig_ROM.ec`; the HAETAE instantiation is supplied by `HAETAE_Scheme.ec`; and the hop sequence is supplied by `HAETAE_HopGames.ec` and `HAETAE_Security.ec`. The checked theorem surface has the implication shape

$$\mathcal{H}_{\text{hop}}(A, B_{\text{MLWE}}, B_{\text{bimodal}}) \wedge \mathcal{H}_{\text{hard}}(B_{\text{MLWE}}, B_{\text{bimodal}}) \implies \text{Adv}_{\text{ECModel,ROM}}^{\text{EUF-CMA}}(A) \leq \text{haetae_euf_counted_rom_b}$$

The hypotheses are the checked hop inequality that reaches the MLWE and bimodal self-target MSIS games, together with the declared hardness bounds for the resulting reduction games. The right-hand side is not a placeholder. It is an explicit EasyCrypt operation whose definition is normalized in `HAETAE_Reductions.ec`. The constants $q_H = q_S = 2$ belong to that bound ledger and to the budgeted wrapper lemmas, not to the syntax of the source EUF-CMA experiment in `Sig_ROM.ec`.

13.2 Exact checked bound expression

The relevant EasyCrypt definitions are the following.

```

1  op mlwe_hardness_term : real.
2  op module_sis_hardness_term : real.
3  op bimodal_to_msis_reduction_loss_term : real = 0%r.
4
5  op bimodal_selftarget_msis_hardness_term =
6      module_sis_hardness_term +
7          bimodal_to_msis_reduction_loss_term.
```

```

8  op signature_query_count : real = 2%r.
9  op hash_query_count : real = 2%r.
10 op signature_query_budget_count : int = 2.
11 op hash_query_budget_count : int = 2.
12 op challenge_support_cardinality_lower_bound : real =
13     288230376151711744%r.
14
15 op rom_hash_query_budget : real = hash_query_count.
16 op rom_signature_query_budget : real =
17     signature_query_count.
18 op rom_total_query_budget =
19     rom_hash_query_budget + rom_signature_query_budget + 1%r
20     .
21 op counted_rom_collision_term =
22     (rom_total_query_budget * rom_total_query_budget) /
23     challenge_support_cardinality_lower_bound.
24
25 op counted_rom_prequery_term =
26     (rom_signature_query_budget * (rom_hash_query_budget +
27     1%r)) /
28     challenge_support_cardinality_lower_bound.
29
30 op counted_rom_min_entropy_term =
31     (rom_signature_query_budget * (rom_hash_query_budget +
32     1%r)) /
33     challenge_support_cardinality_lower_bound.
34
35 op counted_rom_programming_loss_term =
36     counted_rom_collision_term +
37     counted_rom_prequery_term +
38     counted_rom_min_entropy_term.
39
40 op haetae_euf_counted_rom_bound =
41     mlwe_hardness_term +
42     bimodal_selftarget_msis_hardness_term +
43     counted_rom_programming_loss_term.

```

Listing 13.1: Checked bound definitions in HAETA_E_Reductions.ec.

Mathematically, let

$$q_H = \text{hash_query_count} = 2, \quad q_S = \text{signature_query_count} = 2,$$

and let

$$N = \text{challenge_support_cardinality_lower_bound} = 288230376151711744 = 2^{58}.$$

Then

$$L_{\text{coll}} = \frac{(q_H + q_S + 1)^2}{N}, \quad L_{\text{pre}} = \frac{q_S(q_H + 1)}{N}, \quad L_{\text{me}} = \frac{q_S(q_H + 1)}{N}.$$

For the current constants this gives

$$L_{\text{ROM}} = L_{\text{coll}} + L_{\text{pre}} + L_{\text{me}} = \frac{25 + 6 + 6}{2^{58}} = \frac{37}{2^{58}}.$$

This concrete value should not be mistaken for a fully general EUF-CMA query-bound theorem. It is the concrete ROM contribution used by the theorem family checked in `HAETAE_Security.ec`; the constants and arithmetic normalizations are defined in `HAETAE_Reductions.ec`, whose public constants set

$$\text{hash_query_budget_count} = 2, \quad \text{signature_query_budget_count} = 2.$$

The reusable algebraic shape is

$$\frac{(q_H + q_S + 1)^2}{N} + \frac{q_S(q_H + 1)}{N} + \frac{q_S(q_H + 1)}{N},$$

but the machine-checked implication documented here instantiates that expression at the two-hash-query, two-signing-query wrapper constants. A future generalized theorem should promote q_H , q_S , and N from fixed operations to symbolic parameters and recheck all monotonicity and subunit obligations.

Under the hop and hardness hypotheses of the final EasyCrypt theorem family, the expanded checked counted-ROM conclusion is therefore

$$\boxed{\text{Adv}_{\text{ECModel,ROM}}^{\text{EUF-CMA}}(A) \leq L_{\text{MLWE}} + L_{\text{ModSIS}} + L_{\text{bimodal} \rightarrow \text{MSIS}} + \frac{37}{2^{58}}}$$

where

$$L_{\text{MLWE}} = \text{mlwe_hardness_term},$$

$$L_{\text{ModSIS}} = \text{module_sis_hardness_term},$$

and

$$L_{\text{bimodal} \rightarrow \text{MSIS}} = \text{bimodal_to_msis_reduction_loss_term}.$$

The current EasyCrypt file defines the last term as 0, but it is still listed separately because it records a proof-design decision: bimodal self-target MSIS is grouped with Module-SIS through `bimodal_selftarget_msis_hardness_term`.

For precision, this result should be cited as a checked fixed-ledger implication for the formal EasyCrypt model, with the bound constants instantiated at $q_H = q_S = 2$, conditional on the EasyCrypt antecedents of the selected composition lemma. It is not yet the conventional all-polynomial-time, arbitrary-query EUF-CMA theorem. The dissertation's strongest checked claim is

$$\mathcal{H}_{\text{EC}} \implies \text{Adv}_{\text{ECModel,ROM}}^{\text{EUF-CMA}}(A) \leq \text{haetae_euf_counted_rom_bound}.$$

Here $\mathcal{H}_{\text{EC}}$ abbreviates the explicit antecedents of the EasyCrypt composition lemma being cited: the final hop/reduction inequality and the MLWE and Module-SIS hardness bounds for the declared reduction modules supplied to the theorem. The final composition lemma does not itself construct those MLWE or Module-SIS solvers from A ; it composes the explicitly stated probability and hardness premises. One of those antecedents is currently a proof-surface premise rather than a useful concrete-security premise: `fs_with_aborts_min_entropy_term` is defined as 12, and the current sources prove `rejection_sampling_loss_term` ≥ 12 . Therefore a final premise that adds `rejection_sampling_loss_term` to a probability cannot be satisfied by the sum of two game-success probabilities without further repair to the premise or loss normalization. Any statement for larger or symbolic budgets is a proposed generalization unless the corresponding EasyCrypt constants and lifting lemmas are replaced by parameterized versions and rechecked.

Theorem 13.1 (Checked scoped counted-ROM implication). *For every adversary satisfying the declared EasyCrypt module interfaces, with the checked proof and bound ledger instantiated through the fixed wrapper constants $q_H = q_S = 2$, assume the final checked hop/reduction premise*

$$\begin{aligned} & \Pr[\text{UF_NMA}_{\text{ROExactHyperballPaperSim}}(A) \Rightarrow 1] + \text{rejection_sampling_loss_term} \\ & \leq \Pr[\text{MLWE_Game}(B_{\text{MLWE}}) \Rightarrow 1] + \Pr[\text{BimodalSelfTargetMSIS_Game}(B_{\text{bimodal}}) \Rightarrow 1], \end{aligned}$$

and the hardness premises

$$\Pr[MLWE_Game(B_{MLWE}) \Rightarrow 1] \leq mlwe_hardness_term$$

and

$$\Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(B_{bimodal})) \Rightarrow 1] \leq module_sis_hardness_term.$$

Then the machine-checked implication establishes the probability-only random-oracle-model conclusion for the formal EasyCrypt scheme model `ECModel`:

$$\begin{aligned} Adv_{ECModel, ROM}^{EUF-CMA}(A) &\leq mlwe_hardness_term + module_sis_hardness_term \\ &\quad + bimodal_to_msis_reduction_loss_term + counted_rom_collision_term \\ &\quad + counted_rom_prequery_term + counted_rom_min_entropy_term. \end{aligned}$$

Equivalently, under the concrete constants currently present in `HAETAE_Reductions.ec`, the random-oracle programming contribution is $37/2^{58}$.

This implication does not include a checked equivalence $Spec_{260204} \equiv ECModel$. Reading it as a theorem about the February 2026 HAETAE specification therefore additionally relies on the audited correspondences in Table 4.3 and on the signing-randomness modeling bridge in Assumption 4.1. The implication also does not instantiate SHAKE in the standard model, derive concrete MLWE/Module-SIS bit-security estimates, or check the running time of the reduction. Those items are boundary obligations recorded elsewhere in the dissertation.

Vacuity status. The checked implication above is accepted by EasyCrypt, but the current final-hop antecedent is not a non-vacuous concrete-security premise. In `HAETAE_Reductions.ec`, `fs_with_aborts_min_entropy_term` is 12 and the checked facts imply `rejection_sampling_loss_term` ≥ 12 . Since the right-hand side of the displayed final-hop antecedent is a sum of two game-success probabilities, it is at most 2. The checked artifact therefore records the composition that would follow if that hop premise were supplied; it should not be cited as a useful concrete security theorem until the final-hop premise or the rejection loss normalization is repaired.

Dependency form of the theorem. The checked implication has the logical shape

$$\text{Checked}_{\text{EasyCrypt}} \vdash \mathcal{H}_{\text{EC}} \implies \text{Adv}_{\text{ECModel}, \text{ROM}}^{\text{EUF-CMA}}(A) \leq \text{haetae_euf_counted_rom_bound}.$$

The specification-level reading has the strictly stronger, conditional shape

$$\text{Checked}_{\text{EasyCrypt}} + \mathcal{H}_{\text{EC}} + \text{Assumption}_{\text{sign-coins}} + \text{Audit}_{\text{Spec} \leftrightarrow \text{ECModel}} \implies \text{Adv}_{\text{Spec}_{260204}, \text{ROM}}^{\text{EUF-CMA}}(A) \lesssim \text{haetae}$$

The symbol $\lesssim$ is intentional: the dissertation does not supply a machine-checked theorem equating the specification game with the EasyCrypt game. The dependency form is the honest implication boundary and should be used whenever the result is cited outside the formal-model setting.

The earlier broader ledger containing separate final summands for sampling, rejection, extraction, and budget lifting is not the final counted-ROM theorem. Those proof ideas are real, but in the checked counted theorem they are internalized into the hop lemmas, the bimodal/MSIS reduction term, and the ROM programming terms. Treating them as additional final summands would double count proof obligations that the EasyCrypt development has already composed.

13.3 Hardness-term classification

The reduction file separates computational assumptions from proof-accounting losses. The classification is as follows.

Table 13.1: Final counted-ROM bound terms.

Mathematical term	EasyCrypt identifier	Status in the proof
L_{MLWE}	<code>mlwe_hardness_term</code>	Abstract hardness term. The security theorem is conditional on this term being small for the relevant parameter set.
L_{ModSIS}	<code>module_sis_hardness_term</code>	Abstract hardness term for the Module-SIS layer.

Mathematical term	EasyCrypt identifier	Status in the proof
$L_{\text{bimodal} \rightarrow \text{MSIS}}$	<code>bimodal_to_msis_reduction_loss_term</code>	Explicit term, reduction-loss slot, currently defined as 0 in the EasyCrypt source.
L_{coll}	<code>counted_rom_collision_term</code>	Probability-accounting loss for collisions among programmed or observed ROM sites.
L_{pre}	<code>counted_rom_prequery_term</code>	Counted-ROM programmed-site prequery loss, with numerator <code>rom_signature_query_budget(rom_hash_query_1)</code> .
L_{me}	<code>counted_rom_min_entropy_term</code>	Counted-ROM challenge min-entropy ledger term for the <code>bad_min_entropy</code> predicate on observed transcripts; distinct from sampler-prequery freshness.

The table deliberately separates symbolic hardness terms from checked probability-accounting terms. The dissertation does not derive an independent bit-security estimate for MLWE or Module-SIS at HAETAE levels 2, 3, and 5. Those estimates belong to the cryptanalytic parameter analysis of the scheme. The EasyCrypt theorem states that any such estimates may be inserted for `mlwe_hardness_term` and `module_sis_hardness_term`; it does not prove the estimates themselves.

The counted-ROM prequery term should not be identified with the separate `sampler_bad_prequery` wrapper event. The latter is set when a fresh sampler-expansion query is already present either in the adversary hash log or in the sampler self-log from earlier signing calls, and its checked budgeted bound uses

$$\text{rom_signature_query_budget} \cdot \frac{\text{rom_hash_query_budget} + \text{rom_signature_query_budget}}{\text{challenge_support_cardinality_lower_bound}}.$$

That sampler-bad-prequery fact is part of the local active-signing and

lifting evidence; it is not the `counted_rom_prequery_term` summand in `haetae_euf_counted_rom_bound`. Likewise, `counted_rom_min_entropy_term` belongs to the counted-ROM transcript ledger for challenge-entropy failures; it is not the support bound for the sampler self-log event.

Consequently, the dissertation should not be read as a concrete NIST-level security estimate. It verifies the reduction ledger into symbolic MLWE and Module-SIS assumptions. A complete concrete-security chapter would additionally need parameter-specific estimator evidence, assumption normalization, and a clear rule for combining estimator costs with the checked ROM and reduction losses.

The companion operation `haetae_euf_bound` uses `rejection_sampling_loss_term`; it is useful for paper-style abort and reprogramming analysis. The theorem emphasized here is instead the counted-ROM variant `haetae_euf_counted_rom_bound`. The dissertation should not conflate these two theorem shapes.

13.4 Denotational reading of the proof scripts

Each EasyCrypt procedure denotes a subprobability kernel. If S is a state space and R is the return space of a procedure c , then

$$\llbracket c \rrbracket : S \rightarrow \mathcal{D}(S \times R).$$

Losslessness lemmas assert that the kernel has total mass one. An ordinary Hoare judgment asserts partial correctness: every terminating outcome in the support of the kernel satisfies the postcondition,

$$\{P\} c \{Q\} \iff \forall s. P(s) \Rightarrow \text{supp}(\llbracket c \rrbracket(s)) \subseteq \{(s', r) \mid Q(s', r)\}.$$

When c is additionally lossless, this support-inclusion statement can be read as probability-one postcondition preservation:

$$\forall s. P(s) \Rightarrow \Pr_{(s', r) \leftarrow \llbracket c \rrbracket(s)}[Q(s', r)] = 1.$$

An equivalence judgment

$$\mathbf{equiv}[c_1 \sim c_2 : P \Rightarrow Q]$$

asserts a relational coupling between the two kernels. Thus an EasyCrypt game hop is not merely a syntactic rewrite; it is a checked probabilistic relation between two

experiments.

The proof transitions have three mathematical forms:

$$\Pr[G_i : E_i] = \Pr[G_{i+1} : E_{i+1}],$$

$$\Pr[G_i : E_i] \leq \Pr[G_{i+1} : E_{i+1}] + \Pr[G_i : \text{Bad}_i],$$

and

$$\Pr[G_i : E_i] \leq B_i(A).$$

These correspond respectively to perfect couplings, fundamental-lemma transitions, and direct reduction or probability bounds.

13.5 Local clean-conditioned evidence and NMA-lifting gap

A formally checked lemma can still be cryptologically useless if its hypotheses are never established. The development contains local evidence against that failure mode through concrete invariants over `ROMInternalTranscriptBudgetedPaperSimAsNMA.0.sign`. The clean condition contains the absence of a sampler prequery, sampler-ROM coverage, public-key consistency, and current-secret-key consistency. The top-level NMA path, however, still uses some coarse fallback bounds proved from probability boundedness and `rejection_sampling_loss_term ≥ 1`. This section therefore records local clean-conditioned evidence and the remaining NMA-lifting gap, not a non-vacuous final-hop theorem.

The concrete lazy-ROM freshness laws are exposed by the following checked lemma headers.

```

1 lemma concrete_lazy_rom_sampler_expand_query_fresh_le
2   seed_coins (p : random_coins -> bool) &m :
3   sampler_expand_query seed_coins \notin HAETAE_ROM.FROM.m{m
4     } =>
5   Pr[HAETAE_ROM.FROM.get(sampler_expand_query seed_coins) @
6     &m :
7     p (ro_signing_coins res)] <=
8   mu drandom_coins p.
```

```

8 lemma concrete_lazy_rom_sampler_expand_query_fresh_eq
9   seed_coins (p : random_coins -> bool) :
10   phoare[HAETAE_RO.FRO.get :
11     arg = sampler_expand_query seed_coins /\
12     sampler_expand_query seed_coins \notin HAETAE_RO.FRO.m
13     ==> p (ro_signing_coins res)] =
14   (mu drandom_coins p).

```

Listing 13.2: Concrete lazy-ROM sampler freshness laws.

Mathematically, if $r \leftarrow D_{\text{coins}}$ is fresh and $q = \text{sampler_expand_query}(r)$ is not already in the lazy-ROM table, then the distribution of $\text{ro_signing_coins}(H(q))$ is bounded by the ideal coin distribution. This is the exact statement needed to replace a bare freshness assumption by a concrete lazy-ROM law.

The active signing invariant is recorded by the following theorem header.

```

1 lemma
2   concrete_budgeted_ro_signing_attempt_o_sign_clean_sampler_fresh
3   :
4   hoare[ROMInternalTranscriptBudgetedPaperSimAsNMA
5     (A, ROSigningAttemptPaperSimSampler(HAETAE_RO.
6       FRO),
7       HAETAE_RO.FRO).0.sign :
8     sampler_rom_covered
9     HAETAE_RO.FRO.m
10    ROMInternalTranscriptBudgetedPaperSimAsNMA.
11    adversary_hash_queries
12    ROMInternalTranscriptBudgetedPaperSimAsNMA.
13    sampler_expand_queries /\
14    ! ROMInternalTranscriptBudgetedPaperSimAsNMA.
15    sampler_bad_prequery ==>
16    sampler_rom_covered
17    HAETAE_RO.FRO.m
18    ROMInternalTranscriptBudgetedPaperSimAsNMA.
19    adversary_hash_queries
20    ROMInternalTranscriptBudgetedPaperSimAsNMA.
21    sampler_expand_queries].

```

Listing 13.3: Sampler-ROM coverage preservation for one active signing call.

The one-call loss theorem is then applied through the concrete surface lemma

`concrete_budgeted_o_sign_clean_one_call_loss_from_self_logged_surface`, whose conclusion has the form

$$\begin{aligned} & \Pr[O_{\text{attempt}}.\text{sign}(m, ctx) : p(res) \wedge \neg \text{sampler_bad_prequery}] \\ & \leq \Pr[O_{\text{exact}}.\text{sign}(m, ctx) : p(res)] + \text{rejection_sampling_loss_term}. \end{aligned}$$

This displayed conclusion is conditional on the checked premises in the EasyCrypt lemma: the structural sample-loss obligation, sampler-ROM coverage, absence of the current sampler-prequery event, an active signing budget, current public-key and secret-key consistency, and the attempt-to-sample projection premise. This is the local theorem surface that a non-vacuous active-signing lift should consume. The present top-level NMA lemmas do not yet fully use this surface as their final proof path; some later bridge lemmas still fall back to coarse probability bounds whose usefulness depends on the oversized `rejection_sampling_loss_term`. Future maintenance should replace those fallbacks by a final NMA-level composition that consumes the clean one-call surface directly.

13.6 Wrapper counts and runtime cost

The checked implication uses a counted-ROM bound ledger and budgeted wrapper lemmas with fixed hash-query and signing-query wrapper constants. In the current reduction file these constants are fixed to

$$q_H = 2, \quad q_S = 2.$$

These constants account for the wrapper layer; they are not syntactic query caps on the source `Sig_ROM.EUF_CMA` game. The budgeted wrapper proof exposes the state needed to lift one-call clean losses through adaptive oracle interaction without replacing the proof by an unstructured union-bound envelope. The remaining obligation is to make the final NMA-level composition use that structure directly instead of relying on coarse fallback bounds.

Running time is different. The EasyCrypt theorem checked here does not contain a cost semantics for the reduction. The paper-level runtime statement should therefore be written as an external interpretation ledger:

$$\text{Time}(B_A) \leq \text{Time}(A) + T_{\text{ROM}}(q_H) + T_{\text{sign}}(q_S) + T_{\text{extract}}(q_H, q_S).$$

Here T_{ROM} is lazy-table maintenance, T_{sign} is simulated signing and sampler instrumentation, and T_{extract} is the final reduction/extraction overhead. This ledger is not a checked theorem unless a future cost-aware EasyCrypt development adds explicit cost judgments. Consequently, the formal theorem statement in this dissertation should be read as a probability theorem with query-budget accounting, not as a theorem about the concrete running time of the declared reduction modules.

13.7 Trusted base

The trusted base has five parts:

1. EasyCrypt’s logic, libraries, module system, and probability semantics.
2. The manifest discipline selecting the files in `provable-security/proof-files.txt`.
3. The interpretation of `mlwe_hardness_term` and `module_sis_hardness_term` as external lattice-hardness assumptions.
4. The model/spec correspondence between `HAETAE_v260204.pdf` and the formal scheme model.
5. The fact that implementation-level byte parsing, arithmetic, hashing, and constant-time behavior are outside the central provable-security theorem.

This is the precise scope of the dissertation claim: the formal HAETAE ROM proof-surface implication is machine-checked, and the remaining assumptions are stated at the boundary rather than hidden inside the proof narrative.

Chapter 14

Reproducibility and Evaluation

14.1 Build and Proof Commands

The dissertation can be built from the `dissertation/` directory with:

```
1 make
```

The EasyCrypt declaration inventory used in Appendix [B.1](#) is regenerated with:

```
1 make inventory
```

This writes `etc/easycrypt-declaration-counts.csv`. The target is not part of the default PDF build because it inspects the separated proof manifest rather than LaTeX sources.

The EasyCrypt proof gate is checked from `haetae-security/` with:

```
1 EC_ATTEMPTS=8 ./provable-security/verify-provable-security
  .sh
```

A successful proof run ends with `RESULT: PASS`.

14.2 Why Attempts Are Configurable

Long EasyCrypt batches may occasionally fail because the Why3 server cannot be started or contacted. The verification script retries files, but accepts a file only if

EasyCrypt eventually returns success for that file. This distinguishes infrastructure lifecycle failures from real failed proof obligations.

14.3 Evaluation Criteria

A reviewer should check manifest completeness, manifest minimality, successful compilation, theorem traceability, and assumption transparency. The separated folder is designed to support exactly that review.

Chapter 15

Conclusion

This dissertation presents a machine-checked EasyCrypt proof surface for the provable security of the formal HAETAE random-oracle signature model. The word “formal” is essential: the checked theorem surface is a conditional implication about the EasyCrypt model, together with an audited correspondence to the February 2026 HAETAE specification. The proof is organized as a game-based reduction with explicit oracle state, transcript state, sampler state, bad events, query budgets, and algebraic loss composition.

The main contribution is the conversion of informal proof phrases into checked mathematical objects. Games are probabilistic programs. Random oracles are stateful lazy tables. Hybrid transitions are equivalence judgments or probability inequalities. Bad events are Boolean predicates over concrete game state. Active signing has local clean-conditioned one-call proof surfaces and wrapper invariants; the final NMA-level lift still has to avoid coarse fallback bounds before it can be read as a non-vacuous active-signing theorem.

The revised dissertation also separates the checked implication from its boundaries. The result is conditional on the hardness terms represented in `HAETAE_Assumptions.ec`, assembled in `HAETAE_Reductions.ec`, and on the explicit hop/reduction antecedents of the final EasyCrypt composition lemma. One of those antecedents is currently vacuous under the present `rejection_sampling_loss_term` constants, so the central artifact is a ROM-model implication rather than a non-vacuous concrete security theorem. It also does not prove implementation correctness of all concrete parsing, arithmetic, hashing, and rejection-sampling code paths.

Evidence summary

Evidence item	Status recorded in this dissertation
Specification source	HAETAE_v260204.pdf, 46 pages, generated February 4, 2026; Section 4.2, Figure 8, and Table 1 are used for model/spec correspondence.
Proof manifest	provable-security/proof-files.txt; recorded inventory: 16 EasyCrypt files and 22046 checked-source lines.
Main checked bound	haetae_euf_counted_rom_bound; the right-hand side of the conditional probability-only ROM implication for ECModel , with the proof and bound ledger instantiated at $q_H = q_S = 2$, expanded as MLWE hardness, Module-SIS hardness, bimodal-to-MSIS reduction loss, and counted ROM collision/prequery/min-entropy terms.
Strongest exact claim	A checked implication whose conclusion is a probability bound for ECModel in the ROM; the checked bound ledger uses $q_H = q_S = 2$. The bridge from Spec₂₆₀₂₀₄ to ECModel is documented by audit tables and by the explicit signing-randomness modeling assumption in Assumption 4.1, not by a separate checked equivalence theorem. The current final-hop premise contains rejection_sampling_loss_term , which is larger than one under the present constants, so this is a proof-surface implication and not a non-vacuous concrete instantiation.
Concrete counted-ROM loss	With the current constants in HAETAE_Reductions.ec , $q_H = 2$, $q_S = 2$, $N = 2^{58}$, and the counted ROM contribution is $37/2^{58}$.

Local non-vacuity evidence	Concrete lazy-ROM sampler freshness, sampler-ROM coverage, <code>sampler_bad_prequery</code> instrumentation, and active <code>0.sign</code> clean lifting lemmas are identified by name and interpreted mathematically. The final NMA-level path still contains coarse fallback bounds that must be replaced before the top-level antecedent can be called non-vacuous.
Reproducibility aid	<code>dissertation/etc/count-easycrypt-declarations.sh</code> records the counting policy for the proof-code inventory.
Residual obligations	Replacing the vacuous final-hop antecedent with a satisfiable checked hop, mechanizing the model/spec fidelity ledger in Table 4.3, concrete SHAKE instantiation, implementation correctness, concrete MLWE/Module-SIS bit-security estimation, and reduction runtime overhead remain outside the central checked probability implication.

The final lesson is methodological. A machine-checked cryptographic proof is not persuasive merely because it is long or because it compiles. It is persuasive when the theorem statement is exact, the loss ledger is not inflated, vacuous hypotheses are called out, the assumptions are visible, and the model is compared against the specification. The rewritten dissertation is structured around those criteria.

The dissertation should therefore be cited in three layers. First, as a checked EasyCrypt implication and audit of a scoped counted-ROM proof surface for `ECModel`, with the current vacuity warning attached. Second, as an audited specification-model correspondence argument for reading that model as HAETAE v260204. Third, as a roadmap for the remaining work needed for an end-to-end claim: a satisfiable final-hop theorem, mechanized PDF-to-model equivalence, concrete SHAKE instantiation reasoning, implementation correctness, concrete MLWE/Module-SIS bit estimates, and cost-aware reduction accounting. Collapsing these layers would overstate the verification; keeping them separate is the main integrity property of the document.

Future work should connect the formal HAETAE model to implementation-level correctness proofs, especially for parsing, hashing, arithmetic, and rejection sampling. It should also formalize reduction runtime overhead, maintain a versioned

proof-manifest discipline, and update the model/spec correspondence whenever the EasyCrypt proof surface or HAETAE specification changes.

Appendix A

Proof Manifest

The current provable-security manifest is:

```
1  HAETAE_Security.ec
2  Sig_ROM.ec
3  HAETAE_HopGames.ec
4  HAETAE_Reductions.ec
5  HAETAE_ROM.ec
6  HAETAE_ROM_Programming.ec
7  HAETAE_Scheme.ec
8  HAETAE_Transcript.ec
9  HAETAE_Events.ec
10 HAETAE_Assumptions.ec
11 HAETAE_Distributions.ec
12 HAETAE_Rejection.ec
13 HAETAE_Algebra.ec
14 HAETAE_Params.ec
15 HAETAE_FIPS202.ec
16 HAETAE_Keccak1600.ec
```


Appendix B

Detailed EasyCrypt Proof-Code Analysis

This appendix records a file-by-file analysis of the EasyCrypt code that is part of the provable-security verification surface. The source of truth for the analysis is the manifest `provable-security/proof-files.txt`; files not listed there are outside the provable-security gate documented here. In particular, known-answer-test files for concrete FIPS 202 outputs are useful for implementation validation but are not needed for the game-based provable-security argument.

B.1 Manifest-level proof-code inventory

The manifest contains 16 EasyCrypt files and 22046 lines of checked source. The recorded inventory is reproducible with `dissertation/etc/count-easycrypt-declarations.sh`, which fixes the counting policy used for this appendix and reports axioms separately from lemmas. Under the inventory policy used for this dissertation, the scan reports 1088 lemma declarations, 0 axiom declarations, 105 Hoare/equivalence proof commands, 113 module or module-type declarations, 678 operation/predicate declarations, and 63 type declarations. These numbers are a command-oriented inventory, not a semantic theorem count. Broader catalogs that include clones, generated headers, or different Hoare/equiv match rules may legitimately report different totals, but they should cite their counting policy explicitly.

The same scan found no literal occurrence of `axiom`, `admit.`, or `sorry.` in the

copied provable-security proof surface. This does not remove the cryptographic assumptions from the theorem; it means that those assumptions are represented as abstract games, hardness terms, modules, and parameters rather than as unchecked EasyCrypt axioms inside the manifest.

Table B.1: Structural summary of the manifest EasyCrypt files.

File	Lines	Types	Ops	Modules	Lemmas	Axioms	Hoare/equiv
HAETAE_Security.ec	1272	0	1	0	40	0	0
Sig_ROM.ec	206	11	4	16	3	0	0
HAETAE_HopGames.ec	8817	1	40	62	247	0	88
HAETAE_Reductions.ec	504	0	26	0	50	0	0
HAETAE_ROM.ec	111	4	24	2	3	0	0
HAETAE_ROM_Programming.ec	1754	2	43	11	99	0	17
HAETAE_Scheme.ec	66	7	2	1	0	0	0
HAETAE_Transcript.ec	660	1	51	0	35	0	0
HAETAE_Events.ec	57	0	4	0	4	0	0
HAETAE_Assumptions.ec	377	5	13	21	11	0	0
HAETAE_Distributions.ec	1218	2	64	0	97	0	0
HAETAE_Rejection.ec	2204	4	97	0	139	0	0
HAETAE_Algebra.ec	3885	20	246	0	282	0	0
HAETAE_Params.ec	113	1	17	0	19	0	0
HAETAE_FIPS202.ec	105	2	12	0	9	0	0
HAETAE_Keccak1600.ec	697	3	34	0	50	0	0
Total	22046	63	678	113	1088	0	105

The proof surface has four strata. The foundation stratum defines parameters, algebraic data structures, SHAKE/Keccak support, and distributions. The scheme stratum defines the HAETAE random-oracle scheme interface and the signing, verification, transcript, event, and rejection predicates. The game stratum defines EUF-CMA, UF-NMA, ROM, transcript, sampler, and programming games. The reduction stratum composes hop bounds into the final checked implication.

B.2 Trust boundary and checked claims

The machine-checked claim is a conditional game-based security implication. The checked proof establishes that the fixed-ledger EasyCrypt-model EUF-CMA advantage in the modeled random-oracle setting is bounded by the concrete expression assembled in `HAETAE_Reductions.ec`, subject to the explicit final hop/reduction premise, the abstract hardness games, and the parameter assumptions represented in `HAETAE_Assumptions.ec`. One current final-hop premise is a proof-surface premise rather than a non-vacuous concrete-security premise because it contains the over-

sized `rejection_sampling_loss_term`. Thus the checker verifies the game transformations, sampler/event accounting, algebraic well-formedness obligations, and non-negativity/subunit arithmetic needed to compose the final implication. It does not prove Module-SIS hardness, MLWE hardness, the missing non-vacuous final hop, or the physical correctness of an implementation outside the modeled operations.

The absence of manifest-local `axiom`, `admit.`, and `sorry.` occurrences is important because the final implication is not obtained by installing an unchecked theorem statement. The remaining trusted base consists of EasyCrypt’s logic and libraries, the correctness of the manifest selection, the intended interpretation of abstract hardness games, and the fidelity of the HAETAE model to the specification.

B.3 File-by-file analysis

B.3.1 HAETAE_Security.ec

`HAETAE_Security.ec` is the top-level composition file. It imports the signature ROM interface, the HAETAE scheme model, algebra, distributions, reductions, assumptions, rejection logic, transcript logic, ROM programming, and hop games. Its purpose is compositional: it does not define a new low-level model, but connects previously checked lemmas into externally usable security statements.

The first obligation is correctness. The operation `correctness_obligation` packages the required rejection-freeness and verification facts, while `correctness_obligation_holds` and `correctness_perfect` connect those facts to the correctness game. This establishes that the modeled signing algorithm produces signatures accepted by the modeled verifier under the proof assumptions.

The EUF-CMA part is organized as a chain of increasingly explicit theorems. Lemmas such as `euf_cma_security_no_signing`, `euf_cma_security`, and `euf_cma_security_with_hop_interfaces` expose the public theorem at different levels of abstraction. The middle of the file refines the proof through simulated-signing, explicit-boundary, ROM-internal transcript, clear site, counted clear site, structural NMA, public simulation, paper simulation, real-signing paper simulation, HAETAE-rejection paper simulation, exact hyperball, and RO-exact hyperball stages. The final lemmas, including `euf_cma_security_from_paper_sample_lifting_and_counted_bimodal`, `euf_cma_security_from_budgeted_paper_sample_lifting_and_counted_bimodal`,

`euf_cma_security_from_checked_ro_sampling_hop_and_counted_bimodal`, and `euf_cma_security_from_real_signing_exact_hyperball_hop_and_counted_bimodal`, are the checked composition points used by the dissertation security implication.

The code pattern in this file is deliberately conservative. Most lemmas are bound-lifting or theorem-composition lemmas whose proof obligations are arithmetic inequalities and applications of earlier hop lemmas. This makes the top-level implication auditable: each public statement records which lower-level boundary it depends on, and the concrete loss expression remains centralized in `HAETAE_Reductions.ec`.

B.3.2 `Sig_ROM.ec`

`Sig_ROM.ec` defines the generic signature security interface used by the HAETAE proof. It abstracts public keys, secret keys, messages, contexts, signatures, random-oracle queries, and random-oracle outputs. The operations `fresh_msg` and `fresh_sig` define freshness predicates for EUF-CMA and strong-EUF-CMA style games, and the file clones EasyCrypt’s full random-oracle interface through `FullRO`.

The module types `Oracle`, `POracle`, `Scheme`, `CORR_ADV`, `SignOracle`, `Adversary`, and `NMA_Adversary` are the interfaces through which all later HAETAE games are parameterized. The game modules `Correctness`, `EUF_CMA`, `UF_NMA`, and `SUF_CMA` define the standard experiments. The adapter `NMA_As_CMA` embeds a no-message adversary in a chosen-message interface.

The central proof obligation in this file is `nma_as_cma_exact`. It is an equivalence proof showing that the NMA adapter preserves the result of the experiment under equal globals. Later HAETAE reductions rely on this generic lemma when replacing active chosen-message interfaces with passive or simulated transcript interfaces.

B.3.3 `HAETAE_HopGames.ec`

`HAETAE_HopGames.ec` is the largest and most security-critical proof file in the manifest. It contains the detailed game transformations that turn real HAETAE signing into transcript-based and ROM-internal simulators, then into NMA samplers, paper samplers, rejection samplers, exact-hyperball samplers, and budgeted random-oracle lifting games.

The first block defines signing simulators and transcript consistency invariants. Predicates such as `transcript_log_consistent`, `transcript_log_ready`, `internal_transcript_state_sound`, and `sampler_rom_covered` are the invariants that let the proof state that logged transcripts match signatures, public keys, mode data, min-entropy conditions, and random-oracle query coverage. Lemmas including `transcript_log_consistent_cons`, `internal_transcript_state_sound_sign_internal_cons`, and `sampler_rom_covered_fresh_after_clean_seed` are preservation lemmas used in Hoare proofs for signing oracles.

The next major block defines game modules for simulated signing: `EUF_CMA_SimulatedSign`, `EUF_CMA_TranscriptSimulatedSign`, `EUF_CMA_InternalTranscriptSign`, and `EUF_CMA_ROMInternalTranscriptSign`. These modules isolate the semantic changes between real signing and transcript-oriented simulation. The proof obligations here are mainly equivalence and Hoare obligations showing that results and bad events are preserved or bounded when the signing oracle is replaced.

The paper-simulation block defines simulated transcript and signature samples, including public-simulation fields and RO-signing-attempt sampling. Operations such as `paper_sim_sample_token`, `public_sim_signature`, and the RO signing-attempt samplers encode the paper proof's distributional objects in a form that EasyCrypt can reason about. This is the point where the proof maps hand-written sampling arguments into typed programs and event predicates.

The budgeted ROM-internal block is the most delicate part of the file. It defines budgeted wrappers for adversary hash calls and signing calls, logs adversary-side hash queries and sampler self-queries, and introduces sampler-clean events such as `sampler_bad_prequery`. The checked lifting theorems compare `ROSigningAttemptPaperSimSampler` and `ROExactHyperballPaperSimSampler` under clean conditions and query budgets. The important proof idea is a fundamental-lemma argument: the attempt side and exact side are equivalent unless the sampler's fresh `sampler_expand_query` was already present in the joint adversary/self log.

The concentration of 88 Hoare/equivalence commands in this file is expected. Those commands correspond to the stateful game transitions that cannot be captured by pure algebraic rewriting alone. This file is therefore the primary machine-checked replacement for the informal hybrid-game portion of the pen-and-paper proof.

B.3.4 HAETAE_Reductions.ec

HAETAE_Reductions.ec defines the concrete loss terms that appear in the final checked implication. It introduces abstract hardness terms such as `mlwe_hardness_term`, `module_sis_hardness_term`, and `bimodal_selftarget_msis_hardness_term`, then defines query budgets, challenge-support lower bounds, counted ROM collision/prequery/min-entropy terms, rejection-sampling terms, and final EUF bounds.

The code is mostly arithmetic normalization and inequality proof. Lemmas such as `haetae_euf_boundE`, `haetae_euf_bound_groupedE`, `haetae_euf_counted_rom_bound_groupedE`, `counted_rom_programming_loss_term_subunit`, `counted_rom_support_dominates_budget`, and `counted_rom_loss_from_query_budgets_and_support` ensure that the bound is algebraically well-formed, nonnegative, and expressible in terms of the declared budgets and support sizes.

This file is the numerical interface between proof and theorem statement. The proof scripts in the game files produce losses in structured form, while this file rewrites them into the concrete HAETAE bound used by HAETAE_Security.ec.

B.3.5 HAETAE_ROM.ec

HAETAE_ROM.ec specializes EasyCrypt’s full random oracle to HAETAE. It defines the query tags for message hashing, challenge hashing, matrix expansion, and sampler expansion. Its random-oracle output type contains a CRH, a challenge, a matrix, and signing randomness, with projection operations such as `ro_output_to_crh`, `ro_output_to_challenge`, `ro_output_to_matrix`, and `ro_output_to_random_coins`.

The distribution `dro_output` is query-dependent: matrix expansion uses the matrix distribution, sampler expansion uses signing-randomness output, and other query forms use the appropriate projections. The lemmas `matrix_query_distribution_lossless`, `ro_output_distribution_lossless`, and `sampler_expand_query_dro_output_ro_signing_coins` establish the losslessness and projection behavior that the sampler-freshness proof needs.

This file is small but foundational. If query constructors were not separated, the later prequery analysis could not state precisely that only adversary queries equal to `sampler_expand_query` collide with the signer’s fresh sampler seed.

B.3.6 HAETAE_ROM_Programming.ec

HAETAE_ROM_Programming.ec defines counted ROM programming. It introduces programming sites, programming-site queries and outputs, transcript programming sites, challenge entropy checks, min-entropy failures, and counted ROM events. These definitions allow the proof to count hash queries, signature sites, programmed sites, and min-entropy checks in a single event log.

The module CountedLazyROM is the concrete lazy random oracle used by the programming proof. Preservation lemmas such as `counted_lazy_rom_init_clears`, `counted_lazy_rom_get_preserves_clear`, `counted_lazy_rom_get_preserves_bad_clear`, `counted_lazy_rom_observe_transcript_preserves_clear`, and `counted_lazy_rom_program_preserves_bad_clear_if_fresh` are the state invariants used to justify ROM programming without double-programming or programming a prequeried site.

The later lemmas in the file turn event-counting disciplines into probability bounds. They are the bridge between operational ROM code and the arithmetic loss terms in HAETAE_Reductions.ec. The file therefore supports both the classical Fiat–Shamir with aborts programming step and the separate budgeted sampler prequery analysis used in the active-signing lifting surface.

B.3.7 HAETAE_Scheme.ec

HAETAE_Scheme.ec instantiates the generic signature interface with HAETAE types. It clones Sig_ROM.ec with HAETAE public keys, secret keys, messages, contexts, signatures, ROM queries, and ROM outputs. The operation `haetae_mode` leaves the parameter mode abstract so that the security proof applies uniformly across the supported HAETAE parameter sets.

The module HAETAE supplies the key-generation, signing, and verification procedures used by the generic games. Its role is interface alignment, not cryptographic reasoning. The proofs about the behavior of signing and verification are discharged in the algebra, rejection, transcript, and security files that import this scheme module.

B.3.8 HAETAE_Transcript.ec

HAETAE_Transcript.ec defines the transcript abstraction used by the forking, ROM-

programming, and rejection-sampling arguments. Transcript fields record the public key, message, context, commitment data, challenge, response, hint, signature token, rejection branch, and validity conditions needed to relate a signature to the algebraic equations seen by the verifier.

The lemmas in this file prove that transcript predicates expose the exact fields needed by later game hops. Examples include `transcript_fields_match_signature_challenge_query`, `transcript_valid_signature_challenge_query`, `transcript_valid_public_equation`, `transcript_valid_commitment_reconstruction`, and `valid_forking_pair_public_equations`. The extraction lemmas `extract_bimodal_forking_some` and `extract_bimodal_forking_nonempty` connect valid transcript pairs to the bimodal self-target MSIS reduction interface.

This file is the semantic link between game-based forgery events and algebraic hardness instances. Without transcript validity lemmas, a final forgery would remain an accepted signature only; with these lemmas, the proof can extract the structured algebraic relation required by the reduction.

B.3.9 HAETAE_Events.ec

`HAETAE_Events.ec` defines the small collection of event predicates used to state rejection-freeness and acceptance properties. The operations describe key-generation rejection, signing rejection, verification rejection, and accepted transcript rejection-free conditions.

The four lemmas in the file establish that the current model’s key-generation, signing, verification, and accepted transcript paths satisfy the corresponding event predicates. This isolates event vocabulary from the larger proof files: game hops can refer to named events rather than re-expanding rejection and verification definitions at every boundary.

B.3.10 HAETAE_Assumptions.ec

`HAETAE_Assumptions.ec` defines the abstract hardness games and reduction interfaces used by the final checked implication. It contains module types for solvers and reductions, games for Module-SIS, bimodal self-target MSIS, MLWE, and distribution-lifted relations, plus adapter modules that lift one solver interface into

another.

The lemmas are exactness and well-formedness results for these adapters. For example, `bimodal_solver_lift_exact`, `bimodal_mode_solver_lifted_distribution_exact`, and `bimodal_msis_as_module_sis_exact` are equivalence proofs showing that adapter games preserve success events. Lemmas such as `module_sis_eval_wf`, `module_sis_valid_target_wf`, and `bimodal_to_module_sis_valid` provide structural well-formedness for instances passed to these games.

This file is the explicit cryptographic trust boundary. The machine checker verifies that HAETAE’s game reductions reduce to these games and terms; it does not prove that the abstract hardness terms are small. That interpretation is a mathematical and cryptographic assumption outside EasyCrypt’s deductive core.

B.3.11 HAETAE_Distributions.ec

`HAETAE_Distributions.ec` defines all distributions needed for the scheme model and sampler proofs. It includes byte, seed, CRH, signing randomness, coefficient, polynomial, vector, matrix, challenge, structural signing sample, bounded-unit sample, checked-hyperball sample, and exact-hyperball sample distributions.

The early lemmas prove losslessness and domain correctness for byte, seed, CRH, and signing randomness distributions. The signing-randomness token machinery is especially important: it gives the proof a finite support with a point bound, which is later used to bound the probability that a fresh sampler seed collides with a logged adversary query. Lemmas such as `signing_coin_distribution_lossless`, `signing_entropy_token_to_coins_tokenK`, and `signing_randomness_from_source_tokenK` justify that representation.

The hyperball distribution lemmas support the paper-sample to exact-hyperball transition. They prove well-formedness, boundedness, losslessness, and probability point bounds for the distributions that replace the rejection-based signing attempt distribution. These lemmas are consumed by `HAETAE_Rejection.ec` and `HAETAE_HopGames.ec`.

B.3.12 HAETAE_Rejection.ec

`HAETAE_Rejection.ec` formalizes the signing-attempt state, rejection conditions,

simulation predicates, and distributional bridges for HAETAE’s rejection sampling. The file defines fields extracted from signing attempts, including sampled vectors, commitment high and low bits, challenge, response, auxiliary response, hint, signature token, rejection-branch bit, and programmed challenge data.

The first proof block connects signing attempts to valid signatures. Lemmas such as `signing_attempt_verifies`, `signing_simulation_verifies`, and `signing_attempt_simulates` show that states satisfying the modeled relations produce signatures accepted by verification. These facts are used by the correctness theorem and by the game hops that replace concrete signing with simulated signing.

The sampler block proves that structural, bounded-unit, checked-hyperball, and exact-hyperball samples satisfy the predicates needed by the signing-attempt state. Lemmas such as `structural_signing_sample_pair_sampler_ok`, `bounded_unit_signing_sample_pair_sampler_ok`, `checked_hyperball_signing_sample_pair_sampler_ok`, and `exact_hyperball_signing_sample_pair_sampler_ok` are the checked counterparts of the paper proof’s sampling claims.

The rejection-bound block connects abort predicates to the global `rejection_sampling_loss_term`. This is where the proof accounts for the cost of moving from real rejection sampling to the idealized distributions used in the later NMA and exact-hyperball games.

B.3.13 HAETAE_Algebra.ec

`HAETAE_Algebra.ec` defines the algebraic model for HAETAE. It gives types for bytes, seeds, CRHs, random coins, messages, contexts, coefficients, polynomials, challenges, polynomial vectors, matrices, public keys, secret keys, encoded public keys, signatures, and signature tokens. It also defines zero values, well-formedness predicates, modular coefficient arithmetic, polynomial arithmetic, vector arithmetic, matrix-vector multiplication, packing-like operations, high/low-bit operations, challenge operations, and verification relations.

The 282 lemmas in this file are mostly preservation, size, well-formedness, and equational lemmas. Examples include `poly_zero_wf`, `polyveck_zero_wf`, `poly_add_wf`, `poly_mul_wf`, `poly_dot_wf`, and the corresponding vector and matrix facts. These lemmas are not merely implementation details: later transcript and rejection proofs depend on them to ensure that every sampled or derived value has the correct dimension and belongs to the modeled algebraic domain.

The file is intentionally explicit and list-based. That representation keeps the proof close to executable data structures and avoids hiding dimension conditions in an abstract ring interface. The cost is a large number of size and well-formedness obligations, which EasyCrypt checks once and makes reusable throughout the proof.

B.3.14 HAETAE_Params.ec

HAETAE_Params.ec defines the parameter constants and mode-dependent functions for HAETAE. Constants include seed size, CRH size, polynomial degree, modulus, doubled modulus, and the parameter modes `Mode2`, `Mode3`, and `Mode5`. Mode functions define vector dimensions, challenge weight, rejection bounds, hint parameters, signature sizes, and public key sizes.

The lemmas prove positivity and basic arithmetic facts about these parameters. Examples include `seedbytes_gt0`, `crhbytes_gt0`, `n_gt0`, `q_gt0`, `mode_k_gt0`, `mode_l_gt0`, `mode_tau_le_n`, and `mode_publickeybytes_gt0`. These facts discharge side conditions in distribution losslessness, vector sizing, and probability-bound proofs.

This file is part of the trusted modeling layer because it states the numeric parameters used by all later code. EasyCrypt checks consequences of these definitions, but the correspondence between these constants and the intended HAETAE parameter sets is a modeling obligation.

B.3.15 HAETAE_FIPS202.ec

HAETAE_FIPS202.ec models the SHAKE256 portion of FIPS 202 used by the HAETAE algebraic model. It imports the Keccak permutation model, defines the state size, SHAKE256 rate, capacity, domain separator, final padding byte, absorb operation, squeeze operation, and complete SHAKE256 operation.

The lemmas prove constant equalities and output-size properties, including `fips202_state_bytesE`, `shake256_rate_bytesE`, `shake256_capacity_bytesE`, `shake256_rate_capacity_state_bytesE`, `shake256_absorb_once_short_state_size`, `shake256_squeeze_size`, and `shake256_size`. These facts are used by the algebraic model to ensure byte-string sizes match the expected HAETAE encodings.

For provable security, the random oracle abstraction is the main security object. This file contributes concrete hash-support definitions used by the model and by

correspondence with implementation-level code, but the game-based ROM theorem does not rely on known-answer tests.

B.3.16 HAETAE_Keccak1600.ec

HAETAE_Keccak1600.ec defines a list-based model of the Keccak-f1600 permutation. It introduces byte, lane, and state types; normalization and bit projection operations; lane Boolean operations; rotations; byte-lane conversions; state indexing; round constants; and the theta, rho-pi, chi, and iota round functions.

The lemmas prove well-formedness and bit-level equations for these operations. Examples include `keccak_lane_zero_wf`, `keccak_lane_xor_wf`, `keccak_lane_and_wf`, `keccak_lane_rotl_wf`, `keccak_bytes_to_lane_wf`, `keccak_theta_bitE`, `keccak_rho_pi_bitE`, and `keccak_chi_bitE`. These lemmas support the FIPS 202 model’s size and bit-level correctness obligations.

In the provable-security proof, this file is support code rather than the main cryptographic reduction. Its function is to make the hash-related HAETAE model internally consistent. Security is still proved in the ROM abstraction; the Keccak model documents and checks the concrete hash representation used by the formalized scheme layer.

B.4 Cross-file dependency analysis

The dependency flow is acyclic at the conceptual level. Parameters feed algebra; algebra and parameters feed distributions; parameters, algebra, and distributions feed the ROM and scheme; algebra, distributions, events, and transcripts feed rejection; ROM and transcript logic feed ROM programming; ROM programming, rejection, transcript logic, and reductions feed hop games; hop games and reductions feed the top-level security theorem.

The most important design choice is the separation between pure predicates and stateful games. Pure files such as `HAETAE_Params.ec`, `HAETAE_Algebra.ec`, `HAETAE_Distributions.ec`, `HAETAE_Transcript.ec`, and `HAETAE_Reductions.ec` define operations and prove algebraic or probabilistic facts about them. Stateful files such as `Sig_ROM.ec`, `HAETAE_ROM_Programming.ec`, and `HAETAE_HopGames.ec` define games and use Hoare or equivalence proofs to relate adversarial executions. This split is

useful because it localizes program-logic obligations to the files where mutable game state is actually present.

The second important design choice is explicit event logging. Transcript logs, ROM query logs, counted ROM event logs, and sampler self-query logs make the fundamental-lemma proofs inspectable. The budgeted sampler prequery event is not a black-box union bound over an oversized wrapper; it is tied to the concrete event that a fresh `sampler_expand_query` is already present in the logged joint log of adversary hash queries and previous sampler self-queries. This event is distinct from the `counted_rom_prequery_term` summand in the final counted-ROM bound.

The third important design choice is centralization of concrete arithmetic. Loss terms are not duplicated across hop proofs. Hop files prove local probability inequalities, and `HAETAE_Reductions.ec` normalizes them into the final bound. This reduces the risk that two theorem statements silently use different interpretations of the same loss.

B.5 Residual proof obligations outside EasyCrypt

The analysis identifies four residual obligations that are outside the manifest even though the manifest itself is machine checked. First, the abstract hardness terms in `HAETAE_Assumptions.ec` must be interpreted as the intended MLWE, Module-SIS, and bimodal self-target MSIS assumptions. Second, the parameter constants in `HAETAE_Params.ec` must match the parameter set claimed by the surrounding HAETAE specification. Third, the modeled signing, verification, rejection, and transcript operations must be kept aligned with the implementation and the paper specification. Fourth, the manifest `provable-security/proof-files.txt` must remain the audited proof surface; adding or removing EasyCrypt files changes the scope of the claim and requires a fresh inventory and compile check.

Within those boundaries, the EasyCrypt code performs the intended machine-checked provable-security work: it defines the security games, formalizes the HAETAE model, proves the transcript and sampler invariants, carries out the ROM and budgeted lifting hops, proves the concrete arithmetic side conditions, and composes the checked lemmas into the final conditional EUF-CMA implication. The main dissertation now treats this appendix as evidence, not as a substitute for the mathematical proof narrative; curated proof-object analyses appear in Appendix F, and the exhaustive generated individual correspondence is archived under `dissertation/etc/`.

Appendix C

Mathematical Correspondence of the EasyCrypt Proof Objects

Entries in this appendix use schematic metavariables when describing general EasyCrypt syntax. In particular, occurrences of ellipses such as $\mathbf{tau1} \rightarrow \dots \rightarrow \mathbf{taun}$ denote families of language constructs, not omitted parts of a specific HAETAE theorem. Exact theorem and module names are written without ellipses.

This appendix gives the mathematical reading of every kind of EasyCrypt object used in the HAETAE provable-security development. The goal is not to restate the proof script tactically, but to say what the checked declarations mean as mathematics. Let $\llbracket - \rrbracket$ denote the intended mathematical interpretation of an EasyCrypt declaration in the manifest proof surface.

C.1 Global interpretation principle

The EasyCrypt development is a typed formalization of a game-based proof. Mathematically, each manifest file contributes objects in a category of discrete probabilistic programs, predicates over their states, and inequalities between event probabilities. The top-level checked implication is therefore a fixed-ledger formal-model statement of the following form:

$$\mathcal{H}_{\text{hop}} \wedge \mathcal{H}_{\text{hard}} \implies \text{Adv}_{\text{ECModel}, \text{ROM}}^{\text{EUF-CMA}}(A) \leq B_{\text{loss}}(2, 2, \lambda) + L_{\text{ModuleSIS}} + L_{\text{MLWE}} + L_{\text{bimodal} \rightarrow \text{MSIS}},$$

where the right-hand side is the concrete expression defined in the reduction file after applying the declared hardness bounds. The reductions B_i are either explicit adapters in individual hop lemmas or abstract reduction modules supplied through the final lemma’s antecedents. In particular, the final composition lemma does not derive the MLWE or Module-SIS solvers from A ; it uses the explicit hop/reduction and hardness premises present in the EasyCrypt statement. Each EasyCrypt type, operation, module, lemma, and Hoare/equivalence judgment contributes one well-typed component to this implication. A specification-level or arbitrary-query HAETAE theorem would require additional correspondence and parameterization work.

C.2 Types as carrier sets

An EasyCrypt declaration of the form

typet

is interpreted as a carrier set $\llbracket t \rrbracket$. If the type is abstract, the proof treats it as an arbitrary set satisfying only the properties derivable from the surrounding declarations. If the type is an explicit finite datatype, such as the HAETAE mode type, it is interpreted as the listed finite set. Product, list, and distribution types are interpreted in the usual set-theoretic way:

$$\llbracket \tau_1 * \tau_2 \rrbracket = \llbracket \tau_1 \rrbracket \times \llbracket \tau_2 \rrbracket, \quad \llbracket \text{List}(\tau) \rrbracket = \llbracket \tau \rrbracket^*,$$

and $\llbracket \text{Distr}(\tau) \rrbracket$ is the set of discrete probability or subprobability laws over $\llbracket \tau \rrbracket$, with `is_lossless` asserting total mass 1.

Thus all manifest types have one of the following mathematical roles.

Table C.1: Mathematical reading of all manifest type declarations.

EasyCrypt type class	Mathematical correspondence
Signature-game types	Carrier sets for public keys, secret keys, messages, contexts, signatures, random-oracle queries, random-oracle outputs, and signed-query records. These sets define the sample space of the generic correctness, EUF-CMA, SUF-CMA, and UF-NMA experiments.
HAETAE algebra types	Concrete carrier sets for byte strings, seeds, CRHs, random coins, coefficients, polynomials, challenges, polynomial vectors, matrices, public keys, secret keys, and signatures. These sets are modeled as integers, lists, products, and mode-indexed list families.
Parameter-mode types	Finite sets of parameter choices. The declaration of HAETAE modes is mathematically the finite set $\{\text{Mode2}, \text{Mode3}, \text{Mode5}\}$.
Distribution-support types	Carrier sets for random coins, signing entropy tokens, sample pairs, hyperball samples, and rejection-sampling states. These are the domains on which the sampler laws are defined.
Transcript and event types	Carrier sets for transcript records, counted ROM events, programming sites, and bad-event indicators. These are state spaces for hybrid-game invariants.
Hardness-game types	Carrier sets for Module-SIS instances, MLWE instances, bimodal self-target instances, solutions, and solver outputs. These are the mathematical domains of the assumed hard problems.
Hash-support types	Carrier sets for Keccak bytes, lanes, states, and FIPS 202 states. These model bit-level and byte-level hash state spaces.

The mathematical significance is that no lemma may mix these sets unless the EasyCrypt type checker admits the expression. The type checker therefore enforces, for example, that a sampler seed, a ROM query, a challenge, and a signature transcript cannot be confused in a reduction.

C.3 Operations as functions, predicates, distributions, and bounds

An EasyCrypt operation declaration of the form

$$\text{opf} : \tau_1 \rightarrow \dots \rightarrow \tau_n \rightarrow \tau$$

is interpreted as a total mathematical function

$$\llbracket f \rrbracket : \llbracket \tau_1 \rrbracket \times \dots \times \llbracket \tau_n \rrbracket \rightarrow \llbracket \tau \rrbracket.$$

When $\tau = \text{Bool}$, the operation is a predicate or event. When $\tau = \text{Distr}(\sigma)$, it is a probability law or sampling distribution. When $\tau = \mathbb{R}$, it is a concrete loss, advantage, or bound term. When τ is a product or record-like tuple, it is a projection or constructor.

Every operation in the proof surface falls into one of the following mathematical classes.

Table C.2: Mathematical reading of all manifest operation declarations.

Operation class		Mathematical correspondence
Constructors and projections		Functions building or extracting components of keys, signatures, transcripts, programming sites, random-oracle outputs, sample pairs, and rejection states. Mathematically these are product constructors and coordinate projections.
Well-formedness predicates		Predicates $P(x)$ expressing membership in a syntactic or algebraic domain: correct list length, coefficient range, sparse challenge form, valid vector dimension, valid transcript field relation, or valid hardness instance.
Algebraic operations		Total functions implementing modular arithmetic, polynomial addition, negation, subtraction, multiplication, vector arithmetic, matrix-vector multiplication, high/low-bit extraction, commitment reconstruction, and public verification equations.

Operation class		Mathematical correspondence
Hash-query constructors	con-	Injective or tagged constructors for message-hash, challenge-hash, matrix-expansion, and sampler-expansion queries. Mathematically they partition the ROM domain into the query forms used by the security proof.
Distributions		Probability laws for bytes, seeds, CRHs, signing randomness, coefficients, polynomials, vectors, matrices, challenges, structural samples, bounded-unit samples, checked-hyperball samples, exact-hyperball samples, and ROM outputs.
Events and predicates	bad	Boolean predicates defining rejection, verification success, forgery success, prequery, collision, reprogramming failure, min-entropy failure, counted ROM badness, sampler badness, and transcript readiness. These are events in the probability spaces of the games.
Budgets and terms	loss	Real-valued functions and constants representing query budgets, support-size lower bounds, ROM collision terms, prequery terms, min-entropy terms, programming losses, rejection-sampling losses, and final EUF-CMA bounds.
Hardness relations		Predicates and functions defining Module-SIS, MLWE, and bimodal self-target relations, together with adapter maps from one relation or game to another.
Hash-state functions		Bit and byte functions for Keccak lanes, state indexing, round transformations, FIPS 202 absorb/squeeze, and SHAKE256 sizing. These are deterministic functions over finite bit/byte sequences.

This is the mathematical reason the proof can be read as a sequence of ordinary definitions. For example, `sampler_expand_query` is not a tactic artifact; it is a function from signing randomness to the ROM domain. The event `sampler_bad_prequery` is the corresponding predicate that this query already lies in the joint log containing adversary hash queries and earlier sampler-expansion self-queries. The tight bound is then a finite-support probability statement about a freshly sampled argument to that function colliding with the combined log.

C.4 Modules as games, algorithms, oracles, and reductions

An EasyCrypt module is interpreted as a probabilistic imperative algorithm with local state. A module type is an interface specifying the procedures available to such an algorithm. A functorial module, such as a game parameterized by an adversary and an oracle, is interpreted as a mathematical construction sending algorithms to algorithms:

$$(A, H, O) \longmapsto G^{A, H, O}.$$

The procedures of the resulting module define random variables and events by executing probabilistic code.

Table C.3: Mathematical reading of all manifest module declarations.

Module class	Mathematical correspondence
Oracle interfaces	Abstract access to random-oracle procedures and programmable-oracle procedures. Mathematically these are black-box functions or stateful lazy-sampling oracles.
Scheme interfaces	Abstract key-generation, signing, and verification algorithms. These define the cryptographic primitive whose correctness and security games are studied.
Adversary interfaces	Classes of algorithms with specified oracle access. EUF-CMA adversaries may call both hash and signing oracles; UF-NMA adversaries receive only the public interface needed by the no-message game.
Security games	Probabilistic experiments for correctness, EUF-CMA, SUF-CMA, UF-NMA, ROM-internal transcript games, simulated-signing games, and exact-hyperball games. Their outputs are Bernoulli random variables representing success events.
Adapters	Program transformations from one adversary or solver interface to another, such as NMA-as-CMA adapters and bimodal-to-Module-SIS adapters. Mathematically these are reductions between experiments.

Module class	Mathematical correspondence
Lazy ROM modules	Stateful samplers implementing the random oracle by memoization. Their state is a finite map from queries to sampled outputs plus logs and bad-event flags.
Programming modules	Algorithms that reprogram or observe ROM sites. Mathematically these implement the standard ROM programming operation subject to freshness and min-entropy preconditions.
Signing simulators and samplers	Algorithms that generate signatures or transcript samples under real, structural, paper, rejection, RO-attempt, or exact-hyperball distributions. Mathematically these are Markov kernels between message/context inputs and signature/transcript outputs.
Hardness games and reductions	Experiments defining the abstract success probability of MLWE, Module-SIS, and bimodal self-target MSIS solvers, plus reductions translating HAETAE forgeries into solver successes.

This module interpretation is essential because the proof is not merely a collection of static equations. EUF-CMA security is a statement about an interactive experiment. Modules are the formal objects whose executions define that experiment.

C.5 Lemmas as propositions, invariants, equivalences, and inequalities

An EasyCrypt lemma declaration of the form

`lemmaL:Phi.`

is interpreted as a theorem asserting the formula $\llbracket \Phi \rrbracket$. The proof script following the lemma is a machine-checked derivation in EasyCrypt’s logic and program logic. For the HAETAE security proof, every lemma belongs to one of the following mathematical families.

Table C.4: Mathematical reading of all manifest lemma declarations.

Lemma family	Mathematical correspondence
Parameter lemmas	Arithmetic facts about constants and mode-dependent parameters, such as positivity, size lower bounds, and challenge-weight feasibility.
Algebraic lemmas	Equational and well-formedness theorems for coefficients, polynomials, vectors, matrices, challenges, signatures, and verification equations.
Distribution lemmas	Losslessness, support, point-probability, boundedness, and sampler-correctness facts for the distributions used in signing and simulation.
Transcript lemmas	Theorems saying that transcript predicates imply the public equations, challenge-query identities, field consistency, and extraction conditions needed by the hardness reductions.
Event lemmas	Implications relating rejection-free events, acceptance events, verification success, and forgery events.
ROM lemmas	Losslessness and projection facts for query-dependent ROM output distributions, plus preservation and counting facts for lazy ROM state.
ROM-programming lemmas	Freshness, non-prequery, min-entropy, counted-event, and reprogramming-failure bounds. Mathematically these are fundamental-lemma and union-bound steps over finite query logs.
Invariant lemmas	State preservation theorems for transcript logs, internal transcript state, sampler ROM coverage, signing budgets, active public keys, and adversary query logs.
Game-hop lemmas	Inequalities or equalities between success probabilities of adjacent games. These are the checked hybrid steps of the security proof.
Reduction lemmas	Theorems transforming a successful forgery or bimodal event into success in an abstract hardness game, with the corresponding advantage inequality.

Lemma family	Mathematical correspondence
Concrete-bound lemmas	Real-arithmetic theorems normalizing, grouping, proving non-negativity of, or bounding the final concrete loss expression.
Top-level security lemmas	Compositions of all previous families into correctness and conditional EUF-CMA advantage theorems for the HAETAE model.

In mathematician’s language, the 1088 checked lemmas form a dependency graph of propositions. The leaves are parameter, algebraic, distributional, and well-formedness facts. The middle nodes are invariant and hybrid-game propositions. The root is the conditional EUF-CMA security implication. Machine checking means that every edge in this graph is justified by EasyCrypt’s proof kernel, not by an informal appeal to a paper proof.

C.6 Hoare and equivalence judgments as program-logic theorems

The Hoare and equivalence commands are the formal counterpart of reasoning about program executions. If c is a probabilistic procedure, an EasyCrypt Hoare judgment `hoare[c:P==>Q]` is interpreted as partial-correctness reasoning: every terminating execution starting in a state satisfying P ends in a state satisfying Q . A probabilistic Hoare judgment `phoare[c:P==>Q]=p` is interpreted as a statement about the probability that Q holds after executing c from a P -state. An equivalence judgment `equiv[c1~c2:Rpre==>Rpost]` is interpreted as a coupling or relational program theorem: two executions can be related so that the post-relation holds whenever the pre-relation holds. The command `byequiv` is the proof-script form of such a relational argument.

The 105 Hoare/equivalence proof commands in the manifest have the following mathematical roles.

Table C.5: Mathematical reading of all manifest Hoare/e-
quivalence proof commands.

Program-logic family		Mathematical correspondence
Correctness	Hoare	proofs
		Show that modeled key generation, signing, and verification preserve the correctness predicate and end in accepting states.
Adapter equivalences		Show that wrapping one adversary or solver interface into another does not change the relevant success event. Mathematically these are exact reductions with zero loss.
Simulated-signing equivalences		Show that replacing real signing by transcript or internal-transcript signing preserves the adversary’s observable behavior up to the stated bad event.
ROM-internal equivalences		Show that moving hash behavior into a lazy internal ROM preserves query/output relations and adversary result distributions under synchronized states.
Sampler equivalences		Show that structural, paper, rejection, RO-attempt, and exact-hyperball samplers produce related outputs under the coupling conditions proved by sampler lemmas.
Fundamental-lemma proofs		Show that two games are identical on the complement of a bad event, so their success probabilities differ by at most the probability of that bad event.
Budgeted oracle-call proofs		Show that the budgeted wrappers preserve capped counters and logs, and that the calls recorded before the cap are exactly the calls counted in the loss term. Over-budget calls are handled by the modeled wrapper behavior rather than by a syntactic restriction on arbitrary adversary code.

These judgments are the place where informal “the games are identical until bad” reasoning becomes mathematical. The proof must name the program states, state relation, bad event, and postcondition. EasyCrypt then checks that every procedure call, random sample, assignment, branch, and oracle invocation preserves the required relation or incurs the stated loss.

C.7 File-wise mathematical correspondence

The previous sections describe the universal interpretation of the EasyCrypt object classes. The following table records how those classes specialize in each manifest file.

Table C.6: Per-file mathematical correspondence of proof-object classes.

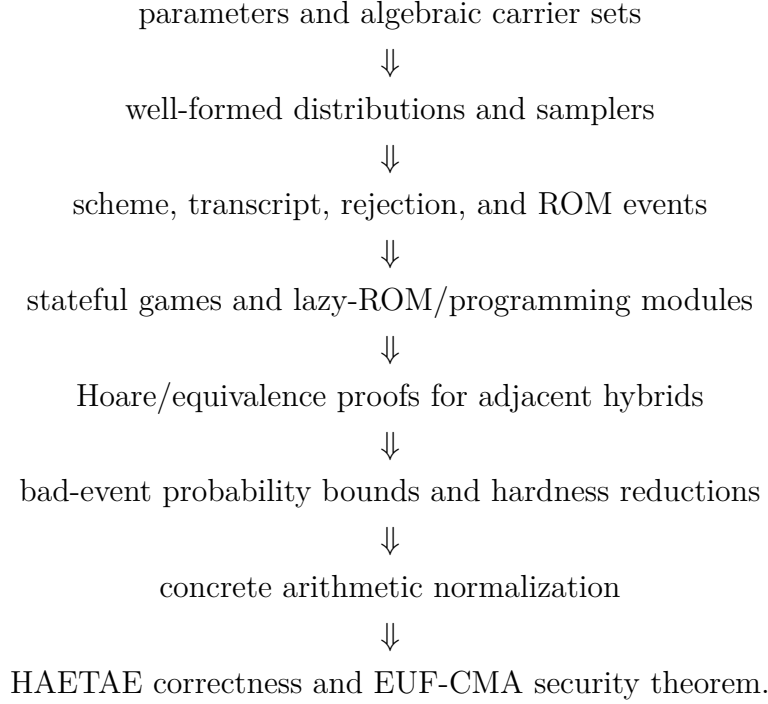
File	Mathematical role of its types, operations, modules, lemmas, and program-logic judgments
<code>Sig_ROM.ec</code>	Types are abstract carrier sets for a generic ROM signature scheme. Operations are freshness predicates and ROM-output distributions. Modules are correctness, EUF-CMA, SUF-CMA, UF-NMA games and adversary/oracle interfaces. Lemmas prove freshness base cases and the exact NMA-as-CMA correspondence. Equivalence judgments are exact game-isomorphism statements.
<code>HAETAE_Params.ec</code>	The mode type is a finite parameter set. Operations are numeric parameter functions. Lemmas are arithmetic facts ensuring positivity, dimensions, and feasibility side conditions. There are no stateful games here.
<code>HAETAE_Algebra.ec</code>	Types are algebraic carrier sets for HAETAE objects. Operations are modular, polynomial, vector, matrix, packing, challenge, and verification functions. Lemmas are algebraic equations and well-formedness preservation theorems.
<code>HAETAE_Keccak1600.ec</code>	Types are bit/byte/lane/state carrier sets. Operations are Keccak state transformations and projections. Lemmas are bit-level equations, well-formedness facts, and size facts for the permutation model.
<code>HAETAE_FIPS202.ec</code>	Types are FIPS 202 byte and state spaces. Operations are SHAKE256 absorb/squeeze functions and constants. Lemmas are state-size and output-size theorems supporting the modeled hash layer.

File	Mathematical role of its types, operations, modules, lemmas, and program-logic judgments
HAETAE_Distributions.ec	Types are sampler domains and sample-pair spaces. Operations are probability laws, support predicates, point-bound expressions, and sample constructors. Lemmas are losslessness, support, boundedness, and sampler-correctness theorems.
HAETAE_ROM.ec	Types are HAETAE ROM query and output spaces. Operations are tagged query constructors, output projections, output injections, and query-dependent output laws. Modules instantiate the full ROM. Lemmas prove losslessness and sampler projection laws.
HAETAE_Scheme.ec	The module is the HAETAE instance of the generic signature interface. The mode operation fixes the parameter family. Mathematically this file identifies the generic game carrier sets with the HAETAE carrier sets.
HAETAE_Events.ec	Operations are named events for rejection-freeness and verification acceptance. Lemmas show that the current modeled events imply the event predicates consumed by correctness and game-hop theorems.
HAETAE_Transcript.ec	The transcript type is the carrier set of signing transcripts. Operations are field projections, consistency predicates, validity predicates, forking relations, and extraction maps. Lemmas prove that valid transcripts encode the algebraic equations required by the hardness reductions.
HAETAE_Rejection.ec	Types are signing-attempt states and sample spaces. Operations are rejection predicates, state projections, signature constructors, acceptance predicates, and sampler predicates. Lemmas prove verification, simulation, sampler-correctness, and rejection-loss facts.
HAETAE_Assumptions.ec	Types are hardness-instance and solution spaces. Operations are relation predicates and distribution adapters. Modules are hardness games, solver interfaces, and reductions. Lemmas and equivalence judgments prove exactness of adapters and well-formedness of lifted instances.

File	Mathematical role of its types, operations, modules, lemmas, and program-logic judgments
HAETAE_ Reductions.ec	Operations are real-valued hardness terms, query budgets, support bounds, ROM losses, rejection losses, and final advantage bounds. Lemmas are arithmetic normalization, non-negativity, subunit, grouping, and support-dominance theorems.
HAETAE_ROM_ Programming. ec	Types are programming sites and counted ROM events. Operations are event counters, freshness predicates, min-entropy predicates, and counted bad events. Modules are lazy ROM and programming games. Lemmas are preservation, freshness, count, min-entropy, and ROM-programming loss theorems.
HAETAE_ HopGames.ec	Types and operations define transcript invariants, sampler coverage, public simulation fields, paper-simulation samples, bad events, and budget disciplines. Modules are the full sequence of real, simulated, transcript, ROM-internal, NMA, paper-sampler, RO-attempt, exact-hyperball, and budgeted-wrapper games. Lemmas and Hoare/equivalence judgments prove the hybrid steps, invariants, fundamental-lemma bounds, sampler-freshness bounds, and budgeted lifting theorems.
HAETAE_ Security.ec	Operations define the top-level correctness obligation. Lemmas compose all lower-level facts into correctness and conditional EUF-CMA theorems. Its Hoare proof is the final correctness program-logic obligation. Mathematically this file is the root of the proof DAG: it turns local game-hop and arithmetic theorems, under the explicit hop and hardness premises of the final composition lemma, into the published security inequality.

C.8 Mathematical reading of the final proof DAG

The entire EasyCrypt development can be summarized as the following mathematical dependency chain.



In this reading, Types supply the spaces, Ops supply the maps and events, Modules supply the experiments, Lemmas supply the propositions, and Hoare/equiv judgments supply the program-logic bridges between experiments. The machine checker verifies that these objects compose without type errors, unproved proof holes, or missing side conditions, thereby converting the pen-and-paper game proof into a formal theorem about the modeled HAETAE scheme.

Appendix D

Formal Mathematical Semantics of the EasyCrypt Security Proof Surface

This appendix contains two kinds of notation. Concrete EasyCrypt identifiers are written with `...` macros and refer to the proof surface discussed in the main text. Schematic fragments containing ellipses, such as $\tau_1 \rightarrow \cdots \rightarrow \tau_n$ or `moduleM(...)`, are templates for the EasyCrypt language forms. They are not quoted theorem headers and should not be read as exact code correspondence claims.

This appendix fixes the exact mathematical interpretation of the EasyCrypt objects listed in the generated declaration index. It is written as a semantic dictionary: every declaration header in Appendix E is read by applying one of the clauses below. The clauses are intentionally uniform, because the soundness claim of the machine-checked proof is not that each identifier has a private informal explanation, but that every declaration is typed, interpreted, and proved in the same formal logic.

D.1 Judgmental setting

Let Γ be the EasyCrypt environment generated by the manifest `provable-security/proof-files.txt`. The checker establishes judgments of the form

$$\Gamma \vdash d : \mathcal{K},$$

where d is a declaration and $\mathcal{K}$ is its syntactic kind:

$$\mathcal{K} \in \{\text{Type}, \text{Op}, \text{Module}, \text{Lemma}, \text{Hoare}, \text{Equiv}\}.$$

The semantic map

$$\llbracket - \rrbracket_\Gamma$$

sends each checked declaration to a mathematical object. For the HAETAE security proof, those objects live in the following universe:

$$\text{Set}, \quad \text{Dist}(X), \quad \text{SubDist}(X), \quad X \rightarrow Y, \quad X \rightarrow \text{Bool}, \quad \text{Prog}, \quad \text{Kern}(X, Y),$$

where $\text{Dist}(X)$ denotes probability distributions over X , $\text{SubDist}(X)$ denotes subdistributions over X , and $\text{Kern}(X, Y) = X \rightarrow \text{SubDist}(Y)$ denotes a general probabilistic program kernel. Losslessness lemmas identify the special cases whose kernels have total mass one and can therefore be read as $X \rightarrow \text{Dist}(Y)$.

D.2 Types

The exact EasyCrypt forms used by the manifest are:

```

1 type t.
2 type t = tau.
3 type t = [ C1 | ... | Cn ].
4 type t = tau1 * ... * taun.
```

The mathematical reading is:

$$\Gamma \vdash \text{type } t : \text{Type} \quad \Longrightarrow \quad \llbracket t \rrbracket_\Gamma = X_t \in \text{Set}.$$

If t is abstract, X_t is an uninterpreted carrier set. If t is a finite variant type, then

$$\llbracket [C_1 | \dots | C_n] \rrbracket_\Gamma = \{C_1, \dots, C_n\}.$$

If t is a product type, then

$$\llbracket \tau_1 * \dots * \tau_n \rrbracket_\Gamma = \llbracket \tau_1 \rrbracket_\Gamma \times \dots \times \llbracket \tau_n \rrbracket_\Gamma.$$

If t is a list type, then

$$\llbracket \tau \text{ list} \rrbracket_{\Gamma} = \llbracket \tau \rrbracket_{\Gamma}^*.$$

For the HAETAE proof this gives the following exact mathematical classes:

$$\begin{aligned} \llbracket \text{pkey} \rrbracket &= \text{PK}, \\ \llbracket \text{skey} \rrbracket &= \text{SK}, \\ \llbracket \text{message} \rrbracket &= \text{Msg}, \\ \llbracket \text{context} \rrbracket &= \text{Ctx}, \\ \llbracket \text{signature} \rrbracket &= \text{Sig}, \\ \llbracket \text{ro_query} \rrbracket &= \text{Q}_{\text{RO}}, \\ \llbracket \text{ro_output} \rrbracket &= \text{Y}_{\text{RO}}, \\ \llbracket \text{mode} \rrbracket &= \{\text{Mode2}, \text{Mode3}, \text{Mode5}\}, \\ \llbracket \text{poly} \rrbracket &= \mathbb{Z}^* \text{ with well-formed subset } \mathbb{Z}^{256}, \\ \llbracket \text{matrix} \rrbracket &= (\mathbb{Z}^*)^{**} \text{ with mode-indexed well-formed subset.} \end{aligned}$$

The proof scripts use predicates such as `poly_wf`, `polyveck_wf`, and `challenge_wf` to restrict list carriers to the intended finite-dimensional algebraic domains.

D.3 Operations

The exact EasyCrypt forms used by the manifest are:

```

1 op c : tau.
2 op f (x1 : tau1) ... (xn : taun) : tau = body.
3 op d : tau distr.
4 op [lossless] d : tau distr.
5 abbrev a = body.
```

The mathematical reading is:

$$\Gamma \vdash \text{op } f : \tau_1 \rightarrow \cdots \rightarrow \tau_n \rightarrow \tau : \text{Op} \quad \Longrightarrow \quad \llbracket f \rrbracket_{\Gamma} : \llbracket \tau_1 \rrbracket_{\Gamma} \times \cdots \times \llbracket \tau_n \rrbracket_{\Gamma} \rightarrow \llbracket \tau \rrbracket_{\Gamma}.$$

The codomain determines the mathematical role:

$$\begin{aligned}
 \tau = \text{bool} &\implies \llbracket f \rrbracket \subseteq \llbracket \tau_1 \rrbracket \times \cdots \times \llbracket \tau_n \rrbracket, && \text{predicate or event,} \\
 \tau = \sigma \text{ distr} &\implies \llbracket f \rrbracket : \llbracket \tau_1 \rrbracket \times \cdots \times \llbracket \tau_n \rrbracket \rightarrow \mathbf{SubDist}(\llbracket \sigma \rrbracket), && \text{sampler or law; losslessness gives a di} \\
 \tau = \text{real} &\implies \llbracket f \rrbracket \in \mathbb{R} \text{ or } \llbracket f \rrbracket : \cdots \rightarrow \mathbb{R}, && \text{advantage or loss,} \\
 \tau = \sigma_1 * \cdots * \sigma_m &\implies \llbracket f \rrbracket : \cdots \rightarrow \llbracket \sigma_1 \rrbracket \times \cdots \times \llbracket \sigma_m \rrbracket, && \text{constructor.}
 \end{aligned}$$

The following examples are exact EasyCrypt declarations from the proof surface and their mathematical readings.

```

1 op fresh_msg (qs : query list) (m : message) (ctx :
   context) : bool =
2   ! mem qs (m, ctx).

```

$$\llbracket \text{fresh_msg} \rrbracket(Q, m, c) \equiv (m, c) \notin Q.$$

```

1 op sampler_expand_query (coins : random_coins) : ro_query
   =
2   SamplerExpandQuery coins.

```

$$\llbracket \text{sampler_expand_query} \rrbracket : \text{Coins} \rightarrow \mathbf{Q}_{\text{RO}}, \quad r \mapsto \text{SamplerExpandQuery}(r).$$

```

1 op counted_rom_programming_loss_term =
2   counted_rom_collision_term + counted_rom_prequery_term
3   + counted_rom_min_entropy_term.

```

$$L_{\text{ROM}} = L_{\text{coll}} + L_{\text{pre}} + L_{\text{me}}.$$

```

1 op dsigning_randomness : random_coins distr =
2   dmap dsigning_randomness_source
   signing_randomness_from_source.

```

$$\mu_{\text{coins}} = (\text{signing_randomness_from_source})_{\#} \mu_{\text{source}},$$

where $(-)_{\#}$ denotes the push-forward of a distribution.

D.4 Modules and module types

The exact EasyCrypt forms used by the manifest are:

```

1 module type I = { proc p(x : tau) : sigma }.
2 module M(A : I1, O : I2) = { ... }.
3 clone import T as U with type t <- tau, op f <- g.

```

A module type is an interface:

$$\llbracket \text{module type } I \rrbracket_{\Gamma} = \mathcal{I} = \{(p_i : X_i \rightarrow \text{SubDist}(Y_i))_i\}.$$

A concrete module implementing this interface is a tuple of probabilistic procedures:

$$\llbracket M \rrbracket_{\Gamma} \in \mathcal{I}.$$

A functorial module is a higher-order construction:

$$\llbracket M(A, O) \rrbracket_{\Gamma} : \llbracket A \rrbracket \times \llbracket O \rrbracket \rightarrow \text{Prog}.$$

For a security game module $G(A)$, the procedure `main` denotes a Bernoulli experiment:

$$\llbracket G(A).\text{main} \rrbracket : \mathbf{1} \rightarrow \text{SubDist}(\text{Bool}), \quad \Pr[G(A).\text{main}() = \text{true}] = \text{Adv}_G(A).$$

For HAETA, the key module interpretations are:

$$\begin{aligned}
\llbracket \text{EUF_CMA}(H, S, A) \rrbracket &= \text{chosen-message unforgeability experiment,} \\
\llbracket \text{UF_NMA}(H, S, A) \rrbracket &= \text{no-message unforgeability experiment,} \\
\llbracket \text{CountedLazyROM} \rrbracket &= \text{lazy random oracle with query and bad-event logs,} \\
\llbracket \text{ROSigningAttemptPaperSimSampler} \rrbracket &= \text{signing-attempt sampler kernel,} \\
\llbracket \text{ROExactHyperballPaperSimSampler} \rrbracket &= \text{exact-hyperball sampler kernel,} \\
\llbracket \text{BimodalMSIS_As_ModuleSIS} \rrbracket &= \text{reduction from bimodal self-target MSIS to ModuleSIS}
\end{aligned}$$

D.5 Lemmas

The exact EasyCrypt form is:

```

1 lemma L (x1 : tau1) ... (xn : taun) : Phi.
2 proof.
3   ...
4 qed.

```

The mathematical reading is the theorem judgment

$$\Gamma \vdash L : \Phi \quad \Longrightarrow \quad \Gamma \models \forall x_1 \in \llbracket \tau_1 \rrbracket, \dots, x_n \in \llbracket \tau_n \rrbracket. \llbracket \Phi \rrbracket.$$

The proof script between `proof.` and `qed.` is a kernel-checked derivation of this proposition.

The main lemma families have the following exact symbolic forms.

Well-formedness lemmas. An EasyCrypt lemma such as

```
1 lemma poly_add_wf p r : poly_wf (poly_add p r).
```

means

$$\forall p, r \in \text{Poly}. \text{WF}_{\text{poly}}(p + r).$$

When the definition of `poly_wf` is unfolded, this is a dimension preservation theorem for the list representation of polynomials.

Losslessness lemmas. An EasyCrypt lemma such as

```
1 lemma signing_coin_distribution_lossless : is_lossless
  drandom_coins.
```

means

$$\sum_{r \in \text{Coins}} \mu_{\text{coins}}(r) = 1.$$

Game-hop lemmas. A game-hop lemma has mathematical form

$$\Pr[G_i^A \Rightarrow 1] \leq \Pr[G_{i+1}^{B(A)} \Rightarrow 1] + L_i.$$

The EasyCrypt statement names the concrete modules and loss term; the proof shows that the displayed probability inequality follows from an equivalence, Hoare invariant, or bad-event bound.

Reduction lemmas. A reduction lemma has mathematical form

$$\text{Adv}_{\text{HAETAE}}^{G_i}(A) \leq \text{Adv}_{\text{Hard}}(B_A) + L.$$

Here B_A is either an explicit EasyCrypt adapter module or an abstract solver/reduction interface. For an explicit adapter lemma, the proof connects source-game success to target-game success up to loss L . For an abstract interface lemma, that connection is an explicit antecedent supplied to the composition theorem.

Top-level lemmas. The final checked implication family in `HAETAE_Security.ec` has the fixed-ledger mathematical shape

$$\mathcal{H}_{\text{hop}} \wedge \mathcal{H}_{\text{hard}} \implies \text{Adv}_{\text{ECModel}, \text{ROM}}^{\text{EUF-CMA}}(A) \leq B_{\text{counted ROM}} + L_{\text{ModuleSIS}} + L_{\text{MLWE}} + L_{\text{bimodal} \rightarrow \text{MSIS}},$$

where $B_{\text{counted ROM}}$ is the sum of the checked collision, prequery, and min-entropy terms in `HAETAE_Reductions.ec`, and $L_{\text{ModuleSIS}}$ and L_{MLWE} are the declared hardness terms obtained from the hardness premises. The separate paper-style rejection/Fiat-Shamir-with-aborts expression `haetae_euf_bound` is not an additional summand in the counted-ROM theorem `haetae_euf_counted_rom_bound`. The antecedents $\mathcal{H}_{\text{hop}}$ and $\mathcal{H}_{\text{hard}}$ are not implicit: they are the explicit hop/reduction and hardness premises of the EasyCrypt lemma being cited.

D.6 Hoare, probabilistic Hoare, and equivalence judgments

The exact EasyCrypt proof forms used by the manifest are:

```

1 hoare .
2 phoare .
3 equiv .
4 byequiv (: P ==> Q) => //.
```

The mathematical judgments are:

$$\vdash \{P\} c \{Q\} \quad \text{for ordinary Hoare proofs,}$$

$$\vdash \Pr[c(s) : Q] \bowtie p \quad (\bowtie \in \{=, \leq, <, \geq, >\}) \quad \text{for probabilistic Hoare proofs,}$$

and

$$\vdash c_1 \sim c_2 : R_{\text{pre}} \Rightarrow R_{\text{post}} \quad \text{for relational equivalence proofs.}$$

In the HAETAE game hops, these judgments are used in three mathematically

distinct ways.

Invariant preservation. For a signing oracle $O.\text{sign}$, a Hoare proof establishes

$$\{I(s)\} O.\text{sign}(m, c) \{I(s') \wedge Q(s, s', m, c)\}.$$

Here I may be transcript-log consistency, active public-key agreement, budget discipline, or sampler-ROM coverage.

Coupling of adjacent games. An equivalence proof establishes a relation between two runs:

$$G_i^A \sim G_{i+1}^A : R_0 \Rightarrow (res_i = res_{i+1}) \vee \text{bad}.$$

This is the formal game-playing statement that the two experiments are identical until bad.

Fundamental lemma of game playing. If the equivalence gives equality outside bad, the probability difference is bounded by

$$|\Pr[G_i^A \Rightarrow 1] - \Pr[G_{i+1}^A \Rightarrow 1]| \leq \Pr[G_i^A : \text{bad}].$$

In the HAETAE sampler lifting proof, the relevant bad event is

$$\text{Bad}_{\text{prequery}} = \{\text{sampler_expand_query}(R) \in Q_A \cup Q_{\text{sampler}}\},$$

where $R \leftarrow \mu_{\text{coins}}$ is fresh signing randomness, Q_A is the adversary hash-query component of the log, and Q_{sampler} is the sampler self-log from earlier signing calls. For one fresh signing call, the corresponding bound uses the combined adversary-hash and sampler-self-query budget. The checked game-level NMA statement also applies a union bound over signing calls:

$$\Pr[\text{Bad}_{\text{prequery}}] \leq q_S \cdot \frac{q_H + q_S}{|\mathcal{C}|}.$$

In EasyCrypt, the checked NMA sampler-prequery fundamental-lemma bound has right-hand side

$$\text{rom_signature_query_budget} \cdot \frac{\text{rom_hash_query_budget} + \text{rom_signature_query_budget}}{\text{challenge_support_cardinality_lower_bound}}.$$

D.7 Exact correspondence of proof-object classes

Table D.1: Exact mathematical correspondence for proof-object classes.

Class	EasyCrypt code form	Mathematical object
Types	<code>typet.</code>	Carrier set $X_t \in \mathbf{Set}$.
Ops	<code>opf:tau1->tau2.</code>	Total function $f : \llbracket \tau_1 \rrbracket \rightarrow \llbracket \tau_2 \rrbracket$.
Predicate ops	<code>opP:tau->bool.</code>	Subset/event $P \subseteq \llbracket \tau \rrbracket$.
Distribution ops	<code>opd:taudistr.</code>	Subprobability law $\mu_d \in \mathbf{SubDist}(\llbracket \tau \rrbracket)$; a <code>[lossless]</code> annotation or <code>is_lossless</code> lemma upgrades it to $\mathbf{Dist}(\llbracket \tau \rrbracket)$.
Loss ops	<code>opL:real.</code>	Real bound term $L \in \mathbb{R}_{\geq 0}$, with non-negativity proved separately.
Module types	<code>moduletypeI=\{\dots\}.</code>	Interface $\mathcal{I}$ of admissible probabilistic algorithms.
Modules	<code>moduleM(\dots)=\{\dots\}.</code>	Probabilistic algorithm, oracle, game, sampler, or reduction.
Game modules	<code>moduleG(A)=\{procmain\dots\}.</code>	Experiment G^A with success probability $\Pr[G^A \Rightarrow 1]$.
Lemmas	<code>lemmaL:Phi.</code>	Theorem $\Gamma \models \Phi$.
Hoare	<code>hoare.</code>	Program theorem $\vdash \{P\}c\{Q\}$.
Probabilistic Hoare	<code>phoare.</code>	Probability theorem $\vdash \Pr[c : Q] \bowtie p$.
Equivalence	<code>equiv. or byequiv\dots</code>	Relational/coupling theorem $c_1 \sim c_2 : R \Rightarrow S$.

Appendix E gives the extracted declaration headers to which these clauses apply. Therefore each listed HAETAE proof object has both a concrete EasyCrypt declaration and a precise mathematical interpretation.

Appendix E

Extracted EasyCrypt Declaration Index

This appendix is generated from `provable-security/proof-files.txt`. It records the exact EasyCrypt declaration headers used by the HAETAE provable-security proof surface. The mathematical interpretation of each listed header is given by Appendix D. Multi-line proof bodies are intentionally not reproduced; this index records the proof objects and program-logic commands that the EasyCrypt checker accepts from the manifest files.

HAETAE_Security.ec

Types

Ops and abbreviations

1 `op correctness_obligation =`

Modules, module types, and clones

Lemmas

```

1 lemma correctness_obligation_holds : correctness_obligation.
2 lemma correctness_perfect &m :
3 lemma euf_cma_security_no_signing &m :
4 lemma euf_cma_security &m :
5 lemma euf_cma_security_with_hop_interfaces &m :
6 lemma euf_cma_security_from_simulated_hop &m :
7 lemma euf_cma_security_from_explicit_boundaries &m :
8 lemma euf_cma_security_from_rom_internal_transcript_hop &m :
9 lemma rom_internal_transcript_hop_from_clear_site_reduction &m :
10 lemma rom_internal_transcript_hop_from_clear_site_counted_reduction &m :
11 lemma bimodal_to_module_sis_reduction_bound &m :
12 lemma euf_cma_security_from_mechanized_boundaries &m :
13 lemma euf_cma_security_from_rom_transcript_and_mechanized_bimodal &m :
14 lemma euf_cma_security_from_rom_transcript_and_counted_bimodal &m :
15 lemma euf_cma_security_from_rom_clear_site_and_mechanized_bimodal &m :
16 lemma euf_cma_security_from_rom_clear_site_and_counted_bimodal &m :
17 lemma euf_cma_security_from_structural_nma_and_counted_bimodal &m :
18 lemma euf_cma_security_from_public_sim_nma_and_counted_bimodal &m :
19 lemma euf_cma_security_from_paper_sim_nma_and_counted_bimodal &m :
20 lemma euf_cma_security_from_rom_paper_sim_nma_and_counted_bimodal &m :
21 lemma euf_cma_security_from_real_signing_paper_sim_nma_and_counted_bimodal &m :
22 lemma euf_cma_security_from_haetae_rejection_paper_sim_nma_and_counted_bimodal &m :
23 lemma exact_hyperball_haetae_rejection_nma_bound_from_paper_sim &m :
24 lemma haetae_rejection_nma_bound_from_exact_hyperball_sampling_hop &m :
25 lemma haetae_rejection_exact_hyperball_sampling_hop_from_real_signing_exact_hyperball_hop &m :
26 lemma haetae_rejection_ro_exact_hyperball_sampling_hop_from_real_signing_ro_exact_hyperball_hop &m :
27 lemma real_signing_ro_exact_hyperball_hop_from_ro_signing_attempt_hop &m :
28 lemma real_signing_ro_exact_hyperball_hop_from_ro_signing_attempt_hop_loss
29 lemma real_signing_ro_exact_hyperball_hop_from_budgeted_lifting &m :
30 lemma haetae_rejection_ro_exact_hyperball_sampling_hop_from_budgeted_lifting
31 lemma euf_cma_security_from_exact_hyperball_paper_sim_bridge_and_counted_bimodal &m :
32 lemma euf_cma_security_from_exact_hyperball_sampling_hop_and_counted_bimodal &m :
33 lemma euf_cma_security_from_ro_exact_hyperball_sampling_hop_and_counted_bimodal &m :
34 lemma euf_cma_security_from_ro_signing_attempt_to_ro_exact_hyperball_hop_and_counted_bimodal &m :
35 lemma real_signing_ro_exact_hyperball_hop_checked_from_loss_budget &m :
36 lemma real_signing_ro_exact_hyperball_hop_from_paper_sample_lifting &m :
37 lemma euf_cma_security_from_paper_sample_lifting_and_counted_bimodal &m :
38 lemma euf_cma_security_from_budgeted_paper_sample_lifting_and_counted_bimodal
39 lemma euf_cma_security_from_checked_ro_sampling_hop_and_counted_bimodal &m :
40 lemma euf_cma_security_from_real_signing_exact_hyperball_hop_and_counted_bimodal &m :

```

Hoare, probabilistic Hoare, and equivalence proof commands

```

1 hoare.

```

Sig_ROM.ec

Types

```

1 type pkey.
2 type skey.
3 type message.
4 type context.
5 type signature.
6 type ro_query.
7 type ro_output.
8 type query = message * context.
9 type signed_query = query * signature.

```

Ops and abbreviations

```
1 op [lossless] dro_output : ro_query -> ro_output distr.
2 op fresh_msg (qs : query list) (m : message) (ctx : context) : bool =
3 op fresh_sig (qs : signed_query list) (m : message) (ctx : context)
```

Modules, module types, and clones

```
1 clone import FullRO as RO with
2 module type Oracle = {
3 module type POracle = {
4 module type Scheme(O : POracle) = {
5 module type CORR_ADV(O : POracle) = {
6 module Correctness(H : Oracle, S : Scheme, A : CORR_ADV) = {
7 module type SignOracle = {
8 module type Adversary(H : POracle, O : SignOracle) = {
9 module type NMA_Adversary(H : POracle) = {
10 module NMA_As_CMA(A : NMA_Adversary) (H : POracle, O : SignOracle) = {
11 module EUF_CMA(H : Oracle, S : Scheme, A : Adversary) = {
12 module UF_NMA(H : Oracle, S : Scheme, A : NMA_Adversary) = {
13 module SUF_CMA(H : Oracle, S : Scheme, A : Adversary) = {
```

Lemmas

```
1 lemma fresh_msg_nil m ctx : fresh_msg [] m ctx.
2 lemma fresh_sig_nil m ctx sig : fresh_sig [] m ctx sig.
3 lemma nma_as_cma_exact &m :
```

Hoare, probabilistic Hoare, and equivalence proof commands

```
1 byequiv (:{glob H, glob S, glob A} ==> {res}) => //.
```

HAETAH_HopGames.ec

Types

```
1 type paper_sim_signature_sample = sig_token * poly * int.
```

Ops and abbreviations

```
1 op transcript_log_consistent
2 op transcript_log_ready (md : mode) (trs : transcript list) : bool =
3 op budgeted_programmed_query_count_discipline
4 op budgeted_paper_sim_signing_count_discipline
5 op budgeted_paper_sim_sampler_freshness_discipline
6 op sampler_rom_covered
```

```

7  op budgeted_programmed_reprogram_discipline
8  op budgeted_programmed_challenge_query_discipline
9  op internal_transcript_state_sound
10 op public_sim_source
11 op public_sim_response_aux_vector :
12 op public_sim_response_vector :
13 op public_sim_hint_vector :
14 op public_sim_commitment_lowbits :
15 op public_sim_commitment_challenge :
16 op public_sim_commitment_highbits :
17 op public_sim_signature_token :
18 op public_sim_signature :
19 op paper_sim_sample_token (s : paper_sim_signature_sample) : sig_token =
20 op paper_sim_sample_lowbits_carrier
21 op paper_sim_sample_entropy (s : paper_sim_signature_sample) : int =
22 op paper_sim_sample_response (s : paper_sim_signature_sample) : polyvec1 =
23 op paper_sim_sample_response_aux
24 op paper_sim_sample_hint (s : paper_sim_signature_sample) : hint_t =
25 op paper_sim_commitment_lowbits
26 op paper_sim_commitment_challenge
27 op paper_sim_commitment_highbits
28 op paper_sim_signature
29 op paper_sim_signature_sample_wf
30 op public_sim_lowbits_carrier
31 op paper_sim_sample_from_public_coins
32 op real_signing_lowbits_carrier
33 op paper_sim_sample_from_real_signing_coins
34 op paper_sim_sample_from_rejection_attempt
35 op paper_sim_abort_fallback_sample
36 op haetae_rejection_sample_from_attempt
37 op structural_to_exact_hyperball_paper_sample_loss_obligation
38 op special_soundness_interfaces_ready (md : mode) : bool =
39 op signing_simulator_sound (md : mode) : bool =
40 op fs_with_aborts_interfaces_ready (md : mode) : bool =

```

Modules, module types, and clones

```

1  module type SigningSimulator(H : SIG.POracle) = {
2  module RealSigningSimulator(S : SIG.Scheme) (H : SIG.POracle) = {
3  module EUF_CMA_SimulatedSign(
4  module EUF_CMA_TranscriptSimulatedSign(
5  module EUF_CMA_InternalTranscriptSign(
6  module EUF_CMA_ROMInternalTranscriptSign(
7  module ROMInternalTranscriptAsNMA(A : SIG.Adversary) (H : SIG.POracle) = {
8  module ROMInternalTranscriptPublicSimAsNMA(A : SIG.Adversary)
9  module type PaperSimSigningSampler = {
10 module ROMPaperSimSigningSampler(H : SIG.POracle) : PaperSimSigningSampler = {
11 module RealSigningPaperSimSampler(H : SIG.POracle) : PaperSimSigningSampler = {
12 module ROSigningAttemptPaperSimSampler(H : SIG.POracle)
13 module HAETAERejectionPaperSimSampler(H : SIG.POracle)
14 module ExactHyperballHAETAERejectionPaperSimSampler(H : SIG.POracle)
15 module ExactHyperballPaperSimSampler(H : SIG.POracle)
16 module ROExactHyperballPaperSimSampler(H : SIG.POracle)
17 module StructuralAttemptPaperSample = {
18 module ExactHyperballPaperSample = {
19 module ROMInternalTranscriptPaperSimAsNMA(
20 module ROMInternalTranscriptBudgetedPaperSimAsNMA(
21 module EUF_CMA_ROMInternalTranscriptCountedSign(
22 module EUF_CMA_ROMProgrammedTranscriptCountedSign(
23 module EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(
24 module EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign(
25 module type ForkingAdversary(H : SIG.POracle) = {
26 module type TranscriptExtractor = {
27 module type BimodalTranscriptExtractor = {
28 module BimodalSpecialSoundness_Game(
29 module SpecialSoundness_Game(
30 module BimodalTranscriptExtractor_As_ModuleSIS(
31 module type BimodalToModuleSIS = {

```

Lemmas

```
1 lemma transcript_log_consistent_nil md pk :
2 lemma transcript_log_consistent_cons md pk qs trs rs m ctx sig tr :
3 lemma transcript_log_consistent_no_min_entropy md pk qs trs rs :
4 lemma transcript_log_consistent_valid md pk qs trs rs :
5 lemma sampler_rom_covered_fresh_after_clean_seed
6 lemma sampler_rom_covered_self_log_preserves
7 lemma sampler_rom_covered_old_log_clean_boundary
8 lemma transcript_log_consistent_ready md pk qs trs rs :
9 lemma transcript_log_valid_member md trs tr :
10 lemma transcript_log_min_entropy_clear_member md trs tr :
11 lemma transcript_log_ready_member_valid md trs tr :
12 lemma transcript_log_ready_member_no_min_entropy md trs tr :
13 lemma transcript_log_ready_no_failure_for_matching_site md trs tr site :
14 lemma transcript_log_ready_no_failure_for_programming_site md trs tr y :
15 lemma transcript_log_ready_no_failure_for_signature_programming_site
16 lemma transcript_log_ready_actual_signature_site_clear_member md trs tr :
17 lemma transcript_log_ready_signature_sites_clear md trs :
18 lemma internal_transcript_state_sound_nil md sk :
19 lemma internal_transcript_state_sound_keygen md sd :
20 lemma internal_transcript_state_sound_no_min_entropy
21 lemma internal_transcript_state_sound_valid md pk sk qs trs rs :
22 lemma internal_transcript_state_sound_log_ready md pk sk qs trs rs :
23 lemma transcript_log_consistent_sign_internal_cons
24 lemma internal_transcript_state_sound_sign_internal_cons
25 lemma sign_internal_transcript_no_min_entropy md pk sk m ctx coins :
26 lemma public_sim_commitment_lowbitsE md sk m ctx coins :
27 lemma public_sim_response_aux_vectorE md sk m ctx coins :
28 lemma public_sim_commitment_challengeE md sk m ctx coins :
29 lemma public_sim_commitment_highbitsE md sk m ctx coins :
30 lemma public_sim_response_vectorE md sk m ctx coins :
31 lemma public_sim_hint_vectorE md sk m ctx coins :
32 lemma public_sim_signature_tokenE md sk m ctx coins :
33 lemma public_sim_signatureE md sk m ctx coins :
34 lemma paper_sim_commitment_lowbits_wf md s :
35 lemma paper_sim_signature_valid md pk m ctx s :
36 lemma public_sim_lowbits_carrier_wf md pk m ctx coins :
37 lemma paper_sim_sample_from_public_coins_wf md pk m ctx coins :
38 lemma paper_sim_sample_response_public_coinsE md pk m ctx coins :
39 lemma paper_sim_sample_response_aux_public_coinsE md pk m ctx coins :
40 lemma paper_sim_sample_hint_public_coinsE md pk m ctx coins :
41 lemma paper_sim_sample_token_public_coinsE md pk m ctx coins :
42 lemma paper_sim_commitment_lowbits_public_coinsE md pk m ctx coins :
43 lemma paper_sim_commitment_challenge_public_coinsE md pk m ctx coins :
44 lemma paper_sim_commitment_highbits_public_coinsE md pk m ctx coins :
45 lemma paper_sim_signature_public_coinsE md pk m ctx coins :
46 lemma real_signing_lowbits_carrier_wf md sk m ctx coins :
47 lemma paper_sim_sample_from_real_signing_coins_wf md sk m ctx coins :
48 lemma paper_sim_sample_response_real_signing_coinsE md sk m ctx coins :
49 lemma paper_sim_sample_response_aux_real_signing_coinsE md sk m ctx coins :
50 lemma paper_sim_sample_hint_real_signing_coinsE md sk m ctx coins :
51 lemma paper_sim_sample_token_real_signing_coinsE md sk m ctx coins :
52 lemma paper_sim_commitment_lowbits_real_signing_coinsE md sk m ctx coins :
53 lemma paper_sim_commitment_challenge_real_signing_coinsE
54 lemma paper_sim_commitment_highbits_real_signing_coinsE
55 lemma paper_sim_signature_real_signing_coinsE md sk m ctx coins :
56 lemma paper_sim_abort_fallback_sample_wf md :
57 lemma paper_sim_sample_from_rejection_attempt_of_coinsE md sk m ctx coins :
58 lemma paper_sim_commitment_lowbits_rejection_attemptE md st :
59 lemma paper_sim_commitment_challenge_rejection_attemptE md pk m ctx st :
60 lemma paper_sim_commitment_highbits_rejection_attemptE md pk m ctx st :
61 lemma paper_sim_signature_rejection_attempt_fieldsE md pk m ctx st :
62 lemma paper_sim_signature_rejection_attempt_matches_state md pk m ctx st :
63 lemma paper_sim_signature_rejection_attempt_of_coinsE md sk m ctx coins :
64 lemma haetae_rejection_sample_from_attempt_signatureE md pk m ctx st :
65 lemma haetae_rejection_sample_from_attempt_no_fallback md pk m ctx st :
66 lemma haetae_rejection_sample_from_attempt_of_coinsE md sk m ctx coins :
67 lemma haetae_rejection_sample_from_attempt_of_coins_wf md sk m ctx coins :
68 lemma budgeted_paper_sim_sign_oracle_preserves_signing_count :
69 lemma adversary_budgeted_paper_sim_signing_count_preserved :
70 lemma budgeted_paper_sim_main_signing_count_preserved :
71 lemma sampler_expand_query_point_bound q :
72 lemma sampler_expand_query_log_fset_bound (qs : ro_query list) :
73 lemma sampler_expand_query_log_budget_bound (qs : ro_query list) hc :
74 lemma sampler_expand_query_joint_log_budget_bound
```

APPENDIX E. EXTRACTED EASYCRYPT DECLARATION INDEX

```
75 lemma concrete_lazy_rom_sampler_expand_query_fresh_le
76 lemma concrete_lazy_rom_sampler_expand_query_fresh_eq
77 lemma concrete_lazy_rom_get_preserves_sampler_rom_covered
78 lemma concrete_lazy_rom_get_preserves_sampler_rom_covered_arg
79 lemma concrete_lazy_rom_get_preserves_sampler_rom_covered_budgeted_logs :
80 lemma concrete_lazy_rom_get_preserves_sampler_rom_covered_hash_cons
81 lemma concrete_lazy_rom_get_preserves_sampler_rom_covered_sampler_cons
82 lemma sampler_bad_prequery_one_step_bound_nonnegative :
83 lemma budgeted_paper_sim_sampler_bad_prequery_forge_fel_bound pk &m :
84 lemma budgeted_paper_sim_sampler_bad_prequery_forge_fel_phoare :
85 lemma budgeted_paper_sim_sampler_bad_prequery_nma_fel_bound &m :
86 lemma concrete_lazy_rom_sampler_expand_query_fresh_phoare
87 lemma concrete_lazy_rom_sampler_expand_query_fresh_arg_phoare
88 lemma concrete_lazy_rom_sampler_expand_query_fresh_arg_clean_phoare
89 lemma concrete_ro_signing_attempt_sample_with_seed_fresh_structural_le
90 lemma concrete_ro_signing_attempt_sample_with_seed_fresh_structural_arg_le
91 lemma concrete_ro_signing_attempt_sample_with_seed_fresh_structural_arg_clean_le
92 lemma concrete_lazy_rom_sampler_expand_query_guarded_clean_phoare
93 lemma concrete_ro_signing_attempt_sample_with_seed_guarded_clean_structural_arg_le
94 lemma concrete_ro_exact_hyperball_sample_with_seed_fresh_structural_eq
95 lemma concrete_ro_exact_hyperball_sample_with_seed_exact
96 lemma concrete_ro_exact_hyperball_sample_with_seed_exact_no_seed
97 lemma concrete_ro_exact_hyperball_sample_with_seed_exact_pr
98 lemma concrete_ro_signing_attempt_sample_with_seed_preserves_sampler_rom_covered
99 lemma concrete_ro_exact_hyperball_sample_with_seed_preserves_sampler_rom_covered
100 lemma concrete_ro_signing_attempt_sample_with_seed_preserves_sampler_rom_covered_budgeted_logs :
101 lemma concrete_ro_exact_hyperball_sample_with_seed_preserves_sampler_rom_covered_budgeted_logs :
102 lemma concrete_budgeted_ro_signing_attempt_o_sign_preserves_sampler_rom_covered :
103 lemma concrete_budgeted_ro_exact_hyperball_o_sign_preserves_sampler_rom_covered :
104 lemma concrete_budgeted_ro_signing_attempt_o_sign_clean_sampler_fresh :
105 lemma internal_transcript_sign_oracle_preserves_state :
106 lemma adversary_internal_transcript_state_preserved :
107 lemma internal_transcript_sign_main_state_sound :
108 lemma internal_transcript_sign_main_transcripts_valid :
109 lemma internal_transcript_sign_main_log_ready :
110 lemma internal_transcript_sign_main_no_min_entropy :
111 lemma rom_internal_transcript_sign_oracle_preserves_state :
112 lemma adversary_rom_internal_transcript_state_preserved :
113 lemma rom_internal_transcript_sign_main_state_sound :
114 lemma rom_internal_transcript_sign_main_transcripts_valid :
115 lemma rom_internal_transcript_sign_main_log_ready :
116 lemma rom_internal_transcript_sign_main_no_min_entropy :
117 lemma rom_internal_transcript_sign_main_no_failure_for_signature_site :
118 lemma rom_internal_transcript_sign_main_signature_sites_clear :
119 lemma rom_internal_transcript_bad_forgery_zero &m :
120 lemma rom_internal_transcript_bad_forgery_bound &m :
121 lemma rom_internal_transcript_bad_forgery_counted_bound &m :
122 lemma counted_rom_internal_transcript_sign_oracle_preserves_budget_state :
123 lemma adversary_counted_rom_internal_transcript_budget_state_preserved :
124 lemma counted_rom_internal_transcript_main_bad_clear :
125 lemma counted_rom_internal_transcript_main_bad_false :
126 lemma counted_rom_internal_transcript_counted_bad_zero &m :
127 lemma counted_rom_internal_transcript_budget_sound &m :
128 lemma counted_rom_internal_transcript_counted_bad_subunit &m :
129 lemma programmed_counted_sign_oracle_preserves_min_entropy_clear :
130 lemma adversary_programmed_counted_min_entropy_preserved :
131 lemma programmed_counted_main_min_entropy_clear :
132 lemma programmed_counted_min_entropy_bad_zero &m :
133 lemma programmed_counted_min_entropy_bad_bound &m :
134 lemma programmed_counted_prequery_reprogramming_bad_bound_from_budget_expr &m :
135 lemma budgeted_counted_lazy_rom_get_true :
136 lemma budgeted_counted_lazy_rom_init_true :
137 lemma budgeted_counted_lazy_rom_bad_true :
138 lemma budgeted_programmed_hash_get_preserves_challenge_query_discipline :
139 lemma budgeted_programmed_sign_oracle_preserves_challenge_query_discipline :
140 lemma adversary_budgeted_programmed_challenge_query_discipline_preserved :
141 lemma budgeted_programmed_main_challenge_query_discipline :
142 lemma budgeted_programmed_main_challenge_query_count_bound :
143 lemma budgeted_programmed_adaptive_prequery_lift
144 lemma budgeted_programmed_adaptive_prequery_lift_checked_31
145 lemma budgeted_programmed_hash_get_preserves_discipline :
146 lemma budgeted_programmed_reprogram_discipline_after_actual_signature
147 lemma budgeted_programmed_sign_oracle_preserves_discipline :
148 lemma adversary_budgeted_programmed_discipline_preserved :
149 lemma budgeted_programmed_main_reprogram_clear :
150 lemma budgeted_programmed_bad_reprogram_zero &m :
151 lemma budgeted_programmed_prequery_reprogram_bad_bound_from_prequery &m :
```

```

152 lemma budgeted_programmed_hash_get_preserves_query_budget :
153 lemma budgeted_programmed_sign_oracle_preserves_query_budget :
154 lemma adversary_budgeted_programmed_query_budget_preserved :
155 lemma budgeted_programmed_main_query_budget_preserved :
156 lemma budgeted_programmed_query_budget_certifies_branch_bound &m :
157 lemma budgeted_programmed_query_budget_certifies_branch_bound_from_prequery
158 lemma budgeted_programmed_query_budget_certifies_checked_31_bound
159 lemma budgeted_sampled_direct_counted_lazy_rom_get_true :
160 lemma budgeted_sampled_direct_counted_lazy_rom_init_true :
161 lemma budgeted_sampled_direct_counted_lazy_rom_bad_true :
162 lemma budgeted_sampled_direct_hash_get_preserves_challenge_query_discipline :
163 lemma budgeted_sampled_direct_sign_oracle_preserves_challenge_query_discipline :
164 lemma adversary_budgeted_sampled_direct_challenge_query_discipline_preserved :
165 lemma budgeted_sampled_direct_main_challenge_query_discipline :
166 lemma budgeted_sampled_direct_hash_get_preserves_discipline :
167 lemma budgeted_sampled_direct_sign_oracle_preserves_discipline :
168 lemma adversary_budgeted_sampled_direct_discipline_preserved :
169 lemma budgeted_sampled_direct_main_reprogram_clear :
170 lemma budgeted_sampled_direct_bad_reprogram_zero &m :
171 lemma budgeted_sampled_direct_one_step_bound_nonnegative :
172 lemma budgeted_sampled_direct_sampler_message_hash_prequery_bound
173 lemma budgeted_sampled_direct_bad_prequery_bound &m :
174 lemma budgeted_sampled_direct_prequery_reprogram_bad_bound &m :
175 lemma budgeted_sampled_direct_prequery_reprogram_bad_checked_31 &m :
176 lemma budgeted_programmed_counted_sampled_direct_prequery_reprogram_bad_exact
177 lemma budgeted_programmed_prequery_reprogram_bad_checked_31_via_direct_sampler
178 lemma haetae_rom_internal_transcript_erasure_exact &m :
179 lemma rom_internal_transcript_structural_nma_bound &m :
180 lemma rom_internal_transcript_clear_site_structural_nma_bound &m :
181 lemma rom_internal_nma_public_sim_exact &m :
182 lemma rom_internal_nma_public_sim_rom_paper_sim_exact &m :
183 lemma rom_internal_nma_real_signing_paper_sim_exact &m :
184 lemma rom_internal_nma_real_signing_ro_attempt_paper_sim_exact &m :
185 lemma concrete_ro_signing_attempt_to_exact_hyperball_sample_with_seed_fresh_loss
186 lemma concrete_ro_signing_attempt_to_exact_hyperball_sample_with_seed_clean_loss
187 lemma concrete_budgeted_o_sign_sample_with_seed_clean_loss_surface
188 lemma concrete_budgeted_o_sign_old_log_sample_with_seed_clean_loss_surface
189 lemma concrete_budgeted_o_sign_old_log_signature_from_sample_clean_loss_surface
190 lemma concrete_budgeted_o_sign_old_log_signature_clean_event_loss_surface
191 lemma concrete_frozen_old_log_sample_clean_event_loss_surface
192 lemma concrete_frozen_old_log_signature_clean_event_loss_surface
193 lemma concrete_frozen_old_log_self_logged_signature_clean_event_loss_surface
194 lemma concrete_budgeted_self_logged_signature_clean_event_loss_surface
195 lemma concrete_lazy_rom_get_lossless :
196 lemma structural_attempt_exact_hyperball_paper_sample_one_step_loss
197 lemma exact_hyperball_paper_sample_one_step_upper
198 lemma rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting &m :
199 lemma rom_internal_budgeted_ro_signing_attempt_signing_call_loss_bound &m :
200 lemma rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting &m :
201 lemma rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_clean_lifting &m :
202 lemma rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_clean_conditioned_lifting &m :
203 lemma rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_checked_clean_lifting &m :
204 lemma rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_budgeted_lifting &m :
205 lemma rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_budgeted_checked_clean_lifting &m :
206 lemma rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_bound &m :
207 lemma concrete_lazy_rom_get_preserves_signature_event
208 lemma concrete_lazy_rom_get_preserves_signature_event_with_pk
209 lemma concrete_lazy_rom_get_preserves_current_signature_event
210 lemma concrete_lazy_rom_get_true :
211 lemma concrete_lazy_rom_get_preserves_budgeted_clean_signature_event
212 lemma concrete_lazy_rom_get_preserves_budgeted_clean_signature_event_hoare
213 lemma concrete_ro_signing_attempt_sample_with_seed_lossless :
214 lemma concrete_budgeted_ro_signing_attempt_o_sign_active_clean_signature_structural_le
215 lemma concrete_budgeted_ro_exact_hyperball_o_sign_active_signature_exact
216 lemma concrete_budgeted_ro_exact_hyperball_sample_with_seed_to_o_sign_active_projection
217 lemma concrete_budgeted_ro_signing_attempt_o_sign_to_self_logged_sample_active_projection
218 lemma concrete_budgeted_o_sign_clean_one_call_loss_from_self_logged_surface
219 lemma concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_clean_conditioned_lifting &
    m :
220 lemma concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_clean_lifting &m
    :
221 lemma
    concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting
    &m :
222 lemma rom_internal_nma_real_signing_haetae_rejection_paper_sim_exact &m :
223 lemma rom_internal_nma_exact_hyperball_rejection_paper_sim_no_fallback_exact &m :
224 lemma public_sim_to_paper_sim_nma_hop_from_sampling_hop &m :

```

```

225 lemma transcript_logging_erasure_exact &m :
226 lemma real_signing_simulator_exact &m :
227 lemma bimodal_special_soundness_lift_exact &m md :
228 lemma special_soundness_interfaces_ready_from_forking md :
229 lemma special_soundness_interfaces_public_reconstruction md :
230 lemma special_soundness_interfaces_ready_sound md :
231 lemma special_soundness_interfaces_ready_sound_with_public_reconstruction md :
232 lemma signing_simulator_sound_current md :
233 lemma rejection_sampling_bound_from_fs_bound :
234 lemma fs_with_aborts_interfaces_ready_from_parts md :
235 lemma fs_with_aborts_interfaces_ready_from_forking md :
236 lemma fs_with_aborts_interfaces_ready_holds md :
237 lemma structural_to_exact_hyperball_paper_sample_loss_from_obligation
238 lemma structural_to_exact_hyperball_paper_sample_loss_from_attempt_loss
239 lemma structural_to_exact_hyperball_paper_sample_loss_obligation_from_attempt_loss
240 lemma structural_to_exact_hyperball_paper_sample_loss_from_coin_sample_pair_loss
241 lemma structural_to_exact_hyperball_paper_sample_loss_obligation_from_coin_sample_pair_loss
242 lemma dexact_hyperball_reference_paper_sim_attempt_boundary_ready
243 lemma dexact_hyperball_full_rejection_paper_sim_attempt_boundary_ready
244 lemma dexact_hyperball_haetae_rejection_sample_no_fallback_mu
245 lemma dexact_hyperball_haetae_rejection_sample_signature_no_fallback_mu
246 lemma dexact_hyperball_paper_sim_rejection_attempt_signature_mu
247 lemma dexact_hyperball_haetae_rejection_sample_signature_mu

```

Hoare, probabilistic Hoare, and equivalence proof commands

```

1   hoare.
2   hoare.
3   hoare.
4   hoare.
5   hoare.
6   hoare.
7   hoare.
8   hoare.
9   hoare.
10  hoare.
11  hoare.
12 equiv budgeted_programmed_counted_sampled_direct_hash_get_equiv :
13 equiv budgeted_programmed_counted_sampled_direct_sign_equiv :
14 equiv adversary_budgeted_programmed_counted_sampled_direct_equiv :
15 equiv budgeted_programmed_counted_sampled_direct_main_equiv :
16 byequiv budgeted_programmed_counted_sampled_direct_main_equiv => //.
17 equiv haetae_rom_internal_oracle_erasure :
18 equiv adversary_haetae_rom_internal_erasure :
19 byequiv (:={glob H, glob A}==>={res})=> //.
20 equiv rom_internal_oracle_structural_nma :
21 equiv adversary_rom_internal_structural_nma :
22 byequiv (:={glob H, glob A}==>res{1}>res{2})=> //.
23 equiv rom_internal_public_sim_nma_oracle :
24 equiv adversary_rom_internal_public_sim_nma :
25 byequiv (:={glob H, glob A}==>={res})=> //.
26 equiv rom_internal_public_sim_rom_paper_sim_nma_oracle :
27 equiv adversary_rom_internal_public_sim_rom_paper_sim_nma :
28 byequiv (:={glob H, glob A}==>={res})=> //.
29 equiv rom_internal_real_signing_paper_sim_nma_oracle :
30 equiv adversary_rom_internal_real_signing_paper_sim_nma :
31 byequiv (:={glob H, glob A}==>={res})=> //.
32 equiv real_signing_ro_attempt_paper_sim_nma_oracle :
33 equiv adversary_real_signing_ro_attempt_paper_sim_nma :
34 byequiv (:={glob H, glob A}==>={res})=> //.
35 equiv real_signing_rejection_paper_sim_nma_oracle :
36 equiv adversary_real_signing_rejection_paper_sim_nma :
37 byequiv (:={glob H, glob A}==>={res})=> //.
38 equiv exact_hyperball_haetae_rejection_paper_sim_nma_oracle :
39 equiv adversary_exact_hyperball_haetae_rejection_paper_sim_nma :
40 byequiv (:={glob H, glob A}==>={res})=> //.
41 equiv transcript_logging_oracle_erasure :
42 equiv adversary_transcript_logging_erasure :
43 byequiv (:={glob H, glob S, glob A, glob Sim}==>={res})=> //.
44 equiv real_signing_oracle_equiv :
45 equiv adversary_real_signing_equiv :
46 byequiv (:={glob H, glob S, glob A}==>={res})=> //.

```

```
47 | byequiv (:={glob H, glob S, glob F, glob X} ==> ={res}) => //.
```

HAETAE_Reductions.ec

Types

Ops and abbreviations

```
1 | op mlwe_hardness_term : real.
2 | op module_sis_hardness_term : real.
3 | op bimodal_to_msis_reduction_loss_term : real = 0%r.
4 | op bimodal_selftarget_msis_hardness_term =
5 | op signature_query_count : real = 2%r.
6 | op hash_query_count : real = 2%r.
7 | op signature_query_budget_count : int = 2.
8 | op hash_query_budget_count : int = 2.
9 | op abort_probability : real = 0%r.
10 | op min_entropy_loss_factor : real = 2%r.
11 | op challenge_support_bits_lower_bound : int = 58.
12 | op challenge_support_cardinality_lower_bound : real =
13 | op rom_hash_query_budget : real = hash_query_count.
14 | op rom_signature_query_budget : real = signature_query_count.
15 | op rom_total_query_budget =
16 | op counted_rom_collision_term =
17 | op counted_rom_prequery_term =
18 | op counted_rom_min_entropy_term =
19 | op counted_rom_prequery_reprogramming_term =
20 | op counted_rom_programming_loss_term =
21 | op signing_query_scale =
22 | op fs_with_aborts_reprogramming_term =
23 | op fs_with_aborts_min_entropy_term =
24 | op rejection_sampling_loss_term =
25 | op haetae_euf_bound =
26 | op haetae_euf_counted_rom_bound =
```

Modules, module types, and clones

Lemmas

```
1 | lemma haetae_euf_boundE :
2 | lemma haetae_euf_bound_groupedE :
3 | lemma haetae_euf_bound_rejection_groupedE :
4 | lemma haetae_euf_counted_rom_bound_groupedE :
5 | lemma signing_query_scaleE :
6 | lemma signing_query_scale_nonnegative :
7 | lemma fs_with_aborts_reprogramming_term_nonnegative :
8 | lemma fs_with_aborts_min_entropy_term_nonnegative :
9 | lemma rom_hash_query_budget_nonnegative :
10 | lemma rom_signature_query_budget_nonnegative :
11 | lemma hash_query_budget_countE :
```

```

12 lemma signature_query_budget_countE :
13 lemma hash_query_budget_count_nonnegative :
14 lemma signature_query_budget_count_nonnegative :
15 lemma min_entropy_loss_factor_nonnegative :
16 lemma challenge_support_cardinality_lower_bound_positive :
17 lemma challenge_support_cardinality_lower_bound_nonnegative :
18 lemma rom_total_query_budget_nonnegative :
19 lemma counted_rom_collision_term_nonnegative :
20 lemma counted_rom_prequery_term_nonnegative :
21 lemma counted_rom_min_entropy_term_nonnegative :
22 lemma counted_rom_prequery_reprogramming_term_nonnegative :
23 lemma counted_rom_programming_loss_term_nonnegative :
24 lemma counted_rom_collision_termE :
25 lemma counted_rom_prequery_termE :
26 lemma counted_rom_min_entropy_termE :
27 lemma counted_rom_prequery_reprogramming_termE :
28 lemma counted_rom_programming_loss_termE :
29 lemma counted_rom_programming_loss_term_subunit :
30 lemma counted_rom_programming_loss_term_positive :
31 lemma counted_rom_programming_loss_term_le1 :
32 lemma counted_rom_prequery_reprogramming_budget_numeratorE :
33 lemma counted_rom_budget_numeratorE :
34 lemma counted_rom_programming_loss_term_cardinalityE :
35 lemma counted_rom_prequery_reprogramming_term_cardinalityE :
36 lemma counted_rom_programming_loss_term_concreteE :
37 lemma counted_rom_prequery_reprogramming_term_concreteE :
38 lemma counted_rom_programming_loss_term_checked_subunit :
39 lemma counted_rom_support_lower_bound_summary :
40 lemma counted_rom_support_dominates_budget :
41 lemma counted_rom_prequery_reprogramming_support_dominates_budget :
42 lemma counted_rom_prequery_reprogramming_term_subunit :
43 lemma counted_rom_prequery_reprogramming_from_query_budgets_and_support :
44 lemma counted_rom_programming_loss_splitE :
45 lemma counted_rom_loss_from_query_budgets_and_support :
46 lemma rejection_sampling_loss_term_nonnegative :
47 lemma fs_with_aborts_min_entropy_termE :
48 lemma fs_with_aborts_min_entropy_term_ge1 :
49 lemma rejection_sampling_loss_term_ge1 :
50 lemma bimodal_to_msis_reduction_loss_termE :

```

Hoare, probabilistic Hoare, and equivalence proof commands

HAETAET_ROM.ec

Types

```

1 type ro_query =
2 type ro_output = crh * challenge * matrix * random_coins.

```

Ops and abbreviations

```

1 op message_hash_query (pk : pkey) (ctx : context) (m : message) : ro_query =
2 op challenge_hash_query
3 op matrix_expand_query (md : mode) (sd : seed) : ro_query =
4 op sampler_expand_query (coins : random_coins) : ro_query =
5 op ro_crh_zero : crh = nseq crhbytes 0.

```

```

6  op ro_challenge_zero : challenge = poly_zero.
7  op ro_matrix_zero : matrix = [].
8  op ro_random_coins_zero : random_coins = nseq seedbytes 0.
9  op ro_output_zero : ro_output =
10 op ro_output_to_crh (y : ro_output) : crh = y.'1.
11 op ro_output_to_challenge (y : ro_output) : challenge = y.'2.
12 op ro_output_to_matrix (y : ro_output) : matrix = y.'3.
13 op ro_output_to_random_coins (y : ro_output) : random_coins = y.'4.
14 op ro_output_of_crh (mu : crh) : ro_output =
15 op ro_output_of_challenge (ch : challenge) : ro_output =
16 op ro_output_of_matrix (a : matrix) : ro_output =
17 op ro_output_of_random_coins (coins : random_coins) : ro_output =
18 op dmatrix_for_query (x : mode * seed) : matrix distr = dmatrix x.'1.
19 op dro_output (q : ro_query) : ro_output distr =
20 op ro_message_hash (y : ro_output) : crh = ro_output_to_crh y.
21 op ro_challenge_hash (y : ro_output) : challenge = ro_output_to_challenge y.
22 op ro_matrix (y : ro_output) : matrix = ro_output_to_matrix y.
23 op ro_signing_coins (y : ro_output) : random_coins =

```

Modules, module types, and clones

```

1  clone import FullRO as HAETAE_RO with
2  module type Oracle = {
3  module type POracle = {

```

Lemmas

```

1  lemma matrix_query_distribution_lossless x :
2  lemma ro_output_distribution_lossless q :
3  lemma sampler_expand_query_dro_output_ro_signing_coins

```

Hoare, probabilistic Hoare, and equivalence proof commands

HAETAE_ROM_Programming.ec

Types

```

1  type programming_site = ro_query * ro_output.
2  type counted_rom_event =

```

Ops and abbreviations

```

1  op programming_site_query (site : programming_site) : ro_query = site.'1.
2  op programming_site_output (site : programming_site) : ro_output = site.'2.
3  op programming_site_of_transcript
4  op signature_programming_site_of_transcript
5  op transcript_signature_challenge_output (tr : transcript) : ro_output =
6  op actual_signature_programming_site
7  op honest_signing_programming_site
8  op honest_signing_programming_site_query
9  op programmed_site_query_min_entropy_bound_for
10 op programmed_site_query_min_entropy_bound
11 op programming_site_matches_transcript
12 op reprogramming_failure
13 op challenge_entropy_support_ok (md : mode) (tr : transcript) : bool =
14 op min_entropy_failure (md : mode) (tr : transcript) : bool =
15 op transcript_log_min_entropy_clear (md : mode) (trs : transcript list) : bool =
16 op signing_record_log_min_entropy_clear
17 op fs_with_aborts_failure
18 op transcript_signature_programming_site_clear
19 op transcript_log_signature_programming_sites_clear
20 op fs_with_aborts_bound_obligation : bool =
21 op counted_event_is_hash_query (ev : counted_rom_event) : bool =
22 op counted_event_is_signature_site (ev : counted_rom_event) : bool =
23 op counted_event_is_programmed_site (ev : counted_rom_event) : bool =
24 op counted_event_is_min_entropy_check (ev : counted_rom_event) : bool =
25 op ro_query_is_challenge (q : ro_query) : bool =
26 op challenge_query_log_count (qs : ro_query list) : int =
27 op counted_trace_hash_queries (evs : counted_rom_event list) : int =
28 op counted_trace_signature_sites (evs : counted_rom_event list) : int =
29 op counted_trace_programmed_sites (evs : counted_rom_event list) : int =
30 op counted_trace_min_entropy_checks (evs : counted_rom_event list) : int =
31 op programming_sites_same_query
32 op programming_sites_conflict
33 op programming_site_prequeried
34 op programming_site_reprograms
35 op programming_site_fresh
36 op programming_transcript_fresh
37 op rom_counted_programming_bad
38 op rom_counted_state_bad
39 op counted_rom_programming_bound_obligation : bool =
40 op programmed_site_entropy_token_from_query (q : ro_query) : int =
41 op programming_site_conflict_free
42 op programming_site_log_conflict_free_for_honest
43 op counted_lazy_rom_bad_flags_budget_sound (p : real) : bool =

```

Modules, module types, and clones

```

1  module CountedLazyROM(0 : Oracle) = {
2  module type SigningCoinSampler = {
3  module DirectSigningCoinSampler : SigningCoinSampler = {
4  module type CountedROMInterface = {
5  module type CountedROMClient(C : CountedROMInterface) = {
6  module CountedLazyROMBadGame(
7  module ProgramChallenge(0 : Oracle) = {
8  module type TranscriptProgrammer(0 : Oracle) = {
9  module TranscriptProgrammerFromSite(0 : Oracle) = {

```

Lemmas

```

1  lemma challenge_query_log_count_cons_le q qs :
2  lemma challenge_query_log_count_nonchallenge_cons q qs :
3  lemma challenge_query_log_count_message_cons pk ctx m qs :
4  lemma challenge_query_log_count_matrix_cons md sd qs :
5  lemma challenge_query_log_count_sampler_cons coins qs :
6  lemma counted_lazy_rom_init_clears :
7  lemma counted_lazy_rom_init_clears_hash_queries :
8  lemma counted_lazy_rom_get_preserves_clear :

```

```

9 lemma counted_lazy_rom_get_nonchallenge_preserves_challenge_query_count n :
10 lemma counted_lazy_rom_get_preserves_bad_clear :
11 lemma counted_lazy_rom_get_preserves_min_entropy_clear :
12 lemma counted_lazy_rom_observe_transcript_preserves_clear :
13 lemma counted_lazy_rom_observe_transcript_preserves_min_entropy_clear :
14 lemma counted_lazy_rom_observe_transcript_preserves_bad_clear :
15 lemma counted_lazy_rom_program_preserves_bad_clear_if_fresh :
16 lemma counted_lazy_rom_program_preserves_min_entropy_clear :
17 lemma counted_lazy_rom_bad_preserves_min_entropy_clear :
18 lemma counted_lazy_rom_bad_false_when_clear :
19 lemma programming_site_self_matches md tr y :
20 lemma programming_site_self_no_reprogramming_failure md tr y :
21 lemma fs_with_aborts_failureE md tr site :
22 lemma programming_site_prequeried_cons qs site :
23 lemma programming_site_prequeried_cons_preserve q qs site :
24 lemma programming_site_prequeried_output_irrelevant qs q y1 y2 :
25 lemma programming_site_prequeried_witness qs q y :
26 lemma honest_signing_programming_site_queryE md sk m ctx coins :
27 lemma honest_signing_programming_site_query_entropy_token
28 lemma honest_signing_programming_site_query_token_injective
29 lemma signing_distribution_programming_site_query_spread
30 lemma honest_signing_programming_site_query_spread
31 lemma honest_signing_programming_site_query_spread_bound
32 lemma honest_signing_programming_site_query_spread_bound_for
33 lemma honest_signing_programming_site_query_is_challenge
34 lemma programming_site_prequeried_honest_challenge_filter
35 lemma programming_site_prequeried_honest_message_consE
36 lemma honest_signing_programming_site_outputE md sk m ctx coins :
37 lemma honest_signing_programming_site_functional_output
38 lemma honest_signing_programming_sites_no_conflict
39 lemma actual_signature_programming_site_sign_internalE
40 lemma actual_signature_programming_site_sign_internal_any_pkE
41 lemma actual_signature_programming_site_prequeried_after_cons
42 lemma actual_signature_programming_site_prequeried_after_message_hash
43 lemma actual_signature_programming_site_prequeried_message_hashE
44 lemma actual_signature_programming_site_prequeried_message_hash_any_pkE
45 lemma programming_site_log_conflict_free_for_honest_nil md sk :
46 lemma programming_site_log_conflict_free_for_honest_cons
47 lemma programming_site_conflict_free_no_reprograms site sites :
48 lemma actual_signature_programming_site_sign_internal_conflict_step
49 lemma programmed_site_reprogram_one_step_conflict_free_zero
50 lemma programmed_site_prequery_one_step_fset_bound_for
51 lemma programmed_site_prequery_one_step_fset_bound
52 lemma programmed_site_prequery_one_step_budget_bound_for
53 lemma programmed_site_prequery_one_step_budget_bound
54 lemma programmed_site_prequery_one_step_challenge_fset_bound_for
55 lemma programmed_site_prequery_one_step_challenge_fset_bound
56 lemma programmed_site_prequery_one_step_challenge_budget_bound_for
57 lemma programmed_site_prequery_one_step_challenge_budget_bound
58 lemma programmed_site_prequery_one_step_challenge_concrete_bound
59 lemma programmed_site_prequery_one_step_challenge_concrete_bound_for
60 lemma challenge_query_log_count_budget_card_bound hc qs :
61 lemma programmed_site_prequery_one_step_challenge_counter_bound
62 lemma programmed_site_prequery_one_step_challenge_counter_bound_for
63 lemma programmed_site_prequery_one_step_message_hash_counter_bound
64 lemma direct_signing_coin_sampler_prequery_bound
65 lemma direct_signing_coin_sampler_budgeted_prequery_bound
66 lemma direct_signing_coin_sampler_message_hash_prequery_bound
67 lemma direct_signing_coin_sampler_message_hash_budgeted_prequery_bound
68 lemma programmed_site_prequery_one_step_concrete_bound
69 lemma programmed_site_prequery_reprogram_one_step_concrete_bound
70 lemma programmed_site_query_min_entropy_bound_constant_impossible
71 lemma programming_site_reprograms_conflict_cons site old programmed :
72 lemma rom_counted_programming_bad_min_entropy_queries programmed :
73 lemma rom_counted_programming_bad_prequery_cons queries programmed site
74 lemma counted_rom_programming_bound_obligation_holds :
75 lemma counted_lazy_rom_bad_flags_universal_bound &m :
76 lemma counted_lazy_rom_bad_flags_query_budget_bound &m :
77 lemma counted_lazy_rom_bad_flags_query_budget_bound_subunit &m :
78 lemma fs_with_aborts_no_programming_failure md tr y :
79 lemma transcript_signature_challenge_output_matches tr :
80 lemma honest_transcript_challenge_entropy_support md sk m ctx coins :
81 lemma honest_transcript_no_min_entropy_failure md sk m ctx coins :
82 lemma signing_transcript_relation_challenge_entropy_support
83 lemma signing_transcript_relation_no_min_entropy_failure
84 lemma signing_record_relation_no_min_entropy_failure md pk r :
85 lemma signing_record_log_sound_no_min_entropy md pk rs :

```

```

86 lemma signing_record_log_sound_transcripts_no_min_entropy md pk rs :
87 lemma fs_with_aborts_no_failure_for_signing_relation
88 lemma fs_with_aborts_no_failure_for_signing_record md pk r site :
89 lemma fs_with_aborts_no_failure_for_honest_programming md sk m ctx coins y :
90 lemma transcript_valid_programming_site_signature_query md tr y :
91 lemma transcript_valid_signature_programming_site_matches md tr y :
92 lemma transcript_valid_signature_programming_site_no_reprogramming
93 lemma transcript_valid_actual_signature_programming_site_matches md tr :
94 lemma transcript_valid_actual_signature_programming_site_no_reprogramming
95 lemma valid_forking_pair_signature_programming_site_query md tr1 tr2 y1 y2 :
96 lemma valid_forking_pair_programming_site_query md tr1 tr2 y1 y2 :
97 lemma valid_forking_pair_self_programming_sites_no_reprogramming
98 lemma fs_bound_parts_nonnegative :
99 lemma fs_with_aborts_bound_obligation_holds :

```

Hoare, probabilistic Hoare, and equivalence proof commands

HAETAEScheme.ec

Types

Ops and abbreviations

```

1 op haetae_mode : mode.

```

Modules, module types, and clones

```

1 clone import Sig_ROM as SIG with
2 module HAETAES(H : SIG.POracle) = {

```

Lemmas

Hoare, probabilistic Hoare, and equivalence proof commands

HAETAETranscript.ec

Types

```
1 type transcript =
```

Ops and abbreviations

```
1 op signature_commitment_highbits :
2 op signature_commitment_lowbits :
3 op signature_message_hash :
4 op signature_challenge :
5 op transcript_pk (tr : transcript) : pkey = tr.'1.
6 op transcript_message (tr : transcript) : message = tr.'2.
7 op transcript_context (tr : transcript) : context = tr.'3.
8 op transcript_commitment_highbits (tr : transcript) : polyveck = tr.'4.
9 op transcript_commitment_lowbits (tr : transcript) : poly = tr.'5.
10 op transcript_message_hash (tr : transcript) : crh = tr.'6.
11 op transcript_challenge (tr : transcript) : challenge = tr.'7.
12 op transcript_signature (tr : transcript) : signature = tr.'8.
13 op transcript_of_signature :
14 op transcript_challenge_query (md : mode) (tr : transcript) : ro_query =
15 op transcript_signature_challenge_query (md : mode) (tr : transcript) :
16 op transcript_verifies (md : mode) (tr : transcript) : bool =
17 op transcript_challenge_matches (tr : transcript) (y : ro_output) : bool =
18 op transcript_signature_challenge_matches
19 op transcript_fields_match_signature (tr : transcript) : bool =
20 op transcript_valid (md : mode) (tr : transcript) : bool =
21 op transcript_public_equation (md : mode) (tr : transcript) : bool =
22 op same_fork_target (tr1 tr2 : transcript) : bool =
23 op distinct_fork_challenges (tr1 tr2 : transcript) : bool =
24 op valid_forking_pair (md : mode) (tr1 tr2 : transcript) : bool =
25 op transcript_response (tr : transcript) : polyveck =
26 op transcript_response_aux (tr : transcript) : polyveck =
27 op transcript_public_key_challenge_term (md : mode) (tr : transcript) :
28 op active_public_reconstruction (md : mode) (tr : transcript) : bool =
29 op inactive_public_reconstruction (md : mode) (tr : transcript) : bool =
30 op forking_difference_solution
31 op fork_response_aux_difference (md : mode) (tr1 tr2 : transcript) :
32 op fork_public_challenge_relation (md : mode) (tr1 tr2 : transcript) : bool =
33 op fork_response_aux_relation (tr1 tr2 : transcript) : bool =
34 op fork_left_active_public_challenge_relation
35 op fork_right_active_public_challenge_relation
36 op fork_public_reconstruction_cases
37 op fork_matrix_source (tr1 tr2 : transcript) : byte list =
38 op fork_module_matrix (md : mode) (tr1 tr2 : transcript) : matrix =
39 op fork_module_target (md : mode) (tr1 tr2 : transcript) : polyveck =
40 op bimodal_selftarget_msis_instance_of_fork :
41 op extract_bimodal_selftarget_msis_solution :
42 op module_sis_instance_of_fork :
43 op extract_module_sis_solution :
44 op bimodal_extraction_success (md : mode) (tr1 tr2 : transcript) : bool =
45 op extraction_success (md : mode) (tr1 tr2 : transcript) : bool =
46 op forking_extraction_obligation (md : mode) : bool =
47 op fork_public_reconstruction_obligation (md : mode) : bool =
48 op bimodal_to_module_sis_lift_obligation (md : mode) : bool =
49 op special_soundness_obligation (md : mode) : bool =
50 op special_soundness_with_public_reconstruction_obligation (md : mode) : bool =
51 op transcript_from_honest_signing
```

Modules, module types, and clones

Lemmas

```

1 lemma extract_bimodal_forking_some md tr1 tr2 :
2 lemma extract_bimodal_forking_nonempty md tr1 tr2 :
3 lemma transcript_fields_match_signature_challenge_query md tr :
4 lemma transcript_fields_match_signature_challenge_matches tr y :
5 lemma transcript_valid_signature_challenge_query md tr :
6 lemma transcript_valid_signature_challenge_matches md tr y :
7 lemma transcript_valid_signature_challengeE md tr :
8 lemma transcript_valid_public_equation md tr :
9 lemma transcript_valid_commitment_reconstruction md tr :
10 lemma valid_forking_pair_public_equations md tr1 tr2 :
11 lemma same_fork_target_challenge_query md tr1 tr2 :
12 lemma valid_forking_pair_challenge_query md tr1 tr2 :
13 lemma valid_forking_pair_signature_challenge_query md tr1 tr2 :
14 lemma valid_forking_pair_distinct_signature_challenges md tr1 tr2 :
15 lemma same_fork_target_commitment_highbits tr1 tr2 :
16 lemma valid_forking_pair_commitment_highbits md tr1 tr2 :
17 lemma valid_forking_pair_reconstructed_highbits_equal md tr1 tr2 :
18 lemma transcript_reconstructionE md tr :
19 lemma transcript_active_reconstructionE md tr :
20 lemma valid_forking_pair_public_challenge_relation md tr1 tr2 :
21 lemma valid_forking_pair_public_reconstruction_cases md tr1 tr2 :
22 lemma forking_difference_solution_wf md tr1 tr2 :
23 lemma fork_module_matrix_rows_wf md tr1 tr2 :
24 lemma fork_module_matrix_size md tr1 tr2 :
25 lemma bimodal_forking_solution_valid md tr1 tr2 :
26 lemma forking_extraction_obligation_holds md :
27 lemma fork_public_reconstruction_obligation_holds md :
28 lemma bimodal_to_module_sis_lift_obligation_holds md :
29 lemma module_sis_extraction_from_bimodal md tr1 tr2 :
30 lemma special_soundness_from_bimodal_lift md :
31 lemma special_soundness_with_public_reconstruction_from_bimodal_lift md :
32 lemma honest_transcript_verifies md sk m ctx coins :
33 lemma transcript_of_signature_fields md pk m ctx sig :
34 lemma transcript_of_signature_matches_signature md pk m ctx sig :
35 lemma honest_transcript_challenge md sk m ctx coins :

```

Hoare, probabilistic Hoare, and equivalence proof commands

HAETAE_Events.ec

Types

Ops and abbreviations

```

1 op keygen_rejection_failure (md : mode) (pk : pkey) (sk : skey) : bool =
2 op signing_rejection_failure (md : mode) (sig : signature) : bool =
3 op verification_rejection_failure (md : mode) (sig : signature) : bool =
4 op any_rejection_failure (md : mode) (pk : pkey) (sk : skey)

```

Modules, module types, and clones

Lemmas

```
1 lemma keygen_rejection_free_current md sd :
2 lemma signing_rejection_free_current md sk m ctx coins :
3 lemma verification_rejection_free_current md sk m ctx coins :
4 lemma accepted_transcript_rejection_free_current md sd m ctx coins :
```

Hoare, probabilistic Hoare, and equivalence proof commands

HAETAE_Assumptions.ec

Types

```
1 type mlwe_instance.
2 type module_sis_instance = matrix * polyveck.
3 type module_sis_solution = polyvecl.
4 type bimodal_selftarget_msis_instance = module_sis_instance.
5 type bimodal_selftarget_msis_solution = module_sis_solution.
```

Ops and abbreviations

```
1 op [lossless] dmlwe_real : mode -> mlwe_instance distr.
2 op [lossless] dmlwe_random : mode -> mlwe_instance distr.
3 op [lossless] dmodule_sis_instance : mode -> module_sis_instance distr.
4 op [lossless] dbimodal_selftarget_msis_instance :
5 op module_sis_eval
6 op module_sis_zero_instance (md : mode) : module_sis_instance =
7 op module_sis_solution_wf (md : mode) (sol : module_sis_solution) : bool =
8 op module_sis_valid
9 op bimodal_selftarget_msis_valid
10 op bimodal_to_module_sis_instance :
11 op bimodal_to_module_sis_solution :
12 op lift_bimodal_solution_option
13 op dmodule_sis_from_bimodal (md : mode) : module_sis_instance distr =
```

Modules, module types, and clones

```
1 module type MLWE_Distinguisher = {
2 module MLWE_Real_Game(B : MLWE_Distinguisher) = {
3 module MLWE_Random_Game(B : MLWE_Distinguisher) = {
4 module type ModuleSIS_Solver = {
5 module ModuleSIS_Relation_Game(B : ModuleSIS_Solver) = {
```

```

6 module type BimodalSelfTargetMSIS_Solver = {
7 module BimodalSelfTargetMSIS_Relation_Game(
8 module BimodalToModuleSIS_Lift_Game(
9 module type ModuleSIS_ModeSolver = {
10 module type BimodalSelfTargetMSIS_ModeSolver = {
11 module ModuleSIS_ModeRelation_Game(
12 module BimodalSelfTargetMSIS_ModeRelation_Game(
13 module ModuleSIS_LiftedDistribution_Relation_Game(
14 module BimodalModeSolver_As_ModuleSIS(
15 module type MLWE_Reduction = {
16 module type BimodalSelfTargetMSIS_Reduction = {
17 module type ModuleSIS_Reduction = {
18 module MLWE_Game(B : MLWE_Reduction) = {
19 module BimodalSelfTargetMSIS_Game(B : BimodalSelfTargetMSIS_Reduction) = {
20 module ModuleSIS_Game(B : ModuleSIS_Reduction) = {
21 module BimodalMSIS_As_ModuleSIS(B : BimodalSelfTargetMSIS_Reduction) = {

```

Lemmas

```

1 lemma module_sis_eval_zero_instance md sol :
2 lemma module_sis_zero_valid md :
3 lemma module_sis_eval_wf md x sol :
4 lemma module_sis_valid_target_wf md x sol :
5 lemma bimodal_to_module_sis_valid md x sol :
6 lemma dmodule_sis_from_bimodalE md :
7 lemma dmodule_sis_from_bimodal_lossless md :
8 lemma bimodal_lift_successE md x sol :
9 lemma bimodal_solver_lift_exact &m md :
10 lemma bimodal_mode_solver_lifted_distribution_exact &m md :
11 lemma bimodal_msis_as_module_sis_exact &m :

```

Hoare, probabilistic Hoare, and equivalence proof commands

```

1 byequiv ( :={glob B, md} ==> ={res}) => //.
2 byequiv ( :={glob B, md} ==> ={res}) => //.
3 byequiv ( :={glob B} ==> ={res}) => //.

```

HAETAEDistributions.ec

Types

```

1 type signing_randomness_source = int.
2 type signing_sample_pair = polyvecl * polyveck.

```

Ops and abbreviations

```

1 op dbyte : byte distr = dinter 0 255.
2 op dseed : seed distr = dlist dbyte seedbytes.
3 op dcrh : crh distr = dlist dbyte crhbytes.
4 op signing_entropy_token_cardinality : int = 288230376151711744.
5 op signing_entropy_token_in_range (x : int) : bool =
6 op signing_randomness_byte_ok (b : byte) : bool = 0 <= b /\ b < 256.

```

```

7  op signing_entropy_token_to_coins (x : int) : random_coins =
8  op signing_randomness_domain (coins : random_coins) : bool =
9  op dsigning_entropy_token : int distr =
10 op dsigning_randomness_source : signing_randomness_source distr =
11 op signing_randomness_source_valid (src : signing_randomness_source) : bool =
12 op signing_randomness_from_source (src : signing_randomness_source) :
13 op dsigning_randomness : random_coins distr =
14 op drandom_coins : random_coins distr = dsigning_randomness.
15 op signing_randomness_point_bound : real =
16 op signing_randomness_token_spread (d : random_coins distr) : bool =
17 op signing_randomness_distribution_ok (d : random_coins distr) : bool =
18 op dcoeff : coeff distr = dinter 0 (q - 1).
19 op dpoly : poly distr = dlist dcoeff n.
20 op dpolyveck (md : mode) : polyveck distr = dlist dpoly (mode_k md).
21 op dpolyvecl (md : mode) : polyvecl distr = dlist dpoly (mode_l md).
22 op dmatrix (md : mode) : matrix distr = dlist (dpolyvecl md) (mode_k md).
23 op dchallenge (md : mode) : challenge distr =
24 op signing_sample_pair_y1 (s : signing_sample_pair) : polyvecl = s.'1.
25 op signing_sample_pair_y2 (s : signing_sample_pair) : polyveck = s.'2.
26 op signing_sample_pair_wf (md : mode) (s : signing_sample_pair) : bool =
27 op signing_sample_pair_unit (md : mode) (s : signing_sample_pair) : bool =
28 op signing_sample_bound (md : mode) : int = mode_ln md.
29 op coeff_bounded_by (b : int) (x : coeff) : bool =
30 op poly_bounded_by (b : int) (p : poly) : bool =
31 op polyvecl_bounded_by (md : mode) (b : int) (xs : polyvecl) : bool =
32 op polyveck_bounded_by (md : mode) (b : int) (xs : polyveck) : bool =
33 op signing_sample_pair_bounded (md : mode) (s : signing_sample_pair) : bool =
34 op signing_sample_pair_sample_ok (md : mode) (s : signing_sample_pair) : bool =
35 op signing_sample_pair_of_coins
36 op dstructural_signing_sample_pair
37 op dsigning_unit_coeff : coeff distr = duniform [-1; 1].
38 op dsigning_unit_poly : poly distr = dlist dsigning_unit_coeff n.
39 op dsigning_unit_polyvecl (md : mode) : polyvecl distr =
40 op dsigning_unit_polyveck (md : mode) : polyveck distr =
41 op signing_bounded_coeff_cardinality (md : mode) : int =
42 op signing_bounded_coeff_point_bound (md : mode) : real =
43 op dsigning_bounded_coeff (md : mode) : coeff distr =
44 op dsigning_bounded_poly (md : mode) : poly distr =
45 op dsigning_bounded_polyvecl (md : mode) : polyvecl distr =
46 op dsigning_bounded_polyveck (md : mode) : polyveck distr =
47 op dbounded_unit_signing_sample_pair (md : mode) :
48 op dchecked_hyperball_signing_sample_pair (md : mode) :
49 op hyperball_coeff_ok (md : mode) (c : coeff) : bool =
50 op hyperball_poly_ok (md : mode) (p : poly) : bool =
51 op hyperball_polyvecl_ok (md : mode) (y1 : polyvecl) : bool =
52 op hyperball_polyveck_ok (md : mode) (y2 : polyveck) : bool =
53 op hyperball_sample_ok (md : mode) (s : signing_sample_pair) : bool =
54 op dhyperball_coeff (md : mode) : coeff distr =
55 op dhyperball_poly (md : mode) : poly distr =
56 op dhyperball_polyvecl (md : mode) : polyvecl distr =
57 op dhyperball_polyveck (md : mode) : polyveck distr =
58 op hyperball_coeff_point_bound (md : mode) : real =
59 op hyperball_poly_point_bound (md : mode) : real =
60 op hyperball_polyvecl_point_bound (md : mode) : real =
61 op hyperball_polyveck_point_bound (md : mode) : real =
62 op hyperball_sample_pair_point_bound (md : mode) : real =
63 op dexact_hyperball_signing_sample_pair (md : mode) :
64 op signing_sample_pair_distribution_ok

```

Modules, module types, and clones

Lemmas

```

1  lemma byte_distribution_lossless : is_lossless dbyte.
2  lemma seed_distribution_lossless : is_lossless dseed.

```

```

3 lemma crh_distribution_lossless : is_lossless dcrh.
4 lemma signing_entropy_token_cardinality_positive :
5 lemma signing_entropy_token_distribution_lossless :
6 lemma signing_randomness_source_distribution_lossless :
7 lemma signing_coin_distribution_lossless : is_lossless drandom_coins.
8 lemma signing_entropy_token_to_coins_tokenK x :
9 lemma signing_randomness_from_source_tokenK src :
10 lemma signing_entropy_token_to_coins_size x :
11 lemma signing_entropy_token_to_coins_byte_domain x :
12 lemma signing_entropy_token_to_coins_domain x :
13 lemma signing_randomness_from_source_domain src :
14 lemma signing_entropy_token_distribution_point_bound x :
15 lemma signing_randomness_source_distribution_point_bound src :
16 lemma signing_coin_distribution_token_point_bound token :
17 lemma signing_coin_distribution_token_spread :
18 lemma signing_coin_distribution_domain coins :
19 lemma signing_coin_distribution_domain_full :
20 lemma signing_coin_distribution_ok :
21 lemma coeff_distribution_lossless : is_lossless dcoeff.
22 lemma poly_distribution_lossless : is_lossless dpoly.
23 lemma polyveck_distribution_lossless md : is_lossless (dpolyveck md).
24 lemma polyvecl_distribution_lossless md : is_lossless (dpolyvecl md).
25 lemma matrix_distribution_lossless md : is_lossless (dmatrix md).
26 lemma challenge_distribution_lossless md : is_lossless (dchallenge md).
27 lemma signing_sample_pair_of_coins_wf md sk m ctx coins :
28 lemma signing_sample_pair_of_coins_unit md sk m ctx coins :
29 lemma signing_sample_bound_gt0 md :
30 lemma unit_coeff_bounded_by_signing_sample_bound md c :
31 lemma poly_unit_bounded_by_signing_sample_bound md p :
32 lemma polyvecl_unit_bounded_by_signing_sample_bound md y1 :
33 lemma polyveck_unit_bounded_by_signing_sample_bound md y2 :
34 lemma signing_sample_pair_unit_bounded md s :
35 lemma signing_sample_pair_of_coins_sample_ok md sk m ctx coins :
36 lemma structural_signing_sample_pair_lossless md sk m ctx coins :
37 lemma structural_signing_sample_pair_support md sk m ctx coins s :
38 lemma structural_signing_sample_pair_sample_ok md sk m ctx coins s :
39 lemma structural_signing_sample_pair_distribution_ok md sk m ctx coins :
40 lemma signing_unit_coeff_lossless :
41 lemma signing_unit_coeff_support c :
42 lemma signing_bounded_coeff_cardinality_positive md :
43 lemma signing_bounded_coeff_lossless md :
44 lemma signing_bounded_coeff_support md c :
45 lemma signing_bounded_coeff_supportP md c :
46 lemma signing_bounded_coeff_point_bound md c :
47 lemma signing_bounded_coeff_point_bound_nonnegative md :
48 lemma dlist_size_point_bound ['a] (d : 'a distr) bd xs :
49 lemma dlist_point_bound ['a] (d : 'a distr) len bd xs :
50 lemma hyperball_coeff_point_bound_nonnegative md :
51 lemma hyperball_coeff_point_boundE md c :
52 lemma signing_unit_poly_lossless :
53 lemma signing_unit_poly_support p :
54 lemma signing_bounded_poly_lossless md :
55 lemma signing_bounded_poly_support md p :
56 lemma signing_bounded_poly_supportP md p :
57 lemma hyperball_poly_point_bound_nonnegative md :
58 lemma hyperball_poly_lossless md :
59 lemma hyperball_poly_support md p :
60 lemma hyperball_poly_supportP md p :
61 lemma hyperball_poly_point_boundE md p :
62 lemma signing_unit_polyvecl_lossless md :
63 lemma signing_unit_polyveck_lossless md :
64 lemma signing_bounded_polyvecl_lossless md :
65 lemma signing_bounded_polyveck_lossless md :
66 lemma signing_unit_polyvecl_support md y1 :
67 lemma signing_unit_polyveck_support md y2 :
68 lemma signing_bounded_polyvecl_support md y1 :
69 lemma signing_bounded_polyvecl_supportP md y1 :
70 lemma signing_bounded_polyveck_support md y2 :
71 lemma signing_bounded_polyveck_supportP md y2 :
72 lemma hyperball_polyvecl_point_bound_nonnegative md :
73 lemma hyperball_polyveck_point_bound_nonnegative md :
74 lemma hyperball_sample_pair_point_bound_nonnegative md :
75 lemma hyperball_polyvecl_lossless md :
76 lemma hyperball_polyveck_lossless md :
77 lemma hyperball_polyvecl_support md y1 :
78 lemma hyperball_polyvecl_supportP md y1 :
79 lemma hyperball_polyveck_support md y2 :

```

```

80 lemma hyperball_polyveck_supportP md y2 :
81 lemma hyperball_polyvecl_point_boundE md y1 :
82 lemma hyperball_polyveck_point_boundE md y2 :
83 lemma bounded_unit_signing_sample_pair_lossless md :
84 lemma bounded_unit_signing_sample_pair_support md s :
85 lemma bounded_unit_signing_sample_pair_sample_ok md s :
86 lemma bounded_unit_signing_sample_pair_distribution_ok md :
87 lemma checked_hyperball_signing_sample_pair_lossless md :
88 lemma checked_hyperball_signing_sample_pair_support md s :
89 lemma checked_hyperball_signing_sample_pair_distribution_ok md :
90 lemma hyperball_sample_ok_sample_ok md s :
91 lemma exact_hyperball_signing_sample_pair_lossless md :
92 lemma exact_hyperball_signing_sample_pair_support md s :
93 lemma exact_hyperball_signing_sample_pair_supportP md s :
94 lemma exact_hyperball_signing_sample_pair_sample_ok md s :
95 lemma exact_hyperball_signing_sample_pair_distribution_ok md :
96 lemma exact_hyperball_signing_sample_pair_point_bound md s :
97 lemma exact_hyperball_signing_sample_pair_checkedE md :

```

Hoare, probabilistic Hoare, and equivalence proof commands

HAETA_E_Rejection.ec

Types

```

1 type signing_transcript_record = message * context * signature * transcript.
2 type signing_attempt_sample = polyvecl * polyveck * polyveck.
3 type signing_attempt_state =
4 type signing_attempt_coin_sample_pair = random_coins * signing_sample_pair.

```

Ops and abbreviations

```

1 op signing_attempt_state_coins
2 op signing_attempt_state_sample
3 op signing_attempt_sample_y1 (s : signing_attempt_sample) : polyvecl =
4 op signing_attempt_sample_y2 (s : signing_attempt_sample) : polyveck =
5 op signing_attempt_sample_commitment_raw
6 op signing_attempt_state_sampled_y1
7 op signing_attempt_state_sampled_y2
8 op signing_attempt_state_sampled_y
9 op signing_attempt_state_highbits
10 op signing_attempt_state_lowbits
11 op signing_attempt_state_challenge
12 op signing_attempt_state_response
13 op signing_attempt_state_response_aux
14 op signing_attempt_state_hint
15 op signing_attempt_state_signature
16 op signature_rejection_branch_byte (sig : signature) : int =
17 op signing_attempt_state_reject2_branch_byte
18 op signing_attempt_state_reject2_branch_bit
19 op polyvecl_double (md : mode) (xs : polyvecl) : polyvecl =
20 op signing_attempt_state_reject2_sampled_y1
21 op signing_attempt_state_reject2_sampled_y2
22 op signing_attempt_state_reject2_balance_response
23 op signing_attempt_state_reject2_balance_aux
24 op signing_attempt_state_lowbits_carrier

```

APPENDIX E. EXTRACTED EASYCRYPT DECLARATION INDEX

```

25 op signing_attempt_state_token
26 op signing_attempt_state_programmed_lowbits
27 op signing_attempt_state_lowbits_consistent
28 op signing_attempt_state_programmed_challenge
29 op signing_attempt_state_challenge_consistent
30 op signing_attempt_state_reconstructed_highbits
31 op signing_attempt_state_public_equation_consistent
32 op signing_attempt_state_signature_of_fields
33 op signing_attempt_state_fields_match_signature
34 op signing_attempt_state_field_accepts
35 op signing_attempt_state_of_coins
36 op signing_attempt_sample_of_pair
37 op signing_attempt_state_of_sample_pair
38 op signing_attempt_state_valid_signature_accepts
39 op signing_attempt_state_response_norm_accepts
40 op signing_attempt_state_hint_norm_accepts
41 op signing_attempt_state_accepts
42 op signing_attempt_reject1_bound (md : mode) : int =
43 op signing_attempt_reject2_bound (md : mode) : int =
44 op signing_attempt_reject2_balance_norm_sq
45 op signing_attempt_reject2_concrete_aborts
46 op signing_attempt_state_reject1_norm_sq
47 op signing_attempt_state_reject1_accepts
48 op signing_attempt_state_balance_norm_sq
49 op signing_attempt_state_bimodal_reject2_branch
50 op signing_attempt_state_reject2_under_bound
51 op signing_attempt_state_reject2_aborts
52 op signing_attempt_state_reject2_accepts
53 op signing_attempt_state_pack_accepts
54 op signing_attempt_state_reference_rejection_accepts
55 op signing_attempt_state_reference_rejection_aborts
56 op signing_attempt_state_reject1_aborts
57 op signing_attempt_state_pack_aborts
58 op signing_attempt_state_verification_accepts
59 op signing_attempt_state_rejection_accepts
60 op signing_attempt_state_rejection_aborts
61 op signing_attempt_state_response_norm_aborts
62 op signing_attempt_state_hint_norm_aborts
63 op signing_attempt_state_rejection_sampling_aborts
64 op signing_attempt_state_aborts
65 op signing_accepts
66 op dsigning_attempt_state
67 op signing_sample_pair_sampler_ok
68 op dsigning_attempt_state_from_sample_pair_sampler
69 op signing_attempt_state_of_coin_sample_pair
70 op dstructural_signing_attempt_coin_sample_pair
71 op dexact_hyperball_signing_attempt_coin_sample_pair
72 op dstructural_signing_attempt_state
73 op dstructural_signing_attempt_state_from_sampler
74 op dbounded_unit_signing_attempt_state
75 op dchecked_hyperball_signing_attempt_state
76 op dexact_hyperball_signing_attempt_state
77 op signing_attempt_state_sample_pair_wf
78 op signing_attempt_state_sample_pair_unit
79 op signing_attempt_state_sample_pair_bounded
80 op signing_attempt_relation
81 op signing_simulation_relation
82 op signing_transcript_relation
83 op signing_record_message (r : signing_transcript_record) : message = r.'1.
84 op signing_record_context (r : signing_transcript_record) : context = r.'2.
85 op signing_record_signature (r : signing_transcript_record) : signature = r.'3.
86 op signing_record_transcript (r : signing_transcript_record) : transcript =
87 op signing_record_query (r : signing_transcript_record) : message * context =
88 op signing_record_relation
89 op signing_record_queries (rs : signing_transcript_record list) :
90 op signing_record_transcripts (rs : signing_transcript_record list) :
91 op signing_record_log_sound
92 op transcript_log_valid (md : mode) (trs : transcript list) : bool =
93 op structural_to_exact_hyperball_sample_pair_point_loss_obligation : bool =
94 op structural_to_exact_hyperball_sample_pair_loss_obligation : bool =
95 op structural_to_exact_hyperball_coin_sample_pair_loss_obligation : bool =
96 op structural_to_exact_hyperball_attempt_loss_obligation : bool =
97 op rejection_sampling_bound_obligation : bool =

```

Modules, module types, and clones

Lemmas

```
1 lemma signing_attempt_verifies md pk sk m ctx coins sig :
2 lemma signing_simulation_verifies md pk m ctx sig :
3 lemma signing_attempt_simulates md pk sk m ctx coins sig :
4 lemma sign_internal_attempt_relation md sk m ctx coins :
5 lemma signing_attempt_sample_of_pair_currentE md sk m ctx coins :
6 lemma signing_attempt_state_of_sample_pair_currentE md sk m ctx coins :
7 lemma structural_signing_sample_pair_sampler_ok md sk m ctx :
8 lemma bounded_unit_signing_sample_pair_sampler_ok md :
9 lemma checked_hyperball_signing_sample_pair_sampler_ok md :
10 lemma exact_hyperball_signing_sample_pair_sampler_ok md :
11 lemma dsigning_attempt_state_from_sample_pair_sampler_lossless
12 lemma dsigning_attempt_state_from_sample_pair_sampler_sample_ok
13 lemma dstructural_signing_attempt_state_lossless md sk m ctx :
14 lemma dstructural_signing_attempt_state_from_sampler_lossless
15 lemma dbounded_unit_signing_attempt_state_lossless md sk m ctx :
16 lemma dbounded_unit_signing_attempt_state_sample_ok md sk m ctx st :
17 lemma dchecked_hyperball_signing_attempt_state_lossless md sk m ctx :
18 lemma dchecked_hyperball_signing_attempt_state_sample_ok md sk m ctx st :
19 lemma dexact_hyperball_signing_attempt_state_lossless md sk m ctx :
20 lemma dexact_hyperball_signing_attempt_state_sample_ok md sk m ctx st :
21 lemma dexact_hyperball_signing_attempt_state_supportP md sk m ctx st :
22 lemma dexact_hyperball_signing_attempt_state_reference_rejection_abort_bound1
23 lemma dexact_hyperball_signing_attempt_state_mu_pullback
24 lemma signing_attempt_state_reference_rejection_aborts_componentE md st :
25 lemma dexact_hyperball_signing_attempt_state_reference_rejection_abort_split
26 lemma dexact_hyperball_signing_attempt_state_reference_rejection_abort_union_bound
27 lemma signing_attempt_state_of_sample_pair_signatureE
28 lemma signing_attempt_state_of_sample_pair_pack_accepts
29 lemma signing_attempt_state_of_sample_pair_pack_no_abort
30 lemma dexact_hyperball_signing_attempt_state_pack_abort_zero
31 lemma dstructural_signing_attempt_state_from_samplerE md sk m ctx :
32 lemma dstructural_signing_attempt_stateE md sk m ctx :
33 lemma dstructural_signing_attempt_state_from_sampler_coin_sample_pairE
34 lemma dexact_hyperball_signing_attempt_state_coin_sample_pairE
35 lemma mu_dlet_le_add_all ['a 'b 'c]
36 lemma structural_to_exact_hyperball_sample_pair_loss_from_point_loss :
37 lemma structural_to_exact_hyperball_coin_sample_pair_loss_from_sample_pair_loss :
38 lemma structural_to_exact_hyperball_attempt_loss_from_coin_sample_pair_loss :
39 lemma signing_attempt_state_of_coins_signatureE md sk m ctx coins :
40 lemma signing_attempt_state_of_coins_sampled_y1E md sk m ctx coins :
41 lemma signing_attempt_state_of_coins_sampled_y2E md sk m ctx coins :
42 lemma signing_attempt_state_of_coins_sampled_yE md sk m ctx coins :
43 lemma signing_attempt_state_of_coins_highbitsE md sk m ctx coins :
44 lemma signing_attempt_state_of_coins_lowbitsE md sk m ctx coins :
45 lemma signing_attempt_state_of_coins_challengeE md sk m ctx coins :
46 lemma signing_attempt_state_of_coins_responseE md sk m ctx coins :
47 lemma signing_attempt_state_of_coins_response_auxE md sk m ctx coins :
48 lemma signing_attempt_state_of_coins_hintE md sk m ctx coins :
49 lemma signing_attempt_state_of_coins_lowbits_carrierE md sk m ctx coins :
50 lemma signing_attempt_state_of_coins_tokenE md sk m ctx coins :
51 lemma signing_attempt_state_of_coins_programmed_lowbitsE md sk m ctx coins :
52 lemma signing_attempt_state_of_coins_lowbits_consistent md sk m ctx coins :
53 lemma signing_attempt_state_of_coins_programmed_challengeE md sk m ctx coins :
54 lemma signing_attempt_state_of_coins_challenge_consistent md sk m ctx coins :
55 lemma signing_attempt_state_of_coins_reconstructed_highbitsE
56 lemma signing_attempt_state_of_coins_public_equation_consistent
57 lemma signing_attempt_state_of_coins_signature_of_fieldsE md sk m ctx coins :
58 lemma signing_attempt_state_of_coins_fields_match_signature md sk m ctx coins :
59 lemma signing_attempt_state_of_coins_field_accepts md sk m ctx coins :
60 lemma signing_attempt_state_of_coins_accepts_current md sk m ctx coins :
61 lemma signing_attempt_reject1_bound_ge_response_bound md :
62 lemma signing_attempt_state_reject1_accepts_from_response_norm_ok md st :
63 lemma signing_attempt_state_balance_norm_sq_concreteE md st :
64 lemma signing_attempt_state_reject2_under_bound_concreteE md st :
65 lemma signing_attempt_state_reject2_aborts_concreteE md st :
66 lemma signing_attempt_state_reject2_accepts_from_branch_clear md st :
```

APPENDIX E. EXTRACTED EASYCRYPT DECLARATION INDEX

```

67 lemma signing_attempt_state_of_sample_pair_reject2_branch_clear
68 lemma signing_attempt_state_of_sample_pair_reject2_accepts
69 lemma signing_attempt_state_of_sample_pair_reject2_no_abort
70 lemma dexact_hyperball_signing_attempt_state_reject2_abort_zero
71 lemma dexact_hyperball_signing_attempt_state_reference_rejection_abort_reduced_bound
72 lemma signing_attempt_state_of_sample_pair_response_norm_accepts
73 lemma signing_attempt_state_of_sample_pair_reject1_accepts
74 lemma signing_attempt_state_of_sample_pair_reject1_no_abort
75 lemma dexact_hyperball_signing_attempt_state_reject1_abort_zero
76 lemma dexact_hyperball_signing_attempt_state_reference_rejection_abort_hint_bound
77 lemma signing_attempt_state_of_sample_pair_hint_norm_accepts
78 lemma signing_attempt_state_of_sample_pair_hint_norm_no_abort
79 lemma dexact_hyperball_signing_attempt_state_hint_norm_abort_zero
80 lemma dexact_hyperball_signing_attempt_state_reference_rejection_abort_zero
81 lemma signing_attempt_state_of_coins_reject2_branch_clear_current
82 lemma signing_attempt_state_of_coins_reject2_sampled_y1E md sk m ctx coins :
83 lemma signing_attempt_state_of_coins_reject2_sampled_y2E md sk m ctx coins :
84 lemma signing_attempt_state_of_coins_reject2_balance_norm_sqE
85 lemma signing_attempt_state_of_coins_balance_norm_sq_current
86 lemma signing_attempt_state_of_coins_reject2_accepts_current
87 lemma signing_attempt_state_pack_accepts_from_valid md st :
88 lemma signing_attempt_state_reference_rejection_accepts_from_accepts md st :
89 lemma signing_attempt_state_of_coins_reference_rejection_accepts_current
90 lemma signing_attempt_state_of_coins_verification_accepts_current
91 lemma signing_attempt_state_of_coins_rejection_accepts_current
92 lemma signing_attempt_state_of_coins_rejection_no_abort_current
93 lemma signing_attempt_state_of_coins_no_abort_current md sk m ctx coins :
94 lemma signing_attempt_state_distribution_lossless md sk m ctx :
95 lemma signing_attempt_state_distribution_token_spread
96 lemma signing_attempt_state_distribution_reference_rejection_abort_zero_current
97 lemma signing_attempt_state_distribution_rejection_abort_zero_current
98 lemma sign_internal_simulation_relation md sk m ctx coins :
99 lemma signing_attempt_transcript_valid md pk sk m ctx coins sig :
100 lemma signing_attempt_transcript_relation md pk sk m ctx coins sig :
101 lemma sign_internal_transcript_relation md sk m ctx coins :
102 lemma sign_internal_record_relation md sk m ctx coins :
103 lemma signing_record_relation_sound md pk r :
104 lemma signing_record_log_sound_cons md pk r rs :
105 lemma signing_record_log_sound_transcripts_valid md pk rs :
106 lemma signing_record_queries_cons r rs :
107 lemma signing_record_transcripts_cons r rs :
108 lemma rejection_bound_parts_nonnegative :
109 lemma rejection_sampling_bound_obligation_holds :
110 lemma signing_attempt_state_of_sample_pair_sampled_yE md sk m ctx coins sample :
111 lemma signing_attempt_state_of_sample_pair_highbitsE md sk m ctx coins sample :
112 lemma signing_attempt_state_of_sample_pair_lowbitsE md sk m ctx coins sample :
113 lemma signing_attempt_state_of_sample_pair_challengeE md sk m ctx coins sample :
114 lemma signing_attempt_state_of_sample_pair_responseE md sk m ctx coins sample :
115 lemma signing_attempt_state_of_sample_pair_response_auxE
116 lemma signing_attempt_state_of_sample_pair_hintE md sk m ctx coins sample :
117 lemma signing_attempt_state_of_sample_pair_lowbits_carrierE
118 lemma signing_attempt_state_of_sample_pair_tokenE md sk m ctx coins sample :
119 lemma signing_attempt_state_of_sample_pair_programmed_lowbitsE
120 lemma signing_attempt_state_of_sample_pair_lowbits_consistent
121 lemma signing_attempt_state_of_sample_pair_programmed_challengeE
122 lemma signing_attempt_state_of_sample_pair_challenge_consistent
123 lemma signing_attempt_state_of_sample_pair_reconstructed_highbitsE
124 lemma signing_attempt_state_of_sample_pair_public_equation_consistent
125 lemma signing_attempt_state_of_sample_pair_signature_of_fieldsE
126 lemma signing_attempt_state_of_sample_pair_fields_match_signature
127 lemma signing_attempt_state_of_sample_pair_field_accepts
128 lemma signing_attempt_state_of_sample_pair_valid_signature_accepts
129 lemma signing_attempt_state_of_sample_pair_accepts_current
130 lemma signing_attempt_state_of_sample_pair_verification_accepts_current
131 lemma signing_attempt_state_of_sample_pair_reference_rejection_accepts
132 lemma signing_attempt_state_of_sample_pair_reference_rejection_no_abort
133 lemma signing_attempt_state_of_sample_pair_rejection_accepts_current
134 lemma signing_attempt_state_of_sample_pair_rejection_no_abort_current
135 lemma dexact_hyperball_signing_attempt_state_reference_rejection_accepts
136 lemma dexact_hyperball_signing_attempt_state_reference_rejection_no_abort
137 lemma dexact_hyperball_signing_attempt_state_rejection_accepts
138 lemma dexact_hyperball_signing_attempt_state_rejection_no_abort
139 lemma dexact_hyperball_signing_attempt_state_rejection_abort_zero

```

Hoare, probabilistic Hoare, and equivalence proof commands

HAETA_E_Algebra.ec

Types

```
1 type byte = int.
2 type seed = byte list.
3 type crh = byte list.
4 type random_coins = byte list.
5 type xof256_state = byte list.
6 type message = byte list.
7 type context = byte list.
8 type coeff = int.
9 type poly = coeff list.
10 type challenge = poly.
11 type polyveck = poly list.
12 type polyvecl = poly list.
13 type polyvecm = poly list.
14 type matrix = poly list list.
15 type hint_t = polyveck.
16 type sig_token = polyvecl * polyveck * hint_t.
17 type pkey = seed * polyveck.
18 type encoded_pkey = byte list.
19 type skey = seed * int.
20 type signature = polyveck * poly * crh * challenge * sig_token * int * int.
```

Ops and abbreviations

```
1 op secret_key_of_seed (_ : mode) (sd : seed) : skey = (sd, 0).
2 op poly_zero : poly = nseq n 0.
3 op polyveck_zero (md : mode) : polyveck = nseq (mode_k md) poly_zero.
4 op polyvecl_zero (md : mode) : polyvecl = nseq (mode_l md) poly_zero.
5 op polyvecm_zero (md : mode) : polyvecm = nseq (mode_m md) poly_zero.
6 op matrix_zero (md : mode) : matrix =
7 op poly_wf (p : poly) : bool = size p = n.
8 op polyveck_wf (md : mode) (xs : polyveck) : bool =
9 op polyvecl_wf (md : mode) (xs : polyvecl) : bool =
10 op crh_wf (mu : crh) : bool = size mu = crhbytes.
11 op challenge_coeff_ok (x : coeff) : bool = x = -1 ∨ x = 0 ∨ x = 1.
12 op challenge_wf (ch : challenge) : bool =
13 op unit_coeff_ok (x : coeff) : bool = x = -1 ∨ x = 1.
14 op challenge_sparse_at (md : mode) (i : int) (x : coeff) : bool =
15 op challenge_sparse (md : mode) (ch : challenge) : bool =
16 op poly_unit (p : poly) : bool = poly_wf p /\ all unit_coeff_ok p.
17 op polyveck_unit (md : mode) (xs : polyveck) : bool =
18 op polyvecl_unit (md : mode) (xs : polyvecl) : bool =
19 op coeff_mod (x : coeff) : coeff = x %% q.
20 op coeff_add (x y : coeff) : coeff = coeff_mod (x + y).
21 op coeff_neg (x : coeff) : coeff = coeff_mod (~x).
22 op coeff_sub (x y : coeff) : coeff = coeff_add x (coeff_neg y).
23 op coeff_mul (x y : coeff) : coeff = coeff_mod (x * y).
24 op coeff_double (x : coeff) : coeff = coeff_add x x.
25 op poly_coeff (p : poly) (i : int) : coeff = nth 0 p i.
26 op poly_normalize (p : poly) : poly =
27 op poly_add (p r : poly) : poly =
28 op poly_neg (p : poly) : poly =
29 op poly_sub (p r : poly) : poly = poly_add p (poly_neg r).
30 op poly_double (p : poly) : poly =
31 op poly_mul_term (p r : poly) (i j : int) : coeff =
32 op poly_mul (p r : poly) : poly =
```

```

33 | op poly_dot (row : poly list) (v : poly list) : poly =
34 | op matrix_vec_mul (md : mode) (a : matrix) (v : polyvec1) : polyveck =
35 | op polyveck_add (md : mode) (xs ys : polyveck) : polyveck =
36 | op polyveck_neg (md : mode) (xs : polyveck) : polyveck =
37 | op polyveck_sub (md : mode) (xs ys : polyveck) : polyveck =
38 | op polyveck_double (md : mode) (xs : polyveck) : polyveck =
39 | op polyvec1_sub (md : mode) (xs ys : polyvec1) : polyvec1 =
40 | op coeff_q_bound (x : coeff) : bool = 0 <= x < q.
41 | op poly_coeffs_q_bound (p : poly) : bool =
42 | op polyveck_coeffs_q_bound (md : mode) (xs : polyveck) : bool =
43 | op z1_alpha : int = 256.
44 | op coeff_decompose_z1_low (x : coeff) : coeff =
45 | op coeff_decompose_z1_high (x : coeff) : coeff =
46 | op coeff_lsb (x : coeff) : coeff = x %% 2.
47 | op coeff_decompose_vk_low (x : coeff) : coeff =
48 | op coeff_decompose_vk_high (x : coeff) : coeff =
49 | op coeff_decompose_hint_high (md : mode) (x : coeff) : coeff =
50 | op coeff_compose_z1 (hi lo : coeff) : coeff = hi * z1_alpha + lo.
51 | op poly_highbits (p : poly) : poly =
52 | op poly_lowbits (p : poly) : poly =
53 | op poly_lsb (p : poly) : poly =
54 | op poly_vk_lowbits (p : poly) : poly =
55 | op poly_vk_highbits (p : poly) : poly =
56 | op poly_compose_z1 (hi lo : poly) : poly =
57 | op poly_hint_highbits (md : mode) (p : poly) : poly =
58 | op polyvec1_highbits (md : mode) (xs : polyvec1) : polyvec1 =
59 | op polyvec1_lowbits (md : mode) (xs : polyvec1) : polyvec1 =
60 | op polyveck_highbits_hint (md : mode) (xs : polyveck) : polyveck =
61 | op polyveck_vk_lowbits (md : mode) (xs : polyveck) : polyveck =
62 | op polyveck_vk_highbits (md : mode) (xs : polyveck) : polyveck =
63 | op polyveck_mul_alpha (md : mode) (xs : polyveck) : polyveck =
64 | op public_key_seed_poly (tag : int) (src : byte list) : poly =
65 | op public_key_seed_polyvec1
66 | op public_key_seed_polyveck
67 | op haetae_keygen_seedbuf_bytes : int = 2 * seedbytes + crhbytes.
68 | op haetae_reference_keygen_seedbuf_bytes : int =
69 | op haetae_reference_keygen_xof_absorb_bytes : int = seedbytes.
70 | op haetae_reference_keygen_xof_squeeze_bytes : int =
71 | op haetae_keygen_rhoprime_offset : int = 0.
72 | op haetae_keygen_sigma_offset : int = seedbytes.
73 | op haetae_keygen_key_offset : int = seedbytes + crhbytes.
74 | op haetae_shake256_rate_bytes : int = shake256_rate_bytes.
75 | op haetae_shake256_domain_separator : byte = shake256_domain_separator.
76 | op haetae_shake256_bytes (input : byte list) (outlen : int) : byte list =
77 | op haetae_seed_slice (src : byte list) (offset len : int) : byte list =
78 | op haetae_xof256_absorb_input
79 | op haetae_xof256_absorb_once
80 | op haetae_xof256_squeeze
81 | op haetae_xof256_absorb_once_squeeze
82 | op haetae_reference_keygen_seedbuf_after_memcpy (sd : seed) : byte list =
83 | op haetae_reference_keygen_xof_input (sd : seed) : byte list =
84 | op haetae_keygen_xof_seedbuf (sd : seed) : byte list =
85 | op haetae_reference_keygen_xof256_seedbuf (sd : seed) : byte list =
86 | op haetae_keygen_seedbuf_rhoprime (seedbuf : byte list) : seed =
87 | op haetae_keygen_seedbuf_sigma (seedbuf : byte list) : crh =
88 | op haetae_keygen_seedbuf_key (seedbuf : byte list) : seed =
89 | op haetae_keygen_seedbuf_layout_wf (seedbuf : byte list) : bool =
90 | op haetae_reference_keygen_xof_call_wf
91 | op haetae_keygen_rhoprime (sd : seed) : seed =
92 | op haetae_keygen_sigma (sd : seed) : crh =
93 | op haetae_keygen_key (sd : seed) : seed =
94 | op haetae_keygen_counter_next (md : mode) (counter : int) : int =
95 | op haetae_keygen_first_accept_counter (_ : mode) (_ : seed) : int = 0.
96 | op haetae_keygen_sampler_seed (sigma : crh) (counter : int) : seed =
97 | op public_matrix_seed (md : mode) (sd : seed) : matrix =
98 | op public_secret_vector_seed (md : mode) (sd : seed) : polyvec1 =
99 | op public_error_vector_seed (md : mode) (sd : seed) : polyveck =
100 | op public_rounding_vector_seed (md : mode) (sd : seed) : polyveck =
101 | op haetae_keygen_a0_matrix (md : mode) (sd : seed) : matrix =
102 | op haetae_keygen_secret_vector (md : mode) (sd : seed) : polyvec1 =
103 | op haetae_keygen_error_vector (md : mode) (sd : seed) : polyveck =
104 | op haetae_keygen_raw_public_vector (md : mode) (sd : seed) : polyveck =
105 | op haetae_keygen_mode23_precompose_vector
106 | op haetae_keygen_mode23_public_vector
107 | op haetae_keygen_mode23_rounding_lowbits
108 | op haetae_keygen_mode23_adjusted_error_vector
109 | op haetae_keygen_mode5_public_vector

```

```

110 op haetae_public_key_vector_from_relation
111 op haetae_keygen_public_vector (md : mode) (sd : seed) : polyveck =
112 op public_key_vector_seed (md : mode) (sd : seed) : polyveck =
113 op public_key_relation
114 op public_key_equation_holds (md : mode) (pk : pkey) : bool =
115 op haetae_reference_polymatkm_expand_matA
116 op haetae_reference_polyveck_expand_vecA
117 op haetae_reference_polyvecmk_expand_S_s1
118 op haetae_reference_polyvecmk_expand_S_s2
119 op haetae_reference_keygen_secret_vector
120 op haetae_reference_keygen_error_vector
121 op haetae_reference_keygen_a0_matrix
122 op haetae_reference_keygen_raw_public_vector
123 op haetae_reference_keygen_mode23_predecompose_vector
124 op haetae_reference_keygen_mode23_public_vector
125 op haetae_reference_keygen_mode23_rounding_lowbits
126 op haetae_reference_keygen_mode23_adjusted_error_vector
127 op haetae_reference_keygen_mode5_public_vector
128 op haetae_reference_keygen_public_vector
129 op packed_haetae_reference_public_key
130 op haetae_reference_keygen_seed_flow_wf (md : mode) (sd : seed) : bool =
131 op haetae_mode23_qj_vector (md : mode) (sd : seed) : polyveck =
132 op haetae_mode23_verification_column
133 op haetae_mode5_verification_column
134 op public_key_mode23_column (md : mode) (pk : pkey) : polyveck =
135 op public_key_mode5_column (md : mode) (pk : pkey) : polyveck =
136 op public_key_verification_column (md : mode) (pk : pkey) : polyveck =
137 op haetae_verification_column_from_unpacked
138 op haetae_verification_matrix_from_unpacked
139 op packed_haetae_public_key (md : mode) (sd : seed) : pkey =
140 op packed_haetae_verification_matrix (md : mode) (pk : pkey) : matrix =
141 op packed_haetae_keygen_verification_matrix (md : mode) (sd : seed) :
142 op public_verification_matrix (md : mode) (pk : pkey) : matrix =
143 op challenge_embedding (md : mode) (ch : challenge) : polyvec1 =
144 op public_key_challenge_term
145 op reconstructed_highbits
146 op public_key_of_secret (md : mode) (sk : skey) : pkey =
147 op haetae_public_key_body_bytes (md : mode) : int =
148 op haetae_public_key_bytes (md : mode) : int =
149 op haetae_public_key_encoding_wf (md : mode) (enc : encoded_pkey) : bool =
150 op haetae_public_key_unpacked_wf (md : mode) (pk : pkey) : bool =
151 op haetae_pack_seed (sd : seed) : byte list =
152 op haetae_unpack_seed (enc : encoded_pkey) : seed =
153 op haetae_pack_poly_q (md : mode) (p : poly) : byte list =
154 op haetae_unpack_poly_q_at
155 op haetae_byte_modulus : int = 256.
156 op haetae_polyq_mode23_bound : int = 32768.
157 op haetae_polyq_mode5_bound : int = 65536.
158 op haetae_byte_norm (x : int) : byte = x %% haetae_byte_modulus.
159 op haetae_polyq_coeff_bound (md : mode) : int =
160 op haetae_polyq_coeffs_wf (md : mode) (p : poly) : bool =
161 op haetae_polyveck_polyq_coeffs_wf
162 op haetae_pack_poly_q_mode23_byte
163 op haetae_pack_poly_q_mode5_byte (p : poly) (j : int) : byte =
164 op haetae_pack_poly_q_ref (md : mode) (p : poly) : byte list =
165 op haetae_unpack_poly_q_mode23_coeff_at
166 op haetae_unpack_poly_q_mode5_coeff_at
167 op haetae_unpack_poly_q_ref_at
168 op haetae_pack_polyveck_body_ref (md : mode) (b : polyveck) : byte list =
169 op haetae_unpack_polyveck_body_ref
170 op haetae_pack_vk_ref (md : mode) (b : polyveck) (sd : seed) : encoded_pkey =
171 op haetae_unpack_vk_public_key_ref
172 op haetae_pack_polyveck_body (md : mode) (b : polyveck) : byte list =
173 op haetae_unpack_polyveck_body
174 op haetae_pack_vk (md : mode) (b : polyveck) (sd : seed) : encoded_pkey =
175 op haetae_unpack_vk_public_key
176 op haetae_unpack_vk_matrix (md : mode) (enc : encoded_pkey) : matrix =
177 op haetae_pack_public_key_bytes (md : mode) (pk : pkey) : encoded_pkey =
178 op haetae_unpack_public_key_bytes
179 op concrete_haetae_keygen_packed_public_key
180 op concrete_haetae_keygen_unpacked_public_key
181 op concrete_haetae_keygen_verification_matrix (md : mode) (sd : seed) :
182 op haetae_pack_public_key_bytes_ref (md : mode) (pk : pkey) :
183 op haetae_unpack_public_key_bytes_ref
184 op concrete_haetae_keygen_packed_public_key_ref
185 op concrete_haetae_keygen_unpacked_public_key_ref
186 op concrete_haetae_keygen_verification_matrix_ref

```

```

187 op concrete_haetae_reference_keygen_packed_public_key_ref
188 op concrete_haetae_reference_keygen_unpacked_public_key_ref
189 op concrete_haetae_reference_keygen_verification_matrix_ref
190 op haetae_public_key_roundtrip_ok (md : mode) (pk : pkey) : bool =
191 op haetae_keygen_public_key_roundtrip_ok (md : mode) (sd : seed) : bool =
192 op valid_mode : mode -> bool = fun _ => true.
193 op valid_keypair (md : mode) (pk : pkey) (sk : skey) : bool =
194 op valid_signature (md : mode) (sig : signature) : bool =
195 op sig_commitment_highbits (sig : signature) : polyveck = sig.'1.
196 op sig_commitment_lowbits (sig : signature) : poly = sig.'2.
197 op sig_message_hash (sig : signature) : crh = sig.'3.
198 op sig_challenge (sig : signature) : challenge = sig.'4.
199 op sig_token_value (sig : signature) : sig_token = sig.'5.
200 op sig_response (sig : signature) : polyvecl = (sig_token_value sig).'1.
201 op sig_response_aux (sig : signature) : polyveck = (sig_token_value sig).'2.
202 op sig_hint (sig : signature) : hint_t = (sig_token_value sig).'3.
203 op encode_poly (p : poly) : byte list = p.
204 op encode_polyveck (xs : polyveck) : byte list = flatten xs.
205 op encode_pkey (pk : pkey) : byte list = pk.'1 ++ encode_polyveck pk.'2.
206 op deterministic_poly (tag : int) (src : byte list) : poly =
207 op deterministic_polyveck (md : mode) (src : byte list) : polyveck =
208 op deterministic_unit_coeff (tag : int) (src : byte list) (i : int) :
209 op deterministic_unit_poly (tag : int) (src : byte list) : poly =
210 op deterministic_unit_polyveck
211 op deterministic_unit_polyvecl
212 op commitment_source
213 op commitment_raw :
214 op response_source
215 op response_aux_vector :
216 op signing_sample_y1 :
217 op signing_sample_y2 :
218 op signing_entropy_token_of_coins (coins : random_coins) : int =
219 op commitment_lowbits :
220 op message_hash : pkey -> context -> message -> crh =
221 op challenge_source (highbits : polyveck) (lowbits : poly) (mu : crh) :
222 op challenge_from_seed (md : mode) (src : byte list) : challenge =
223 op challenge_hash : mode -> polyveck -> poly -> crh -> challenge =
224 op commitment_challenge :
225 op commitment_highbits :
226 op response_vector :
227 op hint_vector :
228 op signature_token :
229 op keygen_internal (md : mode) (sd : seed) : pkey * skey =
230 op sign_internal :
231 op verify_challenge_consistent
232 op verify_reconstructed_highbits
233 op verify_public_equation
234 op coeff_norm_sq (x : coeff) : int = x * x.
235 op poly_norm_sq (p : poly) : int =
236 op polyvecl_norm_sq (md : mode) (xs : polyvecl) : int =
237 op polyveck_norm_sq (md : mode) (xs : polyveck) : int =
238 op signature_response_norm_sq (md : mode) (sig : signature) : int =
239 op signature_verify_norm_sq (md : mode) (sig : signature) : int =
240 op secret_norm_sq (sk : skey) : int = sk.'2.
241 op response_norm_sq (md : mode) (sig : signature) : int =
242 op verify_norm_sq (md : mode) (sig : signature) : int =
243 op secret_norm_ok (md : mode) (sk : skey) : bool =
244 op response_norm_ok (md : mode) (sig : signature) : bool =
245 op verify_norm_ok (md : mode) (sig : signature) : bool =
246 op verify_internal :

```

Modules, module types, and clones

Lemmas

```

1 lemma poly_zero_size :
2 lemma polyveck_zero_size md :
3 lemma polyvecl_zero_size md :
4 lemma polyvecm_zero_size md :
5 lemma polyveck_wf_nth md b row :
6 lemma poly_zero_wf : poly_wf poly_zero.
7 lemma polyveck_zero_wf md : polyveck_wf md (polyveck_zero md).
8 lemma polyvecl_zero_wf md : polyvecl_wf md (polyvecl_zero md).
9 lemma polyvecl_sub_size md xs ys :
10 lemma poly_add_wf p r : poly_wf (poly_add p r).
11 lemma poly_normalize_wf p : poly_wf (poly_normalize p).
12 lemma poly_neg_wf p : poly_wf (poly_neg p).
13 lemma poly_sub_wf p r : poly_wf (poly_sub p r).
14 lemma poly_double_wf p : poly_wf (poly_double p).
15 lemma poly_mul_wf p r : poly_wf (poly_mul p r).
16 lemma poly_dot_wf row v : poly_wf (poly_dot row v).
17 lemma matrix_vec_mul_wf md a v : polyveck_wf md (matrix_vec_mul md a v).
18 lemma polyveck_add_wf md xs ys : polyveck_wf md (polyveck_add md xs ys).
19 lemma polyveck_neg_wf md xs : polyveck_wf md (polyveck_neg md xs).
20 lemma polyveck_sub_wf md xs ys : polyveck_wf md (polyveck_sub md xs ys).
21 lemma polyveck_double_wf md xs : polyveck_wf md (polyveck_double md xs).
22 lemma polyvecl_sub_wf md xs ys : polyvecl_wf md (polyvecl_sub md xs ys).
23 lemma coeff_mod_q_bound x : coeff_q_bound (coeff_mod x).
24 lemma poly_zero_coeffs_q_bound : poly_coeffs_q_bound poly_zero.
25 lemma poly_add_coeffs_q_bound p r :
26 lemma poly_neg_coeffs_q_bound p :
27 lemma poly_sub_coeffs_q_bound p r :
28 lemma poly_double_coeffs_q_bound p :
29 lemma poly_mul_coeffs_q_bound p r :
30 lemma poly_dot_coeffs_q_bound row v :
31 lemma matrix_vec_mul_coeffs_q_bound md a v :
32 lemma polyveck_zero_coeffs_q_bound md :
33 lemma polyveck_add_coeffs_q_bound md xs ys :
34 lemma polyveck_neg_coeffs_q_bound md xs :
35 lemma polyveck_sub_coeffs_q_bound md xs ys :
36 lemma polyveck_double_coeffs_q_bound md xs :
37 lemma poly_highbits_wf p : poly_wf (poly_highbits p).
38 lemma poly_lowbits_wf p : poly_wf (poly_lowbits p).
39 lemma poly_lsb_wf p : poly_wf (poly_lsb p).
40 lemma poly_vk_lowbits_wf p : poly_wf (poly_vk_lowbits p).
41 lemma poly_vk_highbits_wf p : poly_wf (poly_vk_highbits p).
42 lemma poly_compose_zi_wf hi lo : poly_wf (poly_compose_zi hi lo).
43 lemma poly_hint_highbits_wf md p : poly_wf (poly_hint_highbits md p).
44 lemma polyvecl_highbits_wf md xs :
45 lemma polyvecl_lowbits_wf md xs :
46 lemma polyveck_highbits_hint_wf md xs :
47 lemma polyveck_vk_lowbits_wf md xs :
48 lemma polyveck_vk_highbits_wf md xs :
49 lemma polyveck_mul_alpha_wf md xs :
50 lemma coeff_decompose_hint_high_polyq_bound md x :
51 lemma poly_hint_highbits_polyq_wf md p :
52 lemma polyveck_highbits_hint_polyq_wf md xs :
53 lemma coeff_decompose_vk_high_mode23_polyq_bound md x :
54 lemma poly_vk_highbits_mode23_polyq_wf md p :
55 lemma polyveck_vk_highbits_mode23_polyq_wf md xs :
56 lemma coeff_q_bound_mode5_polyq_bound x :
57 lemma poly_q_bound_mode5_polyq_wf p :
58 lemma polyveck_q_bound_mode5_polyq_wf xs :
59 lemma public_key_seed_poly_wf tag src :
60 lemma public_key_seed_polyvecl_wf md tag src :
61 lemma public_key_seed_polyveck_wf md tag src :
62 lemma haetae_keygen_seedbuf_bytes_gt0 :
63 lemma haetae_reference_keygen_seedbuf_bytesE :
64 lemma haetae_reference_keygen_xof_absorb_bytesE :
65 lemma haetae_reference_keygen_xof_squeeze_bytesE :
66 lemma haetae_keygen_rho_prime_offsetE :
67 lemma haetae_keygen_sigma_offsetE :
68 lemma haetae_keygen_key_offsetE :
69 lemma haetae_shake256_rate_bytesE :
70 lemma haetae_shake256_domain_separatorE :
71 lemma haetae_shake256_bytes_size input outlen :
72 lemma haetae_xof256_squeeze_size st outlen :
73 lemma haetae_xof256_absorb_once_squeeze_size input inlen outlen :
74 lemma haetae_reference_keygen_seedbuf_after_memcpy_size sd :
75 lemma haetae_keygen_xof_seedbuf_size sd :
76 lemma haetae_reference_keygen_xof256_seedbuf_size sd :

```

APPENDIX E. EXTRACTED EASYCRYPT DECLARATION INDEX

```

77 lemma haetae_seed_slice_size src offset len :
78 lemma haetae_reference_keygen_xof_input_size sd :
79 lemma haetae_xof256_absorb_input_size input inlen :
80 lemma haetae_reference_keygen_xof_absorb_inputE sd :
81 lemma haetae_reference_keygen_shake256_domain sd :
82 lemma haetae_reference_keygen_xof_squeeze_bytes_lt_rate :
83 lemma haetae_reference_keygen_xof_absorb_once_stateE sd :
84 lemma haetae_keygen_xof_seedbuf_incrementalE sd :
85 lemma haetae_keygen_xof_seedbuf_shake256E sd :
86 lemma haetae_keygen_seedbuf_rhoprime_size seedbuf :
87 lemma haetae_keygen_seedbuf_sigma_size seedbuf :
88 lemma haetae_keygen_seedbuf_key_size seedbuf :
89 lemma haetae_reference_keygen_xof_seedbuf_layout_wf sd :
90 lemma haetae_reference_keygen_xof_call_wf_ok sd :
91 lemma haetae_keygen_rhoprime_ref_sliceE sd :
92 lemma haetae_keygen_sigma_ref_sliceE sd :
93 lemma haetae_keygen_key_ref_sliceE sd :
94 lemma haetae_keygen_rhoprime_size sd :
95 lemma haetae_keygen_sigma_size sd :
96 lemma haetae_keygen_key_size sd :
97 lemma haetae_keygen_sampler_seed_size sigma counter :
98 lemma haetae_keygen_counter_next_gt counter md :
99 lemma public_matrix_seed_size md sd :
100 lemma public_matrix_seed_rows_wf md sd :
101 lemma public_secret_vector_seed_wf md sd :
102 lemma public_error_vector_seed_wf md sd :
103 lemma public_rounding_vector_seed_wf md sd :
104 lemma haetae_reference_polymatkm_expand_matA_size md rhoprime :
105 lemma haetae_reference_polymatkm_expand_matA_rows_wf md rhoprime :
106 lemma haetae_reference_polyveck_expand_vecA_wf md rhoprime :
107 lemma haetae_reference_polyvecmk_expand_S_s1_wf md sigma counter :
108 lemma haetae_reference_polyvecmk_expand_S_s2_wf md sigma counter :
109 lemma haetae_reference_keygen_secret_vector_wf md sd :
110 lemma haetae_reference_keygen_error_vector_wf md sd :
111 lemma haetae_reference_keygen_raw_public_vector_wf md sd :
112 lemma haetae_reference_keygen_raw_public_vector_coeffs_q_bound md sd :
113 lemma haetae_reference_keygen_mode23_predecompose_vector_wf md sd :
114 lemma haetae_reference_keygen_mode23_predecompose_vector_coeffs_q_bound :
115 lemma haetae_reference_keygen_mode23_public_vector_polyq_wf md sd :
116 lemma haetae_reference_keygen_mode23_adjusted_error_vector_wf md sd :
117 lemma haetae_reference_keygen_mode5_public_vector_wf sd :
118 lemma haetae_reference_keygen_mode5_public_vector_coeffs_q_bound sd :
119 lemma haetae_reference_keygen_mode5_public_vector_polyq_wf sd :
120 lemma haetae_keygen_raw_public_vector_wf md sd :
121 lemma haetae_keygen_raw_public_vector_coeffs_q_bound md sd :
122 lemma haetae_keygen_mode23_predecompose_vector_wf md sd :
123 lemma haetae_keygen_mode23_predecompose_vector_coeffs_q_bound md sd :
124 lemma haetae_keygen_mode23_public_vector_polyq_wf md sd :
125 lemma haetae_keygen_mode23_rounding_lowbits_wf md sd :
126 lemma haetae_keygen_mode23_adjusted_error_vector_wf md sd :
127 lemma haetae_keygen_mode5_public_vector_wf sd :
128 lemma haetae_keygen_mode5_public_vector_coeffs_q_bound sd :
129 lemma haetae_keygen_mode5_public_vector_polyq_wf sd :
130 lemma haetae_public_key_vector_from_relation_polyq_wf md sd s e :
131 lemma haetae_reference_keygen_public_vector_polyq_wf md sd :
132 lemma haetae_keygen_public_vector_polyq_wf md sd :
133 lemma public_key_vector_seed_wf md sd :
134 lemma public_key_mode23_column_wf md pk :
135 lemma haetae_verification_column_from_unpacked_wf md sd b :
136 lemma public_key_verification_column_wf md pk :
137 lemma public_verification_matrix_size md pk :
138 lemma public_verification_matrix_rows_wf md pk :
139 lemma challenge_embedding_wf md ch :
140 lemma public_key_of_secret_wf md sk :
141 lemma haetae_pack_seed_size sd :
142 lemma haetae_unpack_seed_size enc :
143 lemma haetae_unpack_seed_pack sd :
144 lemma haetae_unpack_seed_pack_cat sd tail :
145 lemma haetae_polyq_coeff_bound_gt0 md :
146 lemma haetae_byte_norm_bounds x :
147 lemma haetae_byte_norm_small x :
148 lemma haetae_byte_norm_idempotent x :
149 lemma haetae_unpack15_coeff0 c0 c1 :
150 lemma haetae_unpack15_coeff1 c0 c1 c2 :
151 lemma haetae_unpack15_coeff2 c1 c2 c3 :
152 lemma haetae_unpack15_coeff3 c2 c3 c4 :
153 lemma haetae_unpack15_coeff4 c3 c4 c5 :

```

```

154 lemma haetae_unpack15_coeff5 c4 c5 c6 :
155 lemma haetae_unpack15_coeff6 c5 c6 c7 :
156 lemma haetae_unpack15_coeff7 c6 c7 :
157 lemma haetae_pack_poly_q_ref_size md p :
158 lemma haetae_unpack_poly_q_ref_at_wf md enc offset :
159 lemma haetae_pack_polyveck_body_ref_size md b :
160 lemma haetae_unpack_polyveck_body_ref_wf md enc offset :
161 lemma haetae_pack_vk_ref_size md b sd :
162 lemma haetae_unpack_vk_public_key_ref_wf md enc :
163 lemma nth_cat_size_plus ['a] (d : 'a) (xs ys : 'a list) j :
164 lemma haetae_pack_polyveck_body_ref_nth md b row k :
165 lemma haetae_pack_polyveck_body_ref_mode5_nth b row k :
166 lemma haetae_unpack_poly_q_mode23_coeff_at_cat prefix bytes blk ofs :
167 lemma haetae_polyq_mode23_coeff_rng md p j :
168 lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs0 md p blk :
169 lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs1 md p blk :
170 lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs2 md p blk :
171 lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs3 md p blk :
172 lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs4 md p blk :
173 lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs5 md p blk :
174 lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs6 md p blk :
175 lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs7 md p blk :
176 lemma haetae_unpack_poly_q_ref_pack_mode23_coeff md p i :
177 lemma haetae_unpack_poly_q_ref_pack_mode23_at md prefix p :
178 lemma haetae_unpack_poly_q_mode23_coeff_at_body md prefix b row blk ofs :
179 lemma haetae_unpack_polyveck_body_ref_pack_mode23_coeff md prefix b row i :
180 lemma haetae_unpack_polyveck_body_ref_pack_mode23 md prefix b :
181 lemma haetae_pack_vk_ref_roundtrip_mode23 md sd b :
182 lemma haetae_unpack_poly_q_ref_pack_mode5_coeff p i :
183 lemma haetae_unpack_poly_q_ref_pack_mode5_at prefix p :
184 lemma haetae_unpack_polyveck_body_ref_pack_mode5_coeff prefix b row i :
185 lemma haetae_unpack_polyveck_body_ref_pack_mode5 prefix b :
186 lemma haetae_pack_vk_ref_roundtrip_mode5 sd b :
187 lemma haetae_public_key_ref_roundtrip md (pk : pkey) :
188 lemma haetae_keygen_public_key_ref_roundtrip md sd :
189 lemma haetae_pack_polyveck_body_size md b :
190 lemma haetae_pack_poly_q_size md p :
191 lemma haetae_unpack_poly_q_at_wf md enc offset :
192 lemma haetae_unpack_poly_q_pack_at md prefix p :
193 lemma haetae_unpack_polyveck_body_wf md enc offset :
194 lemma haetae_body_index_div md row col :
195 lemma haetae_body_index_mod md row col :
196 lemma haetae_body_index_lt md row col :
197 lemma haetae_unpack_polyveck_body_pack_at md prefix b :
198 lemma haetae_unpack_polyveck_body_pack md sd b :
199 lemma haetae_public_key_roundtrip md pk :
200 lemma haetae_pack_public_key_bytes_size md pk :
201 lemma haetae_pack_public_key_bytes_wf md pk :
202 lemma haetae_pack_public_key_bytes_ref_size md pk :
203 lemma haetae_pack_public_key_bytes_ref_wf md pk :
204 lemma haetae_unpack_public_key_bytes_wf md enc :
205 lemma haetae_unpack_public_key_bytes_ref_wf md enc :
206 lemma concrete_haetae_keygen_packed_public_key_wf md sd :
207 lemma concrete_haetae_keygen_unpacked_public_key_wf md sd :
208 lemma concrete_haetae_keygen_packed_public_key_ref_wf md sd :
209 lemma concrete_haetae_keygen_unpacked_public_key_ref_wf md sd :
210 lemma packed_haetae_reference_public_key_wf md sd :
211 lemma packed_haetae_reference_public_key_polyq_wf md sd :
212 lemma packed_haetae_reference_public_key_unpacked_wf md sd :
213 lemma concrete_haetae_reference_keygen_packed_public_key_ref_wf md sd :
214 lemma concrete_haetae_reference_keygen_unpacked_public_key_ref_wf md sd :
215 lemma haetae_reference_keygen_public_key_ref_roundtrip md sd :
216 lemma public_key_of_secret_relation md sk :
217 lemma public_key_of_secret_equation_holds md sk :
218 lemma deterministic_poly_wf tag src :
219 lemma deterministic_polyveck_wf md src :
220 lemma deterministic_unit_poly_wf tag src :
221 lemma deterministic_unit_poly_unit tag src :
222 lemma deterministic_unit_polyveck_wf md tag src :
223 lemma deterministic_unit_polyveck_unit md tag src :
224 lemma deterministic_unit_polyvecl_wf md tag src :
225 lemma deterministic_unit_polyvecl_unit md tag src :
226 lemma commitment_raw_wf md sk m ctx coins :
227 lemma commitment_highbits_wf md sk m ctx coins :
228 lemma commitment_lowbits_wf md sk m ctx coins :
229 lemma commitment_lowbits_entropy_token md sk m ctx coins :
230 lemma challenge_from_seed_wf md src :

```

```

231 lemma challenge_from_seed_sparse md src :
232 lemma challenge_hash_wf md w1 w0 mu :
233 lemma challenge_hash_sparse md w1 w0 mu :
234 lemma response_vector_wf md sk m ctx coins :
235 lemma response_vector_unit md sk m ctx coins :
236 lemma response_aux_vector_wf md sk m ctx coins :
237 lemma response_aux_vector_unit md sk m ctx coins :
238 lemma signing_sample_y1_wf md sk m ctx coins :
239 lemma signing_sample_y1_unit md sk m ctx coins :
240 lemma signing_sample_y2_wf md sk m ctx coins :
241 lemma signing_sample_y2_unit md sk m ctx coins :
242 lemma hint_vector_wf md sk m ctx coins :
243 lemma hint_vector_unit md sk m ctx coins :
244 lemma message_hash_size pk ctx m :
245 lemma challenge_hash_size md w1 w0 mu :
246 lemma keygen_internal_valid md sd :
247 lemma public_key_of_keygen md sd :
248 lemma public_key_vector_seed_keygen_equation md sd :
249 lemma public_key_vector_seed_mode23_keygen_equation md sd :
250 lemma public_key_vector_seed_mode5_keygen_equation sd :
251 lemma haetae_keygen_raw_public_vector_keygen_equation md sd :
252 lemma haetae_reference_keygen_raw_public_vector_equation md sd :
253 lemma haetae_reference_keygen_mode23_public_vector_equation md sd :
254 lemma haetae_reference_keygen_mode23_adjusted_error_equation md sd :
255 lemma haetae_reference_keygen_mode5_public_vector_equation sd :
256 lemma packed_haetae_reference_public_key_seedE md sd :
257 lemma packed_haetae_reference_public_key_bodyE md sd :
258 lemma haetae_reference_keygen_seed_flow_wf_ok md sd :
259 lemma packed_haetae_public_key_wf md sd :
260 lemma packed_haetae_public_key_polyq_wf md sd :
261 lemma packed_haetae_public_key_unpacked_wf md sd :
262 lemma haetae_keygen_public_key_roundtrip md sd :
263 lemma public_key_of_secret_packed md sk :
264 lemma keygen_public_key_packed md sd :
265 lemma public_verification_matrix_packedE md pk :
266 lemma packed_haetae_keygen_verification_matrixE md sd :
267 lemma honest_public_verification_matrix_eq_packed_keygen md sd :
268 lemma concrete_keygen_public_verification_matrix_correct md sd :
269 lemma concrete_keygen_public_verification_matrix_ref_correct md sd :
270 lemma concrete_reference_keygen_public_verification_matrix_ref_correct md sd :
271 lemma concrete_keygen_public_verification_matrix_size md sd :
272 lemma concrete_keygen_public_verification_matrix_ref_size md sd :
273 lemma concrete_reference_keygen_public_verification_matrix_ref_size md sd :
274 lemma concrete_keygen_public_verification_matrix_rows_wf md sd :
275 lemma concrete_keygen_public_verification_matrix_ref_rows_wf md sd :
276 lemma concrete_reference_keygen_public_verification_matrix_ref_rows_wf md sd :
277 lemma valid_signature_sign_internal md sk m ctx coins :
278 lemma verify_challenge_consistent_sign_internal md sk m ctx coins :
279 lemma secret_norm_ok_current md sd :
280 lemma response_norm_ok_current md sk m ctx coins :
281 lemma verify_norm_ok_current md sk m ctx coins :
282 lemma verify_internal_sign_internal md sk m ctx coins :

```

Hoare, probabilistic Hoare, and equivalence proof commands

HAETA_E_Params.ec

Types

```

1 type mode = [ Mode2 | Mode3 | Mode5 ].

```

Ops and abbreviations

```
1 op seedbytes : int = 32.
2 op crhbytes : int = 64.
3 op n : int = 256.
4 op q : int = 64513.
5 op dq : int = 2 * q.
6 op mode_k (md : mode) : int =
7 op mode_l (md : mode) : int =
8 op mode_tau (md : mode) : int =
9 op mode_b0sq (md : mode) : int =
10 op mode_b1sq (md : mode) : int =
11 op mode_b2sq (md : mode) : int =
12 op mode_ln (_ : mode) : int = 8192.
13 op mode_alpha_hint (md : mode) : int =
14 op mode_m (md : mode) : int = mode_l md - 1.
15 op mode_crypto_bytes (md : mode) : int =
16 op mode_polyq_packedbytes (md : mode) : int =
17 op mode_publickeybytes (md : mode) : int =
```

Modules, module types, and clones

Lemmas

```
1 lemma seedbytes_gt0 : 0 < seedbytes by rewrite /seedbytes.
2 lemma crhbytes_gt0 : 0 < crhbytes by rewrite /crhbytes.
3 lemma n_gt0 : 0 < n by rewrite /n.
4 lemma q_gt0 : 0 < q by rewrite /q.
5 lemma dqE : dq = 129026 by rewrite /dq /q.
6 lemma mode_k_gt0 md : 0 < mode_k md.
7 lemma mode_l_gt0 md : 0 < mode_l md.
8 lemma mode_tau_gt0 md : 0 < mode_tau md.
9 lemma mode_tau_le_n md : mode_tau md <= n.
10 lemma mode_b0sq_gt0 md : 0 < mode_b0sq md.
11 lemma mode_b1sq_gt0 md : 0 < mode_b1sq md.
12 lemma mode_b2sq_gt0 md : 0 < mode_b2sq md.
13 lemma mode_ln_gt0 md : 0 < mode_ln md.
14 lemma mode_alpha_hint_gt0 md : 0 < mode_alpha_hint md.
15 lemma mode_m_gt0 md : 0 < mode_m md.
16 lemma mode_crypto_bytes_gt0 md : 0 < mode_crypto_bytes md.
17 lemma mode_polyq_packedbytes_gt0 md : 0 < mode_polyq_packedbytes md.
18 lemma n_le_mode_polyq_packedbytes md : n <= mode_polyq_packedbytes md.
19 lemma mode_publickeybytes_gt0 md : 0 < mode_publickeybytes md.
```

Hoare, probabilistic Hoare, and equivalence proof commands

HAETAEC_FIPS202.ec

Types

```
1 type fips_byte = int.
2 type fips202_state = fips_byte list.
```

Ops and abbreviations

```
1 op fips202_state_bytes : int = 200.
2 op shake256_rate_bytes : int = 136.
3 op shake256_capacity_bytes : int = 64.
4 op shake256_domain_separator : fips_byte = 31.
5 op fips202_final_padding_byte : fips_byte = 128.
6 op fips202_keccak_f1600 (st : fips202_state) : fips202_state =
7 op shake256_absorb_once_short_domain
8 op shake256_absorb_once_short_block_byte
9 op shake256_absorb_once_short_state
10 op shake256_absorb_once
11 op shake256_squeeze (st : fips202_state) (outlen : int) : fips_byte list =
12 op shake256
```

Modules, module types, and clones

Lemmas

```
1 lemma fips202_state_bytesE :
2 lemma shake256_rate_bytesE :
3 lemma shake256_capacity_bytesE :
4 lemma shake256_domain_separatorE :
5 lemma fips202_final_padding_byteE :
6 lemma shake256_rate_capacity_state_bytesE :
7 lemma shake256_absorb_once_short_state_size input inlen :
8 lemma shake256_squeeze_size st outlen :
9 lemma shake256_size input inlen outlen :
```

Hoare, probabilistic Hoare, and equivalence proof commands

HAETAE_Keccak1600.ec

Types

```
1 type keccak_byte = int.
2 type keccak_lane = bool list.
3 type keccak_state = keccak_lane list.
```

Ops and abbreviations

```
1 op keccak_lane_bits : int = 64.
2 op keccak_state_lanes : int = 25.
3 op keccak_state_bytes : int = 200.
4 op keccak_byte_norm (b : keccak_byte) : keccak_byte = b %% 256.
5 op keccak_byte_bit (b : keccak_byte) (i : int) : bool =
6 op keccak_word_norm (w : int) : int = w %% (2 ^ keccak_lane_bits).
7 op keccak_word_bit (w : int) (i : int) : bool =
8 op keccak_lane_zero : keccak_lane = nseq keccak_lane_bits false.
9 op keccak_lane_bit (lane : keccak_lane) (i : int) : bool =
10 op keccak_lane_wf (lane : keccak_lane) : bool =
11 op keccak_lane_xor (a b : keccak_lane) : keccak_lane =
12 op keccak_lane_and (a b : keccak_lane) : keccak_lane =
13 op keccak_lane_not (a : keccak_lane) : keccak_lane =
14 op keccak_lane_rotl (a : keccak_lane) (offset : int) : keccak_lane =
15 op keccak_lane_of_int (x : int) : keccak_lane =
16 op keccak_lane_to_byte (lane : keccak_lane) (byte_index : int) : keccak_byte =
17 op keccak_bytes_to_lane (bs : keccak_byte list) (lane_index : int) :
18 op keccak_lanes_of_bytes (bs : keccak_byte list) : keccak_state =
19 op keccak_bytes_of_lanes (st : keccak_state) : keccak_byte list =
20 op keccak_lane_index (x y : int) : int =
21 op keccak_state_lane (st : keccak_state) (x y : int) : keccak_lane =
22 op keccak_rho_offsets : int list =
23 op keccak_round_constants : int list =
24 op keccak_theta_c (st : keccak_state) (x : int) : keccak_lane =
25 op keccak_theta_d (st : keccak_state) (x : int) : keccak_lane =
26 op keccak_theta (st : keccak_state) : keccak_state =
27 op keccak_rho_pi (st : keccak_state) : keccak_state =
28 op keccak_chi (st : keccak_state) : keccak_state =
29 op keccak_iota (st : keccak_state) (rc : keccak_lane) : keccak_state =
30 op keccak_round (st : keccak_state) (rc : keccak_lane) : keccak_state =
31 op keccak_f1600_lanes (st : keccak_state) : keccak_state =
32 op keccak_f1600_bytes (st : keccak_byte list) : keccak_byte list =
33 op keccak_theta_c_bit_index_value
34 op keccak_theta_d_bit_index_value
```

Modules, module types, and clones

Lemmas

```
1 lemma keccak_lane_bitsE :
2 lemma keccak_state_lanesE :
3 lemma keccak_state_bytesE :
4 lemma keccak_lane_zero_wf :
5 lemma keccak_lane_xor_wf a b :
6 lemma keccak_lane_and_wf a b :
7 lemma keccak_lane_not_wf a :
8 lemma keccak_lane_rotl_wf a offset :
9 lemma keccak_lane_of_int_wf x :
10 lemma keccak_lane_mod_range x :
11 lemma keccak_word_modulus_pos :
12 lemma keccak_word_norm_small w :
13 lemma keccak_pow2_0E :
14 lemma keccak_pow2_1E :
15 lemma keccak_pow2_2E :
16 lemma keccak_pow2_3E :
17 lemma keccak_pow2_4E :
18 lemma keccak_pow2_5E :
19 lemma keccak_pow2_6E :
20 lemma keccak_pow2_7E :
21 lemma keccak_bytes_to_lane_wf bs lane_index :
22 lemma keccak_rho_offsets_size :
23 lemma keccak_round_constants_size :
```

```

24 lemma keccak_lanes_of_bytes_size bs :
25 lemma keccak_lanes_of_bytes_lane_wf bs lane :
26 lemma keccak_lanes_of_bytes_bitE bs lane bit :
27 lemma keccak_lane_xor_bitE a b bit :
28 lemma keccak_lane_and_bitE a b bit :
29 lemma keccak_lane_not_bitE a bit :
30 lemma keccak_lane_rotl_bitE a offset bit :
31 lemma keccak_lane_of_int_bitE x bit :
32 lemma keccak_state_lane_bitE st x y bit :
33 lemma keccak_theta_c_bitE st x bit :
34 lemma keccak_theta_c_bit_indexE st x bit :
35 lemma keccak_theta_d_bitE st x bit :
36 lemma keccak_theta_d_bit_indexE st x bit :
37 lemma keccak_theta_bitE st lane bit :
38 lemma keccak_theta_bit_indexE st lane bit :
39 lemma keccak_rho_pi_bitE st lane bit :
40 lemma keccak_rho_pi_bit_indexE st lane bit :
41 lemma keccak_chi_bitE st lane bit :
42 lemma keccak_chi_bit_indexE st lane bit :
43 lemma keccak_iota_bitE st rc lane bit :
44 lemma keccak_round_bitE st rc lane bit :
45 lemma keccak_bytes_of_lanes_size st :
46 lemma keccak_bytes_of_lanes_byteE st i :
47 lemma keccak_lane_to_byte_bitsE lane byte_index :
48 lemma keccak_lane_to_byte_from_bitsE
49 lemma keccak_lane_to_byte_70E lane byte_index :
50 lemma keccak_f1600_bytes_size st :

```

Hoare, probabilistic Hoare, and equivalence proof commands

Appendix F

Curated EasyCrypt Proof-Object Case Studies

This appendix gives close readings of representative EasyCrypt objects. The archived exhaustive catalog remains in `dissertation/etc/`; the purpose here is different. Each selected code fragment is a proof object with a mathematical meaning in the HAETAE security argument.

F.1 The generic signature interface

```
1  type pkey .
2  type skey .
3  type message .
4  type context .
5  type signature .
6  type ro_query .
7  type ro_output .
8
9  type query = message * context .
10 type signed_query = query * signature .
```

Listing F.1: Signature objects and ROM-query objects in `Sig_ROM.ec`.

Mathematically, this declaration introduces carrier sets

$$\mathcal{PK}, \mathcal{SK}, \mathcal{M}, \mathcal{C}, \Sigma, \mathcal{Q}_{RO}, \mathcal{V}_{RO}.$$

The type `query` is the product $\mathcal{M} \times \mathcal{C}$. The type `signed_query` is $(\mathcal{M} \times \mathcal{C}) \times \Sigma$. The product $\mathcal{M} \times \mathcal{C}$ is the substrate for EUF-CMA freshness: a final forgery must verify and its message/context pair must not have been previously signed. The signed-query product is used for the stronger freshness predicate in the strong-EUF-CMA style game, where the exact message/context/signature triple must not have been returned before.

```
1 module type POracle = {
2   proc get(q : ro_query) : ro_output
3 }.
4
5 module type Scheme (H : POracle) = {
6   proc kg() : pkey * skey
7   proc sign(sk : skey, m : message, ctx : context) :
8     signature
9   proc verify(pk : pkey, m : message, ctx : context,
10    sig : signature) : bool
11 }.
12
13 module type SignOracle = {
14   proc sign(m : message, ctx : context) : signature
15 }.
16
17 module type Adversary (H : POracle, O : SignOracle) = {
18   proc forge(pk : pkey) : message * context * signature
19 }.
```

Listing F.2: Scheme and adversary interfaces in `Sig_ROM.ec`.

The mathematical reading is a family of kernels indexed by an oracle implementation H . A concrete signature scheme is a functor from random oracles to probabilistic procedures. A chosen-message adversary is likewise a module parameterized by the hash oracle and the signing oracle.

F.2 The HAETAE scheme wrapper

The signing procedure in `HAETAE_Scheme.ec` makes the proof-relevant oracle sites explicit.

```

1 coins <$ drandom_coins;
2 ro_y <@ H.get(sampler_expand_query coins);
3 coins <- ro_signing_coins ro_y;
4 pk <- public_key_of_secret haetae_mode sk;
5 ro_y <@ H.get(message_hash_query pk ctx m);
6 mu <- ro_message_hash ro_y;
7 highbits <- commitment_highbits haetae_mode sk m ctx coins
  ;
8 lowbits <- commitment_lowbits haetae_mode sk m ctx coins;
9 ro_y <@ H.get(challenge_hash_query haetae_mode highbits
  lowbits mu);
10 sig <- sign_internal haetae_mode sk m ctx coins;
11 return sig;

```

Listing F.3: Signing wrapper in HAETAE_Scheme.ec.

This code denotes the kernel

$$\text{Sign}^H(sk, m, ctx) : 1 \rightarrow \mathcal{D}(\Sigma).$$

The first ROM query is the sampler-expansion site, the second is the message hash site, and the third is the challenge hash site. The proof depends on these sites being disjoint typed constructors; otherwise ROM programming for the sampler could interfere with ROM programming for the challenge.

The snippet also exposes a model/spec issue. Figure 8 of the specification derives signing randomness deterministically from K and μ . The formal wrapper uses `drandom_coins` and a typed sampler ROM query. The security proof is therefore a theorem about this modeled interface, and the dissertation records the remaining obligation to relate it to the deterministic specification path. The main text now states this relation as the explicit modeling assumption in Assumption 4.1; this appendix should be read as code-level evidence for the EasyCrypt side of that bridge, not as a machine-checked proof of the bridge itself.

F.3 Parameters as mathematical constants

```

1 op seedbytes : int = 32.
2 op crhbytes  : int = 64.
3 op n         : int = 256.

```

```
4 op q : int = 64513.
5 op dq : int = 2 * q.
6
7 type mode = [ Mode2 | Mode3 | Mode5 ].
8
9 op mode_k (md : mode) : int =
10   with md = Mode2 => 2
11   with md = Mode3 => 3
12   with md = Mode5 => 4.
13
14 op mode_l (md : mode) : int =
15   with md = Mode2 => 4
16   with md = Mode3 => 6
17   with md = Mode5 => 7.
18
19 op mode_tau (md : mode) : int =
20   with md = Mode2 => 58
21   with md = Mode3 => 80
22   with md = Mode5 => 128.
```

Listing F.4: Representative parameter declarations in HAETAE_Params.ec.

Mathematically, `mode` indexes three parameterized schemes HAETAE_2 , HAETAE_3 , and HAETAE_5 . The functions `mode_k`, `mode_l`, and `mode_tau` are not runtime computations in the proof argument; they are mode-indexed constants. Lemmas over `haetae_mode` therefore mean theorems for the selected parameter instance.

F.4 The counted-ROM bound

```
1 op counted_rom_programming_loss_term =
2   counted_rom_collision_term +
3   counted_rom_prequery_term +
4   counted_rom_min_entropy_term.
5
6 op haetae_euf_counted_rom_bound =
7   mlwe_hardness_term +
8   bimodal_selftarget_msis_hardness_term +
9   counted_rom_programming_loss_term.
```

Listing F.5: Final counted-ROM bound expression.

The mathematical conclusion of the final fixed-ledger composition lemmas is

$$\mathcal{H}_{\text{hop}} \wedge \mathcal{H}_{\text{hard}} \implies \text{Adv}_{\text{ECModel,ROM}}^{\text{EUF-CMA}}(A) \leq L_{\text{MLWE}} + L_{\text{ModSIS}} + L_{\text{bimodal} \rightarrow \text{MSIS}} + L_{\text{coll}} + L_{\text{pre}} + L_{\text{me}}.$$

The EasyCrypt operation `bimodal_selftarget_msis_hardness_term` expands to `module_sis_hardness_term + bimodal_to_msis_reduction_loss_term`. Thus the final expression is a concrete algebraic object, not an informal bound name.

F.5 Lazy-ROM freshness

```

1 lemma concrete_lazy_rom_sampler_expand_query_fresh_le
2   seed_coins (p : random_coins -> bool) &m :
3   sampler_expand_query seed_coins \notin HAETAE_ROM.FROM.m{m
4   } =>
5   Pr[HAETAE_ROM.FROM.get(sampler_expand_query seed_coins) @
6   &m :
7   p (ro_signing_coins res)] <=
8   mu_drandom_coins p.
```

Listing F.6: Concrete freshness lemma for a sampler ROM point.

This lemma says that if the sampler query is fresh for the lazy ROM table, then asking the random oracle at that point produces signing coins distributed no worse than the ideal `drandom_coins` measure for every predicate p . In measure-theoretic notation, for a fresh point q ,

$$\Pr[p(\text{ro_signing_coins}(H(q)))] \leq \mu_{D_{\text{coins}}}(p).$$

The proof is the local semantic reason that the simulator may treat an unqueried sampler point as fresh randomness.

F.6 Active signing and clean lifting

```

1 lemma
  concrete_budgeted_ro_signing_attempt_o_sign_clean_sampler_fresh
  :
2  hoare [ROMInternalTranscriptBudgetedPaperSimAsNMA
3        (A, ROSigningAttemptPaperSimSampler(HAETAE_RO.
4        FRO),
5        HAETAE_RO.FRO).O.sign :
6  sampler_rom_covered
7  HAETAE_RO.FRO.m
8  ROMInternalTranscriptBudgetedPaperSimAsNMA.
9  adversary_hash_queries
10 ROMInternalTranscriptBudgetedPaperSimAsNMA.
11 sampler_expand_queries /\
12 ! ROMInternalTranscriptBudgetedPaperSimAsNMA.
13 sampler_bad_prequery ==>
  sampler_rom_covered
  HAETAE_RO.FRO.m
  ROMInternalTranscriptBudgetedPaperSimAsNMA.
  adversary_hash_queries
  ROMInternalTranscriptBudgetedPaperSimAsNMA.
  sampler_expand_queries].

```

Listing F.7: Clean sampler-ROM preservation across one active signing call.

This displayed header is the local theorem surface needed for the mathematical reading:

$$\{C\} O_{\text{attempt-sign}} \{C\},$$

where C contains sampler-ROM coverage and the absence of `sampler_bad_prequery`. This Hoare judgment establishes that the clean condition needed for the one-call surface lemma remains available after a signing query. By itself it does not prove that the final NMA theorem path is non-vacuous; the later composition still has to consume this local surface instead of relying on coarse fallback probability bounds.

F.7 Hoare and equivalence judgments as proof objects

A Hoare judgment

$$\text{hoare}[c : P ==> Q]$$

is a partial-correctness statement: every terminating execution of c from a P -state ends in a Q -state. When c is additionally proved lossless, the same support-inclusion fact can be read as probability-one postcondition preservation. An equivalence judgment

$$\text{equiv}[c_1 \sim c_2 : P ==> Q]$$

is a coupling statement between two probabilistic programs. In the HAETAE proof, Hoare judgments maintain invariants such as budget bounds and clean ROM state, while equivalence judgments justify exact game hops. Probability lemmas then turn bad-event bounds and reduction assumptions into the final advantage inequality.

F.8 Why these cases are representative

The full proof contains many declarations, but the main mathematical burden is carried by a small number of object classes: the signature-game interface, the HAETAE scheme wrapper, mode-indexed parameters, lazy-ROM freshness lemmas, active-signing Hoare invariants, sampler/exactness lifting lemmas, and the final bound operations. The archived exhaustive catalog can be used to audit names; the curated cases above explain why the names matter mathematically.

Bibliography

- [Bar+11] Gilles Barthe, Benjamin Grégoire, Sylvain Heraud, and Santiago Zanella Béguelin. “Computer-Aided Security Proofs for the Working Cryptographer”. In: *CRYPTO*. 2011.
- [BR93] Mihir Bellare and Phillip Rogaway. “Random Oracles are Practical: A Paradigm for Designing Efficient Protocols”. In: *Proceedings of CCS*. 1993.
- [Che+26] Jung Hee Cheon et al. *HAETAE Specification, Version v260204*. Local specification document `HAETAE_v260204.pdf`; homepage [https : / / kpqc . cryptolab . co . kr](https://kpqc.cryptolab.co.kr); reference code [https : / / github . com / CryptoLabInc / HAETAE](https://github.com/CryptoLabInc/HAETAE). Local copy generated February 4, 2026; reference-code SHA-256 hash in the specification: 9b69afb55ed96a20d9626b3ea729c291f9e4bfd8d880b740fcb30f6955c4ca72. 2026.
- [Eas26] EasyCrypt Development Team. *EasyCrypt: Computer-Aided Cryptographic Proofs*. <https://www.easycrypt.info/>. 2026.
- [HAE26] HAETAE EasyCrypt Development. *Machine-Checked Provable-Security Proof Surface for HAETAE*. Local EasyCrypt files under `haetae-security/provable-security/`. 2026.
- [Sho04] Victor Shoup. *Sequences of Games: A Tool for Taming Complexity in Security Proofs*. IACR Cryptology ePrint Archive. 2004.
- [Why26] Why3 Development Team. *Why3: A Platform for Deductive Program Verification*. <https://why3.lri.fr/>. 2026.